
Konzeption, Implementierung und Evaluation eines VR-basierten Aufmerksamkeitstrainings für die Forschung unter Einsatz agiler Entwicklungsmethoden

Bachelorarbeit zur Erlangung des akademischen Grades

Bachelor of Science

im Studiengang Code & Context

an der Fakultät für Informatik

der Technischen Hochschule Köln

vorgelegt von:

Florian Krawinkel

Matrikel-Nr.:

[REDACTED]

Adresse:

[REDACTED]

[REDACTED]

[REDACTED]

eingereicht bei:

Prof. Dr. Jonas Schild

Zweitgutachter*in:

Jan Michel Nöring

Köln, 02.06.2026

Kurzfassung

Die Aufmerksamkeit von Ärztinnen und Ärzten spielt eine wichtige Rolle für die Qualität der Arzt-Patienten-Interaktion und kann einen Einfluss auf den Behandlungserfolg haben. Virtual-Reality-Anwendungen bieten die Möglichkeit, entsprechende Verhaltensweisen in einer kontrollierten und gleichzeitig immersiven Umgebung zu trainieren. Im Rahmen eines vorangegangenen Praxisprojekts wurde hierfür bereits ein erster Prototyp eines VR-basierten Aufmerksamkeitstrainings mit dem Namen UPAT (Unseen Patient Attention Training) entwickelt.

Ziel dieser Bachelorarbeit war die konzeptionelle und technische Weiterentwicklung dieses Prototyps, um ihn für den Einsatz in wissenschaftlichen Studien vorzubereiten. Hierzu wurden zunächst bestehende Forschungsarbeiten zu VR-Trainingssimulationen, Aufmerksamkeitstrainings, Eye-Tracking und Visual Delegates analysiert. Ergänzend erfolgte eine Untersuchung vergleichbarer Anwendungen, um Anforderungen an wissenschaftlich nutzbare VR-Aufmerksamkeitstrainings abzuleiten.

Auf Grundlage dieser Erkenntnisse wurden ein Anwendungskonzept sowie ein Softwarearchitekturkonzept entwickelt und anschließend technisch umgesetzt. Zu den wichtigsten Ergebnissen gehören die Einführung einer modularen Visual-Delegate-Architektur, die Forschenden die einfache Erstellung und Konfiguration neuer Visual Delegates ermöglicht, ein Startmenü zur Anpassung von Trainingsparametern sowie die automatisierte Erfassung und Speicherung von Aufmerksamkeitsdaten in CSV-Dateien. Darüber hinaus wurde die Anwendung so gestaltet, dass unterschiedliche Trainingskonfigurationen ohne Änderungen am Quellcode getestet werden können.

Allerdings wird auch deutlich, dass die entwickelte Anwendung bislang lediglich die infrastrukturelle Grundlage für zukünftige Forschungsarbeiten bereitstellt. Aussagen über die tatsächliche Wirksamkeit des Trainings oder dessen Einfluss auf reale Arzt-Patienten-Interaktionen können zum aktuellen Zeitpunkt noch nicht getroffen werden. Hierfür sind zukünftige empirische Studien erforderlich.

Schlagwörter: Virtual Reality, Aufmerksamkeitstraining, Eye-Tracking, Visual Delegates, Arzt-Patienten-Kommunikation, Forschungsanwendungen, Unity, VR-Simulation.

Inhalt

Kurzfassung	I
Tabellenverzeichnis	IV
Abbildungsverzeichnis	V
1 Einleitung	7
2 Theoretischer Hintergrund.....	8
2.1 VR-Trainingssimulationen	9
2.1.1 VR-Trainingssimulationen in verschiedenen Fachbereichen	9
2.1.2 Virtual Reality Trainingssimulation in der Medizin	10
2.1.3 Virtual Reality Aufmerksamkeitstrainings	14
2.1.4 Fazit des Kapitels.....	17
2.2 Visual Delegates	17
2.2.1 Die Herkunft des Begriffs Visual Delegate	18
2.2.2 Ein Framework zur Einordnung von Visual Delegates.....	21
2.2.3 Visual Delegates in Virtual Reality Anwendungen	24
2.2.4 Fazit des Kapitels.....	25
3 Vorarbeiten.....	26
3.1 Erstellung der zugrunde liegenden Anwendung	26
3.1.1 Technische Grundlagen der Entwicklungsumgebung.....	27
3.1.2 Trainingsumgebung und der virtuelle Patient	28
3.1.3 Erfassung und Verarbeitung von Blickbewegungen mittels Eye-Tracking	29
3.1.4 Einsatz und Funktionsweise der verwendeten Visual Delegates	30
3.1.5 Abschließender Entwicklungsstand und Ausblick.....	33
3.2 Anwendungsanalysen ähnlicher Anwendungen	34
3.2.1 Wildcard.....	35
3.2.2 VERA.....	36
3.2.3 Dimensional Search.....	37
3.2.4 Anpassungen für UPAT	38
4 Konzeptionierung	39
4.1 Anwendungskonzept.....	41
4.2 Softwarearchitekturkonzept.....	43
4.3 Beispielhafter Studienablauf.....	45
5 Technische Erweiterung von UPAT.....	46
5.1 Visual Delegates	47
5.1.1 Visual Delegate Basis	47
5.1.2 Visual Delegate Subklassen	49
5.1.3 Specific Visual Delegate	50
5.1.4 Visual Delegate Manager.....	53
5.1.5 Visual Delegate Collections	54
5.2 Startmenü	55

5.2.1 Visual Delegate Collection Dropdown Menü	55
5.2.2 Darstellung der Visual Delegates in der ausgewählten Collection	56
5.2.3 Trainingsparameter-Einstellungen	58
5.2.4 Start-Training Button	59
5.3 Research Data Collector	60
5.4 Trainingstimer	61
5.5 Aufbau innerhalb der Szene	62
5.6 Berücksichtigung der Gestaltungsprobleme aus der Epicsave-Studie	63
6 Diskussion	63
7 Fazit	66
Literaturverzeichnis	68
Anhang	71
Erklärung	72

Tabellenverzeichnis

Tabelle 1: Designfehler und mögliche Lösungsansätze aus der EPICSAVE-Studie nach Misztal et al. (2018)	11
Tabelle 2: Aktualisierte Tabelle der in der EPICSAVE-Studie identifizierten Designprobleme und Lösungsansätze, bereinigt um anwendungsspezifische Aspekte.....	12
Tabelle 3: Einordnung des Visual Delegates aus Abbildung 1 in das VD-Frame	22
Tabelle 4: Einordnung des Visual Delegates aus Abbildung 2 in das VD-Frame	23
Tabelle 5: Genutzter Tachstack innerhalb des Praxisprojekts mit kompatiblen Softwareversionen	27
Tabelle 6: Einordnung des Vignette Visual Delegates in das VD-Frame.....	31
Tabelle 7: Einordnung des Film-Grain Visual Delegates in das VD-Frame	32
Tabelle 8: Einordnung des Grau-Filter Visual Delegates in das VD-Frame.....	32

Abbildungsverzeichnis

Abbildung 1: Links: Abbildung des "Electrical Maze" VR-Aufmerksamkeitstrainings, rechts: Abbildung des „Match the Shape“ VR-Aufmerksamkeitstrainings entnommen aus Tarng et al. (2022)	14
Abbildung 2: Beispiel für die Anwendung von Visual Delegates zur Darstellung von Schmerzen bzw. Verletzungen. Abstrakte rote Flecken am Bildschirmrand visualisieren den Gesundheitszustand des Spielercharakters und stehen symbolisch für Blutverlust (Treyarch/Activision, 2012).....	19
Abbildung 3: Beispiel für ein Visual Delegate zur Darstellung der Flugkurve eines vom Spieler geworfenen Objekts. Eine gepunktete Linie visualisiert die voraussichtliche Flugbahn und zeigt, wo das Objekt aufkommen wird (Crytek, 2019).	20
Abbildung 4: Beispiel für die Anwendung von Visual Delegates zur Darstellung von Emotionen bzw. Gefühlslagen des Spielercharakters. Eine schwarze Vignette verengt das Sichtfeld und erzeugt dadurch eine bedrohliche, angespannte Atmosphäre (Acid Nerve/Devolver Digital, 2021).	21
Abbildung 5: Darstellung des VD-Frames mit allen Kategorien sowie den jeweiligen Ausprägungen in ausgeschriebener Form entnommen aus Misztal & Schild (2022).	22
Abbildung 6: Abbildung des VD-Frames einschließlich der Nutzerpräferenzskala sowie der den einzelnen Kategorieausprägungen zugeordneten User-Experience-Eigenschaften entnommen aus Misztal & Schild (2022).	24
Abbildung 7: Links ist der eingesetzte Alterssimulationsanzug dargestellt, der die Bewegungsfreiheit der Arme durch physische Einschränkungen reduziert. Rechts ist der virtuelle Charakter innerhalb der VR-Umgebung zu sehen, bei dem die Bewegungseinschränkung durch das Visual Delegate in Form des Ghost Effects visualisiert wird entnommen aus Misztal et al. (2022).	25
Abbildung 8: Visualisierung des Raycastkegels, der das Sichtfeld des Nutzers darstellt	29
Abbildung 9: Start-Menü in Dimensional Search. Durch das Plussymbol können neue <i>Blocks</i> dem Training hinzugefügt werden. Innerhalb der einzelnen <i>Blocks</i> können die Trainingsparameter der Anwendung angepasst werden. Innerhalb des Trainings werden alle hier angelegten <i>Blocks</i> hintereinander durchlaufen..	38
Abbildung 10: Flowchart des Anwendungsablauf	42
Abbildung 11: Flowchart der Softwarearchitektur	44
Abbildung 12: Flowchart der Studienvorbereitung	46
Abbildung 13: Abbildung eines Visual Delegate Scriptable-Objekts im Unity Editor..	48

Abbildung 14: Das eingeblendete Kontextmenü in Unity über das sowohl einzelne Visual Delegates, als auch Visual Delegate Collections erstellt werden können.	51
Abbildung 15: Ansicht des Vignette-Visual-Delegates im Unity-Editor.	52
Abbildung 16: Ansicht des Visual Delegate Manager Skripts im Unity Editor.....	53
Abbildung 17: Darstellung der im Praxisprojekt genutzten Visual Delegate Collection, bestehend aus Grau-Filter-, Film-Grain- und Vignette-Visual Delegate, im Unity Editor	54
Abbildung 18: Darstellung des Startmenü's. In diesem können Visual Delegate Collections ausgewählt werden, sowie verschiedene Trainingsparameter eingestellt werden.	55
Abbildung 19: Darstellung des Visual Delegate Collections Dropdown Menü's.	56
Abbildung 20: Darstellung der verschiedenen Visual Delegates innerhalb der ausgewählten Collection.	57
Abbildung 21: Aufbau des Visual Delegate Item Prefabs im Unity-Editor.....	57
Abbildung 22: Darstellung der Schieberegler, mit denen die einzelnen Zeitparameter des Trainings eingestellt werden können.	58
Abbildung 23: Darstellung des unteren Teils des Startmenü's. In diesem lassen sich die Participant-ID, die Trainingszeit, sowie die Art des Durchlaufs einstellen. ...	59
Abbildung 24: Aufbau der Trainingsszene in Unity. Jedes Manager skript hat ein eigenes Objekt in der Szene. Zudem wurde der Szenenaufbau durch Trennobjekte übersichtlich gehalten.	62

1 Einleitung

Eine vertrauensvolle Beziehung zwischen Ärzt:innen und Patient:innen ist ein wesentlicher Faktor im Heilungsprozess (Jagosh et al., 2011). Insbesondere bei Langzeiterkrankungen wie Krebs hängt der Behandlungserfolg unter anderem davon ab, wie ausgeprägt das Vertrauensverhältnis zu behandelnden Ärzt:innen ist (Faller, 2012). Eine stabile Arzt-Patienten-Beziehung ist somit ein zentrales Ziel im medizinischen Kontext.

In der Praxis kommt es jedoch häufig vor, dass sich Patient:innen von ihrem Arzt oder ihrer Ärztin nicht ausreichend verstanden oder wahrgenommen fühlen. Besonders Frauen und jüngere Personen berichten vermehrt davon, in ärztlichen Gesprächen nicht ernst genommen zu werden (Hildenbrand et al., 2022). Ein wesentlicher Auslöser hierfür kann eine als unzureichend empfundene ärztliche Aufmerksamkeit sein, beispielsweise wenn Patienten Anliegen nicht aktiv erfragt oder frühzeitig unterbrochen werden (Singh Ospina et al., 2019). Die gezielte Verbesserung dieser Aufmerksamkeit kann daher einen wichtigen Beitrag zur Optimierung der Arzt-Patienten-Interaktion und damit auch zum Behandlungserfolg leisten. Hierbei stellt sich jedoch die Frage, wie die Aufmerksamkeit von Ärzt:innen in der Interaktion mit Patient:innen gezielt verbessert werden kann.

Virtual Reality (VR) wird seit mehreren Jahren in verschiedenen Branchen zur Schulung von Personal eingesetzt. Unternehmen nutzen VR-Trainings, um Mitarbeitende realitätsnah auf komplexe oder kritische Situationen vorzubereiten (Scorgie et al., 2024). Auch im medizinischen Bereich kommen entsprechende Trainings zum Einsatz (Kyaw et al., 2019). Studien zeigen hierbei unter anderem eine erhöhte Wirksamkeit im Vergleich zu herkömmlichen Trainingsmethoden, sowie ein gesteigertes Engagement der Nutzenden, was insbesondere auf die immersive und sichere Lernumgebung zurückzuführen ist (Lampropoulos et al., 2025; Merchant et al., 2014; Sung et al., 2024).

Darüber hinaus finden VR-Trainings auch in sozialen und therapeutischen Kontexten Anwendung. So konnten beispielsweise bei Grundschüler:innen durch VR-basierte Aufmerksamkeitstrainings positive Effekte erzielt werden (Tarnig et al., 2022). Ebenso ergeben sich Einsatzmöglichkeiten bei der Unterstützung von Personen mit ADHS (Selaskowski et al., 2023). In diesen Anwendungen wird häufig Eye-Tracking-Technologie genutzt, um die Blickbewegungen der Nutzenden zu erfassen und daraus abzuleiten, welchen Objekten im virtuellen Raum Aufmerksamkeit geschenkt wird (Patti et al., 2024).

Auf dieser Grundlage lässt sich ableiten, dass ein speziell für den medizinischen Kontext entwickeltes VR-Aufmerksamkeitstraining dazu beitragen kann, die Aufmerksamkeit von Ärzt:innen in Patientengesprächen zu verbessern und damit indirekt den Heilungsprozess positiv zu beeinflussen.

Im Vorfeld dieser Bachelorarbeit wurde daher im Rahmen eines Praxisprojekts ein erster Prototyp eines solchen VR-basierten Aufmerksamkeitstrainings entwickelt. Ziel der vorliegenden Arbeit ist es, diesen Prototyp weiter auszuarbeiten und sowohl konzeptionell als auch technisch zu optimieren. Ein besonderer Fokus liegt dabei auf der Auslegung der Anwendung für den Forschungsbetrieb. Die entwickelte Anwendung soll so gestaltet werden, dass sie nach Abschluss der Arbeit als Grundlage für wissenschaftliche Studien zur Wirksamkeit von VR-basierten Aufmerksamkeitstrainings genutzt werden kann. Hierzu wird sie gezielt hinsichtlich der Anforderungen an die Durchführung empirischer Untersuchungen erweitert und angepasst. Im folgenden wird die entwickelte Anwendung mit dem Namen UPAT benannt, was für *Unseen-Patient-Attention-Training* steht.

Zudem wird ein Konzept erarbeitet, anhand dessen künftig vergleichbare Anwendungen entwickelt werden können, die den Anforderungen an eine für wissenschaftliche Studien geeignete Software entsprechen. In einer abschließenden Evaluation werden sowohl das entwickelte Konzept als auch die implementierte Anwendung bewertet.

Das gesamte Vorgehen im Rahmen dieser Bachelorarbeit erfolgt unter Anwendung agiler Entwicklungsmethoden. Durch einen kontinuierlichen Austausch mit einem der späteren Nutzenden aus der Forschung wird sichergestellt, dass sowohl die konzeptionellen als auch die technischen Anforderungen fortlaufend überprüft und angepasst werden. Ziel ist es, eine Anwendung zu entwickeln, die optimal auf die Bedürfnisse von Forschenden zugeschnitten ist und die Durchführung wissenschaftlicher Studien bestmöglich unterstützt.

Zugunsten einer besseren und vereinfachten Lesbarkeit dieser Arbeit wird nachfolgend auf Doppelnennungen sowie auf die Verwendung genderneutraler Formulierungen verzichtet. Wo immer eine geschlechtsneutrale Formulierung möglich ist, ohne Verständniseinbußen zu riskieren, wird diese angestrebt. In allen übrigen Fällen wird sich bei der Verwendung von Personenbezeichnungen stets sowohl auf weibliche als auch auf männliche sowie diverse Personen bezogen. Sofern nicht anders ausgeschrieben, soll das generische Maskulinum in dieser Arbeit stets inklusiv verstanden werden.

2 Theoretischer Hintergrund

Bei der Betrachtung des theoretischen Hintergrunds wird zunächst ein grundlegendes Verständnis der Thematik geschaffen und eine klare Definition aller in dieser Arbeit verwendeten fachspezifischen Begriffe gegeben. Hierzu wird der aktuelle Forschungsstand analysiert und in den wissenschaftlichen Kontext dieser Arbeit eingeordnet. Darüber hinaus werden die Zusammenhänge zwischen den einzelnen Komponenten erläutert und deren Relevanz für die Entwicklung und Konzeption eines VR-basierten Aufmerksamkeitstrainings herausgearbeitet.

2.1 VR-Trainingssimulationen

Zu Beginn des theoretischen Hintergrunds wird der aktuelle Forschungsstand zu VR-Trainingssimulationen betrachtet. In zahlreichen Fachbereichen sind entsprechende Anwendungen bereits etabliert. Unternehmen nutzen Virtual-Reality-Trainings, um Mitarbeitende in unterschiedlichen Themenfeldern gezielt zu schulen.

Im Folgenden wird daher der aktuelle Stand der Forschung zu VR-Trainings dargestellt, mit besonderem Fokus auf Anwendungen im medizinischen Kontext. Ziel ist es, eine gemeinsame Wissensgrundlage zu schaffen und die Ausgangsbasis dieser Arbeit klar zu definieren.

2.1.1 VR-Trainingssimulationen in verschiedenen Fachbereichen

In einigen Fachbereichen haben sich VR-Trainingssimulationen bereits seit mehreren Jahren als fester Bestandteil der Schulungs- und Weiterbildungsprozesse etabliert. Internationale Unternehmen wie beispielsweise Walmart (Strivir, 2026) oder United Rentals (Strivir, 2026) setzen bereits VR-Trainings ein und berichten dabei von sehr positiven Ergebnissen. Diese Angaben stammen aus unternehmensinternen Berichten und sind daher kritisch zu betrachten. Sie deuten jedoch darauf hin, dass VR-basierte Trainings bereits in großem Maßstab in der Praxis eingesetzt werden.

Aber auch die aktuelle Forschungslage stützt die Wirksamkeit von VR-Trainings in verschiedenen Anwendungsbereichen. In einer Meta-Analyse aus dem Jahr 2021 untersuchten Howard et al. (2021) insgesamt 184 Studien zur Wirksamkeit von VR-basierten Trainings. Dabei zeigte sich, dass ein etwa 30-minütiges interaktives VR-Training signifikant effektiver ist als klassische Simulator- oder Präsenztrainings ($d = 0.43$), ein kurzes Video bzw. eine kurze Einführung ($d = 0.80$) sowie kein Training ($d = 0.87$).

Ein wichtiger Befund der Analyse ist zudem, dass die Wirksamkeit eines VR-Trainings stark davon abhängt, wie gut es auf die jeweilige Zielaufgabe abgestimmt ist. Generische Trainings erzielen demnach geringere Effekte als kontextspezifisch gestaltete Anwendungen. Wird beispielsweise in einem chirurgischen VR-Training mit realistischen Werkzeugen gearbeitet und der zu behandelnde Patient realitätsnah abgebildet, fällt der Trainingseffekt deutlich höher aus ($d \approx 0.82$) als bei der Verwendung abstrakter Eingabegeräte wie Maus und Tastatur sowie bei der Darstellung lediglich eines Körperausschnitts. Die Ergebnisse unterstreichen somit die Bedeutung einer konsequent auf die Trainingsaufgabe ausgerichteten Gestaltung von VR-Anwendungen.

Darüber hinaus zeigt die Analyse, dass VR-Trainings insbesondere auf Verhaltensänderungen der Nutzer abzielen und diese effektiv beeinflussen können. Daraus lässt sich ableiten, dass VR-Trainings besonders geeignet sind, wenn konkrete Verhaltensweisen gezielt geschult oder verändert werden sollen (Howard et al., 2021).

Weitere Studien, die sich auf einzelne Fachbereiche konzentrieren, belegen ebenfalls den Nutzen von VR-Trainings. So existieren beispielsweise Anwendungen, die es

Mitarbeitenden ermöglichen, den korrekten Aufbau eines Roboterarms in einer virtuellen Umgebung zu erlernen (Mehrfard et al., 2020) oder den Umgang mit komplexen Maschinenprozessen und sicherheitskritischen Aufgaben (Lin et al., 2025; Scorgie et al., 2024). Beide Studien kommen zum Schluss, dass VR-Trainings im Vergleich zu herkömmlichen Trainingsmethoden zu besseren Lernergebnissen führen.

Anhand dieser Studien lässt sich nicht nur belegen, dass VR-Trainings bereits in verschiedensten Fachbereichen weit verbreitet sind, sondern auch, dass diese einen positiven Einfluss auf den Lernerfolg der Nutzer haben können.

2.1.2 Virtual Reality Trainingssimulation in der Medizin

Auch im medizinischen Kontext werden VR-Trainingssimulationen eingesetzt, um Fachpersonal zu schulen und gezielt auf bevorstehende Aufgaben vorzubereiten. Eine aktuelle Studie aus dem Jahr 2025 (Lampropoulos et al., 2025) beschreibt den Einsatz von Virtual-Reality-Trainings als Ergänzung zu herkömmlichen Ausbildungsformaten, um sowohl fachliches Wissen als auch praktische Fähigkeiten zu erweitern. Dabei werden insbesondere der Umgang mit Patienten sowie die Entscheidungsfindung in klinischen Situationen trainiert, ohne die Sicherheit und das Wohlbefinden realer Patienten zu gefährden.

Darüber hinaus konnten in der Studie Verbesserungen in verschiedenen Lernbereichen festgestellt werden. Dazu zählen unter anderem die allgemeine Leistungsfähigkeit, die Entwicklung klinischer Kompetenzen sowie das kritische Denkvermögen. Diese Effekte werden insbesondere auf die immersive und zugleich geschützte Lernumgebung von VR-Anwendungen zurückgeführt.

Ein praktisches Beispiel für ein entsprechendes VR-Training ist EPICSAVE. In dieser Anwendung werden Rettungssanitäter gezielt auf selten auftretende Einsatzsituationen vorbereitet. Dazu zählt beispielsweise der anaphylaktische Schock, der im Einsatz vergleichsweise selten auftritt und daher in der klassischen Ausbildung nur begrenzt realitätsnah trainiert werden kann. Innerhalb von EPICSAVE kann ein solcher Anwendungsfall von mehreren Teilnehmenden gemeinsam in einer virtuellen Umgebung durchgespielt werden. Die zugehörige Studie (Schild et al., 2018) zeigt, dass eine hohe Präsenz in der simulierten Umgebung maßgeblich zu einer verbesserten Lernerfahrung beiträgt. Entsprechend ist es entscheidend, diese Präsenz während der Anwendung möglichst aufrechtzuerhalten, um den Trainingserfolg zu maximieren. Schild et al. (2018) identifizieren in ihrer Arbeit verschiedene Designfehler und mögliche Lösungen, die die wahrgenommene Präsenz negativ beeinflussen können und daher bei der Entwicklung entsprechender Anwendungen vermieden werden sollten. Bei den identifizierten Designfehlern, dargestellt in Tabelle 1, handelt es sich primär um anwendungsspezifische Aspekte. Einzelne dieser Fehler lassen sich jedoch auch auf VR-Trainings im Allgemeinen übertragen. Da die betrachtete EPICSAVE-Anwendung im gleichen medizinischen Fachkontext wie die im Rahmen dieser Bachelorarbeit entwickelte Anwendung entwickelt wurde, werden die beschriebenen Designfehler im weiteren Verlauf berücksichtigt und in die Entwicklung einbezogen.

Area	Issue	Proposed Solutions
Navigation	In general	Use tracked volume for focused scenario
	Cyber sickness	Teleport, visual extensions, slow speed
	De-/re-coupling of sharing virtual/ real space	Visualization of partner avatar location in real volume
Interaction	In general	Virtual hand for items, virtual pointer for menus
	Context-aware inventory	Virtual belt
	Menus bias user decision making	Pure audio interfaces
	Fine interactions	Different input devices or gloves?
	Missing haptics	Visualize haptic experiences
	Lost dropped items	Let loose items temporarily float in air
Visualization	Realism vs. comic visuals	Not highest production value, but medical details need to be adequate
	Multi-user awareness	Full-body tracking and visualization, abstract solutions, ghosts
Multi-user	Emotionalization	Use eye tracking for gaze animation, real-time animations of mimics
	Communication	Support different channels between trainees and with trainers

Tabelle 1: Designfehler und mögliche Lösungsansätze aus der EPICSAVE-Studie nach Misztal et al. (2018)

Werden aus den dargestellten Designfehlern diejenigen ausgeschlossen, die aufgrund der spezifischen Rahmenbedingungen nicht auf die im Rahmen dieser Bachelorarbeit entwickelte Anwendung übertragbar sind, ergibt sich eine reduzierte Übersicht über die relevanten Gestaltungsprobleme. Insbesondere werden dabei sämtliche Designfehler, die sich auf Multi-User-Szenarien beziehen, sowie solche, die den Umgang mit gehaltenen oder verwendeten Objekten betreffen, nicht berücksichtigt.

Auf dieser Grundlage entsteht eine angepasste Tabelle, die ausschließlich die für die vorliegende Anwendung relevanten Designfehler enthält.

Area	Issue	Proposed Solutions
Navigation	In general	Use tracked volume for focused scenario
	Cyber sickness	Teleport, visual extensions, slow speed
Interaction	In general	Virtual hand for items, virtual pointer for menus
	Menus bias user decision making	Pure audio interfaces
	Fine interactions	Different input devices or gloves?
Visualization	Realism vs. comic visuals	Not highest production value, but medical details need to be adequate

Tabelle 2: Aktualisierte Tabelle der in der EPICSAVE-Studie identifizierten Designprobleme und Lösungsansätze, bereinigt um anwendungsspezifische Aspekte

Diese Designfehler sowie insbesondere die dazugehörigen Lösungsansätze dienen als Orientierung für die Konzeption der im Rahmen dieser Bachelorarbeit entwickelten VR-Trainingsanwendung. Insgesamt zeigt die EPICSAVE-Studie, dass eine hohe wahrgenommene Präsenz entscheidend für den Lernerfolg in VR-Trainings ist und dass gezielte Designentscheidungen maßgeblich dazu beitragen können, diese Präsenz zu fördern. Die hier identifizierten Gestaltungsprinzipien bilden somit die Grundlage für die Entwicklung effektiver und nutzerzentrierter VR-Trainingsanwendungen.

Eine weitere Studie untersucht gezielt den Einsatz eines VR/AR-Trainings zur Verbesserung der Kommunikationsfähigkeiten behandelnder Ärzte (Zielke et al., 2017). In diesem Kontext wird ein sogenanntes Natural User Interface (NUI) zur Interaktion mit einem virtuellen Patienten in der Anwendung eingesetzt. Darunter versteht man ein Interface, das es dem Nutzer ermöglicht, mit dem System durch natürliche menschliche Handlungen wie Gesten, Sprache oder Augenbewegungen zu interagieren.

Im Gegensatz dazu stehen klassische Benutzerschnittstellen, etwa in Form von Fernbedienungen oder Controllern, deren Bedienung zunächst erlernt werden muss und daher weniger intuitiv ist. Ein weiteres wesentliches Merkmal von NUIs besteht darin, dass sie weitgehend ohne sichtbare Bedienelemente auskommen und sich so nahtlos in den Nutzungskontext integrieren. Dadurch wird dem Nutzer ein direkter und möglichst unterbrechungsfreier Zugriff auf die Anwendung gewährt (Zielke et al., 2017).

Im Rahmen der Studie wird zudem die Verwendung virtueller Patienten in monitorbasierten Trainings sowie in Virtual- und Augmented-Reality-Anwendungen untersucht. Monitorbasierte Trainings mit virtuellen Patienten zeigen bereits positive Effekte, insbesondere bei der Vermittlung grundlegender Gesprächsführungskompetenzen und medizinischen Fachwissens. Sie ermöglichen Lernenden, Gesprächssituationen zu durchlaufen und erste Erfahrungen im Umgang mit Patienten zu sammeln. Gleichzeitig weisen diese Ansätze Einschränkungen auf, da die

Interaktion häufig textbasiert erfolgt und daher nur begrenzt realitätsnah ist. Insbesondere nonverbale Aspekte der Kommunikation, wie Blickkontakt, Mimik oder Körpersprache, können in solchen Systemen nur unzureichend abgebildet werden. Diese Einschätzung wird auch von den Studierenden gestützt, die an den monitorbasierten Trainings teilgenommen haben. In Befragungen äußerten sie den Wunsch nach einer visuell realistischeren Darstellung virtueller Patienten sowie nach einer stärkeren Integration nonverbaler Kommunikationsmittel in die Interaktion. Dazu zählt insbesondere der gezielte Einsatz von Blickkontakt, der den virtuellen Patienten stärker in das Gespräch einbinden und ein Gefühl aktiver Beteiligung vermitteln kann.

Der Einsatz immersiver Technologien wie VR und AR bietet hier potenzielle Vorteile. Durch die stärkere Einbindung des Nutzers in eine dreidimensionale Umgebung kann die wahrgenommene Präsenz erhöht werden, was wiederum zu einer intensiveren Auseinandersetzung mit der Trainingssituation führt. Zudem ermöglichen VR/AR-Anwendungen eine realitätsnähere Darstellung virtueller Patienten, einschließlich visueller und verhaltensbezogener Merkmale. Dadurch lassen sich auch nonverbale Kommunikationsaspekte sowie empathisches Verhalten gezielter trainieren.

Die Autoren gehen davon aus, dass der Einsatz virtueller Patienten in Kombination mit immersiven Technologien dazu beitragen kann, sowohl die wahrgenommene Präsenz als auch das Training empathischer Kommunikationsfähigkeiten zu verbessern. Diese Annahme stützt sich auf die Erkenntnis, dass eine höhere Immersion mit gesteigerter emotionaler Beteiligung der Lernenden einhergeht, wodurch nonverbale Lernziele realistischer und erfolgreicher trainiert werden können.

Darüber hinaus werden in der Studie Herausforderungen bei der Konzeption und Umsetzung entsprechender VR-Trainings diskutiert. Auffällig ist, dass diese in vielen Teilen mit den zuvor in Tabelle 2 herausgearbeiteten Designproblemen übereinstimmen. Dies deutet darauf hin, dass bestimmte Gestaltungsaspekte unabhängig vom konkreten Anwendungsszenario einen wesentlichen Einfluss auf die Qualität von VR-Trainings haben.

Auch wenn die Studie keine direkten empirischen Nachweise zur Wirksamkeit virtueller Patienten in VR-basierten Trainings liefert, werden dennoch relevante Erkenntnisse aus Nicht-VR-Trainingskontexten herangezogen. Diese zeigen insbesondere, dass virtuelle Patienten bereits in monitorbasierten Lernumgebungen zur Verbesserung der Gesprächsführung, der Empathie und des professionellen Verhaltens beitragen, jedoch in ihrer Realitätsnähe und Interaktivität eingeschränkt sind. Daraus lässt sich ableiten, dass der Einsatz immersiver Technologien das Potenzial hat, diese bestehenden Ansätze durch höhere Präsenz, realistischere Interaktionen und die Integration nonverbaler Kommunikation deutlich zu erweitern. Diese Erkenntnisse bilden somit eine fundierte Grundlage für den Einsatz virtueller Patienten in VR-Anwendungen sowie für die Ableitung entsprechender Gestaltungsanforderungen.

2.1.3 Virtual Reality Aufmerksamkeitstrainings

Nachdem in den vorherigen Kapiteln die Wirksamkeit von Virtual-Reality-Trainings in verschiedenen Fachbereichen dargestellt sowie zentrale Erfolgsfaktoren, insbesondere im Kontext virtueller Patienten, herausgearbeitet wurden, wird in diesem Unterkapitel die Nutzung von VR-Trainings zur gezielten Schulung von Aufmerksamkeit betrachtet.

In einer Studie aus dem Jahr 2022 untersuchten Tarng et al. die Wirksamkeit eines Virtual-Reality-basierten Aufmerksamkeitstrainings bei Grundschulkindern. Dabei greifen die Autoren auf die von Sohlberg und Mateer (1987) definierte Einteilung von Aufmerksamkeit in die fünf Dimensionen *Focused Attention*, *Sustained Attention*, *Selective Attention*, *Alternating Attention* und *Divided Attention* zurück. Diese Klassifikation bildet die klinische Grundlage für die Beschreibung unterschiedlicher Aufmerksamkeitsarten.

Zur Evaluation der Wirksamkeit wurde das von Sohlberg und Mateer entwickelte Attention Process Training (APT) in computerbasierter Form als Referenz herangezogen und mit zwei eigens entwickelten VR-basierten Aufmerksamkeitstrainings, wie in Abbildung 1 dargestellt, verglichen.



Abbildung 1: Links: Abbildung des "Electrical Maze" VR-Aufmerksamkeitstrainings, rechts: Abbildung des „Match the Shape“ VR-Aufmerksamkeitstrainings entnommen aus Tarng et al. (2022)

Die Ergebnisse zeigen differenzierte Effekte in den einzelnen Aufmerksamkeitsdimensionen:

- *Focused Attention*: Signifikante Verbesserung ausschließlich in der VR-Gruppe ($p < 0.001$), keine signifikanten Effekte im APT
- *Sustained Attention*: Signifikante Verbesserungen in beiden Gruppen, jedoch stärkere Effekte in der VR-Gruppe ($p = 0.016$)
- *Selective Attention*: Signifikante Verbesserungen in beiden Gruppen, mit deutlich stärkeren Effekten in der VR-Gruppe ($p < 0.001$)
- *Alternating Attention*: Verbesserungen in beiden Gruppen, ebenfalls mit signifikant besseren Ergebnissen für die VR-Gruppe ($p = 0.016$)
- *Divided Attention*: Verbesserungen in beiden Gruppen, jedoch ohne signifikanten Unterschied zwischen VR und APT ($p = 0.078$)

Insgesamt deuten die Ergebnisse darauf hin, dass VR-basierte Aufmerksamkeitstrainings in den meisten untersuchten Aufmerksamkeitsdimensionen eine höhere Effektivität aufweisen als klassische Trainingsansätze. Eine Ausnahme stellt die geteilte Aufmerksamkeit dar, für die kein signifikanter Vorteil durch den Einsatz von VR nachgewiesen werden konnte.

Für die im Rahmen dieser Bachelorarbeit entwickelte Anwendung sind insbesondere die Aufmerksamkeitsdimensionen *Alternating Attention*, also die Fähigkeit, flexibel zwischen verschiedenen Aufmerksamkeitszielen zu wechseln, sowie *Focused Attention*, also die gezielte Fokussierung auf einen einzelnen Reiz oder Gesprächspartner, von besonderer Relevanz. Die in der Studie festgestellten signifikanten Verbesserungen in diesen Bereichen sind daher für den vorliegenden Anwendungskontext von besonderem Interesse.

Darüber hinaus wurde im Rahmen der Untersuchung festgestellt, dass die Teilnehmenden bei der Nutzung von VR-basierten Aufmerksamkeitstrainings eine signifikant höhere Motivation und Akzeptanz aufwiesen als in der APT-Kontrollgruppe. So gaben Teilnehmende der VR-Gruppe eine deutlich höhere Bereitschaft an, das Training regelmäßig durchzuführen („I want to do attention training every day“, $p < .001$), sowie eine stärkere generelle Präferenz für das Training („I like to do attention training“, $p < .001$). Zudem wurde der Trainingsinhalt als signifikant interessanter bewertet („I think the content of attention training is interesting“, $p = .005$).

Diese Ergebnisse lassen sich unter anderem darauf zurückführen, dass VR-Anwendungen als immersiver, interaktiver und insgesamt ansprechender wahrgenommen werden, während das computerbasierte APT eher monoton erscheint. Allerdings ist zu berücksichtigen, dass die untersuchten VR-Anwendungen spielerisch gestaltet waren und die befragte Stichprobe aus Grundschulkindern bestand. Eine direkte Übertragbarkeit der Ergebnisse auf erwachsene Zielgruppen ist daher nur eingeschränkt gegeben und wird daher im Rahmen dieser Arbeit entsprechend vorsichtig eingeordnet.

Die Anwendung VERA (Delvigne, 2020) ist ein VR-basiertes Aufmerksamkeitstraining, das insbesondere auf Kinder mit ADHS ausgerichtet ist. Im zugehörigen Paper (Delvigne et al., 2020) liegt der Fokus jedoch weniger auf der unmittelbaren Verbesserung der kognitiven Leistungsfähigkeit der Teilnehmenden, sondern vielmehr auf der präzisen Erfassung und Modellierung von Aufmerksamkeitsprozessen als Grundlage für zukünftige Trainingsansätze.

Zur Datenerhebung wurden mehrere VR-Szenarien entwickelt, in denen die Probanden unterschiedliche Aufgaben zur *Selective Attention* und zur *Focused Attention* bearbeiteten. Während der Durchführung wurden kontinuierlich physiologische Daten erfasst, darunter EEG-, Eye-Tracking- und Kopfbewegungsdaten.

Eine wichtige Rolle spielt dabei Eye-Tracking, da es die direkte Erfassung von Aufmerksamkeitsprozessen anhand der Blickrichtung und der Fixationsdauer auf

relevante Zielreize ermöglicht. Darüber hinaus ermöglicht es eine zeitliche Analyse von Aufmerksamkeitsverläufen, indem beispielsweise Veränderungen der Dauer der visuellen Fokussierung oder die Zeit bis zur erneuten Fokussierung nach einem Aufmerksamkeitsverlust (Refocus Time) ausgewertet werden können.

Die Ergebnisse der Studie deuten darauf hin, dass Eye-Tracking eine wesentliche Grundlage für die Analyse von Aufmerksamkeit in VR-Umgebungen darstellt. Durch die direkte Messung visueller Fokussierung lassen sich sowohl kurzfristige Aufmerksamkeitsreaktionen als auch längerfristige Muster erfassen. Für die vorliegende Arbeit ist dies insbesondere im Hinblick auf die Erfassung von *Focused Attention* relevant. Eye-Tracking bietet damit eine geeignete Grundlage für die Entwicklung interaktiver VR-Anwendungen, in denen Aufmerksamkeit nicht nur analysiert, sondern perspektivisch auch gezielt beeinflusst und trainiert werden kann.

Da das VERA-Aufmerksamkeitstraining in seiner Zielsetzung und Ausgestaltung große Ähnlichkeiten mit der im Rahmen dieser Arbeit entwickelten Anwendung aufweist, ist es für die eigene Konzeption von besonderer Relevanz. Aus diesem Grund erfolgt in Kapitel 3.2 eine detaillierte Analyse der Anwendung VERA, in der insbesondere die getroffenen Designentscheidungen näher betrachtet werden.

In einer weiteren Studie untersuchen Patti et al. (2024) den Einsatz der VR-Anwendung Wildcard (Buttiglione, 2022). Diese soll zur Verbesserung der Aufmerksamkeit bei Personen mit neurologischen Entwicklungsstörungen wie ADHS oder Autismus beitragen. Die Untersuchung konzentriert sich auf die Wirksamkeit eines VR-basierten Aufmerksamkeitstrainings, das Eye-Tracking einsetzt. Zudem wird die Benutzerfreundlichkeit mithilfe eines SUS-Fragebogens ermittelt.

Hierfür wurde die Anwendung gegenüber einer früheren Version technologisch erweitert und nutzt nun die VR-Brille Pico Neo 3 Pro Eye mit integriertem Eye-Tracking. Dadurch erfolgt die Interaktion nicht mehr über Kopfbewegungen, sondern direkt über die Blickrichtung der Nutzenden. Dies ermöglicht eine präzisere Erfassung von Aufmerksamkeitsprozessen sowie eine genauere Steuerung der Anwendung.

Die Trainingsaktivitäten umfassen verschiedene Aufgaben, bei denen visuelle Ziele gezielt ausgewählt oder über einen bestimmten Zeitraum hinweg fokussiert werden müssen. Zur Evaluation der Trainingseffekte wurde eine vierwöchige Studie durchgeführt, in deren Rahmen zu Beginn und am Ende der Untersuchungsphase der Trail Making Test (TMT) als etablierter neuropsychologischer Test zur Messung visueller Aufmerksamkeit eingesetzt wurde.

Die Ergebnisse zeigen eine Verbesserung der Aufmerksamkeitsleistung: Die durchschnittliche Bearbeitungszeit im TMT konnte von 183,22 auf 161,04 Sekunden reduziert werden. Zusätzlich zeigte sich während der Trainingssitzungen eine kontinuierliche Reduktion der Bearbeitungszeiten, was auf einen Lerneffekt hindeutet. Dieser ist jedoch mit Vorsicht zu interpretieren, da er teilweise auch auf eine zunehmende Vertrautheit mit den Aufgaben zurückzuführen sein kann.

Ein großer Beitrag der Studie besteht in der Integration von Eye-Tracking als wesentliches Interaktionselement. Durch die präzise Erfassung der Blickrichtung lässt sich nicht nur bestimmen, worauf sich die Aufmerksamkeit richtet, sondern auch, wie stabil und zielgerichtet diese ist. Gleichzeitig ermöglicht diese Form der Interaktion eine besonders intuitive Nutzung, da Objekte durch visuelle Fixierung direkt ausgewählt werden können.

Ergänzend wurde die Usability der Anwendung evaluiert. Mit einem durchschnittlichen SUS-Score von 80,26 weist die Anwendung eine hohe Benutzerfreundlichkeit und Akzeptanz auf.

Insgesamt verdeutlichen die Ergebnisse, dass VR-basierte Aufmerksamkeitstrainings in Kombination mit Eye-Tracking einen wirksamen Ansatz zur Verbesserung der visuellen Aufmerksamkeit darstellen. Insbesondere die enge Kopplung von Blickverhalten und Interaktion bietet Potenzial, Aufmerksamkeit nicht nur präzise zu messen, sondern auch gezielt zu trainieren. Zudem lässt die Usability-Evaluation darauf schließen, dass eine intuitive und technisch gut umgesetzte Interaktion maßgeblich zur Akzeptanz und damit auch zur regelmäßigen Nutzung entsprechender Trainingsanwendungen beiträgt.

2.1.4 Fazit des Kapitels

Die bisherigen Ausführungen verdeutlichen, dass VR-Trainings in verschiedenen Fachbereichen einen wirksamen Einsatz finden und insbesondere im medizinischen Kontext ein bedeutendes Potenzial aufweisen. Während virtuelle Patienten bereits in monitorbasierten Schulungen zur Verbesserung der Gesprächsführung und des Fachwissens beitragen, sind sie in ihrer Realitätsnähe und insbesondere bei der Abbildung nonverbaler Kommunikation eingeschränkt. VR- und AR-Technologien können diese Defizite durch erhöhte Immersion, realistischere Darstellungen und natürlichere Interaktionen ausgleichen und damit insbesondere das Training empathischer Kompetenzen fördern.

Auch im Bereich der Aufmerksamkeitstrainings zeigen VR-basierte Ansätze Vorteile gegenüber klassischen Methoden, insbesondere hinsichtlich der Verbesserung fokussierter und wechselnder Aufmerksamkeit sowie der Motivation und Akzeptanz. Ergänzend ermöglicht der Einsatz von Eye-Tracking eine präzise Erfassung und zugleich eine intuitive Interaktion anhand des Blickverhaltens.

Insgesamt verdeutlichen die Studien, dass immersive Technologien neue Möglichkeiten für realitätsnahe und effektive Trainings eröffnen. Diese bilden die Grundlage für den Einsatz von Visual Delegates, die dazu dienen, nicht direkt wahrnehmbare Zustände in solchen Umgebungen visuell zu vermitteln, und werden im Folgenden näher erläutert.

2.2 Visual Delegates

Zunächst sollen die Herkunft und die grundlegende Definition des Begriffs der Visual Delegates erläutert werden. Im weiteren Verlauf wird der aktuelle Forschungsstand

dargestellt und ein Framework zur systematischen Einordnung von Visual Delegates vorgestellt, das im weiteren Verlauf dieser Arbeit wiederholt herangezogen wird.

Abschließend wird insbesondere auf den Einsatz von Visual Delegates in Virtual-Reality-Anwendungen eingegangen, wobei die Unterschiede zu klassischen Simulationen und konventionellen Anwendungen herausgearbeitet werden.

2.2.1 Die Herkunft des Begriffs Visual Delegate

In ihrem 2020 erschienenen Paper definieren Misztal et al. Visual Delegates als visuelle Effekte, die dazu dienen, komplexe Sinneseindrücke und emotionale Zustände zu substituieren. Solche Visual Delegates finden sich insbesondere in Videospielen, da diese ein hohes Maß an Immersion anstreben, jedoch in ihrer Fähigkeit eingeschränkt sind, reale Sinneseindrücke oder Emotionen direkt zu vermitteln.

Ein typisches Beispiel ist die Darstellung von Verletzungen oder Schmerzen mithilfe von Blutflecken auf dem Bildschirm. Dieser Ansatz wird vor allem in Spielen aus der Ego-Perspektive, etwa in First-Person-Shootern, eingesetzt. Da der Spieler physische Empfindungen wie Schmerz nicht tatsächlich wahrnehmen kann, werden diese durch visuelle Reize ersetzt, um dennoch eine entsprechende Wahrnehmung zu erzeugen.

In Abbildung 1 ist ein Beispiel für diesen Anwendungsfall aus dem First-Person-Shooter *Call of Duty: Black Ops 2* dargestellt. Wird der Spielercharakter von gegnerischen Schüssen getroffen, erscheinen rote Flecken am Bildschirmrand. Mit zunehmendem Schaden verstärken sich diese Effekte: Bereits vorhandene Flecken werden dunkler, und weitere kommen hinzu. Auf diese Weise wird dem Spieler visuell signalisiert, dass der Charakter verletzt ist und sich in einem kritischen Zustand befindet.

Bleibt der Spieler hingegen über einen gewissen Zeitraum unbeschadet, verringert sich die Intensität der visuellen Effekte schrittweise, bis sie vollständig verschwinden. Dadurch wird dem Spieler vermittelt, dass sich der Charakter erholt hat und sich wieder in einem stabilen Zustand befindet. Dieses Beispiel verdeutlicht, wie physische Zustände, die nicht direkt wahrnehmbar sind, mithilfe von Visual Delegates effektiv visuell kommuniziert werden können.



Abbildung 2: Beispiel für die Anwendung von Visual Delegates zur Darstellung von Schmerzen bzw. Verletzungen. Abstrakte rote Flecken am Bildschirmrand visualisieren den Gesundheitszustand des Spielercharakters und stehen symbolisch für Blutverlust (Treyarch/Activision, 2012).

Neben Schmerz lassen sich auch andere Sinneseindrücke wie etwa die Flugkurve eines geworfenen Objekts durch visuelle Effekte vermitteln. In Abbildung 2 ist ein entsprechendes Beispiel aus dem Videospiel *Hunt: Showdown 1896* dargestellt. Möchte der Spieler ein Objekt werfen, wird eine gepunktete Linie direkt in der Spielwelt visualisiert, die die voraussichtliche Flugkurve des Wurfobjekts darstellt. Auf diese Weise kann der Spieler die Bewegung des Objekts besser einschätzen, obwohl reale Einflussfaktoren wie Gewicht oder eigene Kraft nicht physisch wahrnehmbar sind.

In diesem Fall wird ein Visual Delegate eingesetzt, um ein komplexes physikalisches Verhalten visuell darzustellen und es für den Nutzer intuitiv verständlich zu machen. Dieses Visual Delegate ersetzt somit direkt mehrere sensorische Informationen.



Abbildung 3: Beispiel für ein Visual Delegate zur Darstellung der Flugkurve eines vom Spieler geworfenen Objekts. Eine gepunktete Linie visualisiert die voraussichtliche Flugbahn und zeigt, wo das Objekt aufkommen wird (Crytek, 2019).

Auch emotionale Zustände oder die Gefühlslage eines virtuellen Charakters lassen sich auf diese Weise für den Spieler verständlich machen. In Abbildung 3 wird hierzu ein Visual Delegate in Form einer Vignette eingesetzt, um die emotionale Situation des Spielercharakters zu vermitteln.

In der dargestellten Szene befindet sich der Spieler in einer Gesprächssituation, in der ihm ein anderer Charakter mitteilt, dass er zu diesem Zeitpunkt im Spiel sterblich ist. Um die daraus resultierende Gefühlslage, insbesondere Bedrohung und Anspannung, visuell zu vermitteln, wird ein Vignetteneffekt eingesetzt, der das Sichtfeld des Spielers zunehmend einengt. Dadurch entsteht eine intensivere und bedrückendere Wahrnehmung der Situation.



Abbildung 4: Beispiel für die Anwendung von Visual Delegates zur Darstellung von Emotionen bzw. Gefühlslagen des Spielercharakters. Eine schwarze Vignette verengt das Sichtfeld und erzeugt dadurch eine bedrohliche, angespannte Atmosphäre (Acid Nerve/Devolver Digital, 2021).

Im Gegensatz zu vorherigen Beispielen wird hier kein physischer Sinneseindruck ersetzt, sondern ein emotionaler Zustand visuell vermittelt. Dieses Beispiel verdeutlicht, dass Visual Delegates nicht nur zur Darstellung sensorischer Informationen, sondern auch zur gezielten Kommunikation von Emotionen eingesetzt werden können.

Zusammenfassend lässt sich festhalten, dass Visual Delegates eine essenzielle Rolle bei der Gestaltung immersiver Anwendungen spielen, indem sie nicht direkt erfahrbare sensorische oder emotionale Zustände in visuell wahrnehmbare Reize übersetzen. Ursprünglich wurden diese vor allem in Videospielen eingesetzt, dienen aber auch dazu, physische Einschränkungen, komplexe Prozesse oder emotionale Zustände verständlich und intuitiv zu vermitteln.

2.2.2 Ein Framework zur Einordnung von Visual Delegates

In einem anschließenden Paper widmen sich Misztal und Schild (2022) gezielt zur Einordnung von Visual Delegates. Hierzu wurde das sogenannte Visual Delegate Framework (VD-Frame) entwickelt, das eine strukturierte Klassifizierung und Analyse dieser visuellen Effekte ermöglicht. Mithilfe dieses Frameworks können Visual Delegates bereits während des Entwicklungsprozesses bewertet und eingeordnet werden. Ziel ist es, frühzeitig abschätzen zu können, welche emotionalen und kognitiven Auswirkungen ein Visual Delegate auf den Nutzer haben kann, ohne dafür zunächst aufwendige Nutzerstudien durchführen zu müssen. Dieses Framework wird im Folgenden detailliert beschrieben, da es eine äußerst wichtige Rolle für die im Rahmen dieser Bachelorarbeit entwickelte Anwendung spielt.

Grundlage für die Entwicklung des VD-Frames bildet die Analyse verschiedenster Visual Delegates. Auf Basis dieser Untersuchung wurden die drei Kategorien *Depth*,

Abstraction und *Control* identifiziert, entlang derer Visual Delegates beschrieben und klassifiziert werden können. Dazu zählen unter anderem die Art der dargestellten Information, die visuelle Ausprägung sowie die beabsichtigte Wirkung auf den Nutzer.

VD Category		VD Property		
Depth	MENU (in-game menu)	FIL (filter) REF (referenced)	CHAR (character)	ENV (virtual environment)
Abstraction	DESC (descriptive)	SYM (symbolic)		NAT (natural)
Control	NOCO (no control)	ACT (active)		MOD (modifiable)

Abbildung 5: Darstellung des VD-Frames mit allen Kategorien sowie den jeweiligen Ausprägungen in ausgeschriebener Form entnommen aus Misztal & Schild (2022).

Innerhalb der drei Hauptkategorien des VD-Frames existieren jeweils spezifische Ausprägungen, die die Eigenschaften eines Visual Delegates näher beschreiben. Die Kategorie *Depth* umfasst die Ausprägungen *Menu*, *Referenced*, *Filter*, *Character* und *Environment*. Diese beschreiben, auf welcher Ebene innerhalb der Anwendung ein Visual Delegate platziert ist. Die Kategorie *Abstraction* wird in die drei Ausprägungen *Descriptive*, *Symbolic* und *Natural* unterteilt. Diese geben an, wie abstrakt oder realitätsnah die visuelle Darstellung erfolgt. Die Kategorie *Control* umfasst die Ausprägungen *No Control*, *Active* und *Modifiable*. Sie beschreibt, in welchem Maß der Nutzer Einfluss auf das Visual Delegate nehmen kann.

Auch die zuvor genannten Beispiele für Visual Delegates lassen sich mithilfe des VD-Frames einordnen. Durch die Anwendung des Frameworks wird deutlich, auf welchen Ebenen die jeweiligen Visual Delegates wirken und welche Art von Informationen sie vermitteln.

Wird Abbildung 1 erneut betrachtet, lässt sich das dort dargestellte Visual Delegate im VD-Frame einordnen:

Depth	Abstraction	Control
FIL	SYM	NOCO

Tabelle 3: Einordnung des Visual Delegates aus Abbildung 1 in das VD-Frame

Da die roten Flecken nicht direkt in die virtuelle Umgebung integriert sind, sondern als Overlay direkt auf dem Sichtfeld des Nutzers erscheinen, sind sie innerhalb der Kategorie *Depth* der *Filter* Ausprägung zuzuordnen.

In Bezug auf die Kategorie *Abstraction* lässt sich das Visual Delegate als *Symbolic* einordnen. Dies liegt daran, dass die dargestellten roten Flecken eine abstrahierte Darstellung von Blut wiedergeben und daher nicht als realistische Abbildung wahrgenommen werden. Bei einer detaillierteren und realitätsnäheren Darstellung wäre hingegen auch eine Einordnung als *Natural* denkbar.

Innerhalb der Kategorie *Control* fällt das Visual Delegate in die Ausprägung *No Control*, da der Spieler keinen direkten Einfluss auf das Verschwinden der Effekte hat. Der

Heilungsprozess erfolgt im gezeigten Spiel automatisch im festgelegten Zeitverlauf und ist nicht aktiv steuerbar.

Auch das in Abbildung 2 dargestellte Visual Delegate lässt sich entsprechend innerhalb des VD-Frames klassifizieren:

Depth	Abstraction	Control
REF	DESC	ACT/MOD

Tabelle 4: Einordnung des Visual Delegates aus Abbildung 2 in das VD-Frame

In der Kategorie *Depth* lässt sich dieses Visual Delegate der Ausprägung *Referenced* zuordnen. Diese Ausprägung stellt eine Zwischenebene zwischen *Menu* und *Environment* dar. Die dargestellte Wurfkurve befindet sich zwar direkt in der virtuellen Spielwelt, ist jedoch eindeutig an ein spezifisches Objekt gebunden. Gleichzeitig handelt es sich nicht um ein eigenständiges physikalisches Objekt, sondern um eine visuelle Referenz, die das Verhalten eines solchen Objekts vorwegnimmt.

In der Kategorie *Abstraction* ist das Visual Delegate als *Descriptive* einzuordnen, da es eine konkrete Eigenschaft – in diesem Fall die Flugbahn eines Objekts – visuell darstellt. Im Gegensatz zu einer Einordnung als *Symbolic* basiert die Darstellung nicht auf einer abstrakten oder metaphorischen Repräsentation eines real existierenden Objekts, sondern vermittelt unmittelbar nachvollziehbare Informationen über das Verhalten des geworfenen Objekts.

In der Kategorie *Control* zeigt sich eine Besonderheit bei der Einordnung dieses Visual Delegates. Auf den ersten Blick lässt es sich eindeutig der Ausprägung *Active* zuordnen, da die dargestellte Wurfkurve vom Spieler unmittelbar beeinflusst werden kann, beispielsweise durch die Ausrichtung der Kamera oder die Wahl des Wurfwinkels. Darüber hinaus kann das Visual Delegate jedoch auch der Ausprägung *Modifiable* zugeordnet werden. Dies liegt daran, dass sich die zugrunde liegenden Parameter im Spielverlauf verändern lassen, etwa durch Fähigkeiten oder Talente, die das Wurfverhalten des Charakters dauerhaft beeinflussen. Somit ist das Visual Delegate nicht nur situativ steuerbar, sondern auch langfristig durch Spielmechaniken modifizierbar.

Im Rahmen des Papers von Misztal und Schild wurden zudem verschiedene wissenschaftliche Arbeiten analysiert und Nutzertests mit erfahrenen Spielern durchgeführt, um den Einfluss unterschiedlicher Visual Delegates auf die User Experience und das Nutzerverhalten zu untersuchen. Dabei wurden die Präferenzen der befragten Spieler erfasst und mittels einer Sternskala dargestellt. Zudem wurde festgestellt, dass die Art der Visual Delegates einen erheblichen Einfluss auf das Spielerverhalten haben kann. Anhand der Spielerbefragungen konnten den einzelnen Ausprägungen in den Kategorien bestimmte Eigenschaften zugeschrieben werden. Mithilfe dieser Eigenschaften lassen sich die Effekte der Visual Delegates auf die Nutzer abschätzen, ohne zuvor aufwendige Nutzerstudien durchführen zu müssen.

VD Property	Preferences	Associated PUXs and Associated Properties
Depth		
MENU	★★★★☆	attention, scattering, visual load
REF	★★★★☆	flatness, overpowered, player-world bond, visual load
FIL	★★★★★	attention, immersion, interactivity, player-character bond, urgency
CHAR	★★★★☆	attention, interactivity, player-character bond
ENV	★★★★☆	attention, immersion, interactivity, learnable (interpretation), player-character bond
Abstraction		
DESC	★★☆☆☆	accuracy, comprehensible, informative, prescriptive, refusal, visual load
SYM	★★☆☆☆	accuracy, efficiency, ignorable, incomprehensibility, informative, interactivity, visual load
NAT	★★★★☆	attention, difficult to understand, immersion, player-character bond
Control		
NOCO	★★☆☆☆	commonness
ACT	★★★★☆	interactivity, player-character bond
MOD	★★☆☆☆	complicated, distracting, interesting, player-character bond

Abbildung 6: Abbildung des VD-Frames einschließlich der Nutzerpräferenzskala sowie der den einzelnen Kategorieausprägungen zugeordneten User-Experience-Eigenschaften entnommen aus Misztal & Schild (2022).

In Abbildung 5 ist eine erweiterte Darstellung des VD-Frames zu sehen, in der die zuvor beschriebenen Eigenschaften der einzelnen Kategorieausprägungen ergänzt wurden. Für jede Ausprägung werden sowohl die Auswirkungen auf die User Experience als auch die ermittelte Nutzerpräferenz dargestellt.

Das VD-Frame stellt somit ein wichtiges Werkzeug dar, um Visual Delegates nicht nur intuitiv, sondern auch methodisch fundiert zu entwickeln und zu bewerten. Insbesondere im Kontext dieser Arbeit ermöglicht das Framework eine strukturierte Analyse der eingesetzten Visual Delegates sowie eine gezielte Erweiterung um weitere Effekte, die bestimmte Kategorien oder Wirkungsweisen abdecken. Im weiteren Verlauf der Arbeit wird das VD-Frame daher herangezogen, um die in der Anwendung eingesetzten Visual Delegates einzuordnen.

2.2.3 Visual Delegates in Virtual Reality Anwendungen

Auch in VR-Anwendungen werden Visual Delegates gezielt eingesetzt, um die ohnehin hohe Immersion weiter zu steigern. Ein anschauliches Beispiel hierfür liefert eine Studie von Misztal et al. (2022), in der altersbedingte Bewegungseinschränkungen mithilfe eines Visual Delegates simuliert wurden, um gezielt Einfluss auf das Nutzerverhalten zu nehmen.

Im Rahmen der Untersuchung sollten die Teilnehmenden in einem virtuellen Supermarkt Produkte aus Regalen unterschiedlicher Höhe greifen. Dabei wurde ein physischer Alterssimulationsanzug, der die Armbewegungen mechanisch einschränkt, mit einem visuellen Ansatz verglichen.

Das eingesetzte Visual Delegate, ein sogenannter *Ghost Effect*, visualisiert die tatsächliche Armhaltung der Nutzenden als leicht transparente Darstellung. Gleichzeitig wird eine eingeschränkte Armposition normal dargestellt und bleibt auch dann sichtbar, wenn sich der reale Arm bereits außerhalb dieses Bereichs befindet. Auf diese Weise entsteht eine visuelle Diskrepanz zwischen tatsächlicher und dargestellter Bewegung,

die eine wahrgenommene Einschränkung der Bewegung erzeugt, ohne dass eine physische Begrenzung vorliegt.



Abbildung 7: Links ist der eingesetzte Alterssimulationsanzug dargestellt, der die Bewegungsfreiheit der Arme durch physische Einschränkungen reduziert. Rechts ist der virtuelle Charakter innerhalb der VR-Umgebung zu sehen, bei dem die Bewegungseinschränkung durch das Visual Delegate in Form des Ghost Effects visualisiert wird entnommen aus Misztal et al. (2022).

Die Ergebnisse der Studie zeigen, dass das genutzte Visual Delegate eine signifikant stärkere wahrgenommene Bewegungseinschränkung erzeugt ($p = .001$; $M = 4.30$ auf einer Skala von 0 = keine Einschränkung bis 6 = starke Einschränkung) als der physische Alterssimulationsanzug ($M = 3.10$). Dies weist darauf hin, dass Visual Delegates auch in VR in der Lage sind, sensorische Eindrücke nicht nur zu substituieren, sondern sie teilweise sogar intensiver erfahrbar zu machen als reale physische Einschränkungen. Dies wird durch die Leistungsdaten untermauert, die eine signifikant längere Zeit für die Objektauswahl unter Verwendung des Visual Delegates zeigen, was auf eine tatsächlich erschwerte Interaktion hindeutet.

Trotz dieses starken Eingriffs in die Wahrnehmung blieb die Präsenz der Teilnehmenden in der virtuellen Umgebung erhalten. Zudem wurden die virtuellen Arme weiterhin als Teil des eigenen Körpers wahrgenommen. Auch die Usability wurde mit einem SUS-Wert von 81.50 ($SD = 8.60$) als hoch bewertet, während nur geringe Simulator Sickness auftrat ($SSQ: M = 16.73$).

Insgesamt verdeutlicht die Studie, dass Visual Delegates eine effektive und zugleich immersive Möglichkeit bieten, Sinneseindrücke in VR-Trainings zu vermitteln. Dabei kann die wahrgenommene Präsenz aufrechterhalten werden, während bestimmte Empfindungen sogar stärker als durch physische Mittel erfahrbar gemacht werden können.

2.2.4 Fazit des Kapitels

Insgesamt verdeutlichen die beschriebenen Ergebnisse, dass Visual Delegates ein wirkungsvolles Gestaltungsmittel für VR-Anwendungen darstellen. Sie ermöglichen es, Wahrnehmung gezielt zu beeinflussen, Interaktionen zu steuern und komplexe Zustände verständlich zu vermitteln, ohne dabei die Immersion negativ zu beeinträchtigen. Damit bilden sie eine wichtige Grundlage für die Entwicklung effektiver und immersiver VR-Trainingsanwendungen und werden daher auch in der im Rahmen dieser Arbeit

entwickelten Anwendung eingesetzt, um Emotionen für die Nutzenden visuell erfahrbar zu machen.

In den folgenden Kapiteln wird die konkrete Nutzung von Visual Delegates in der entwickelten Anwendung weiter vertieft. Dabei wird sowohl auf deren Implementierung als auch auf die Analyse vergleichbarer Anwendungen eingegangen, um verschiedene Gestaltungsprinzipien und deren Einfluss auf die Nutzererfahrung herauszuarbeiten.

3 Vorarbeiten

Im Vorfeld dieser Bachelorarbeit wurde im Rahmen eines Praxisprojekts das zugrunde liegende VR-Aufmerksamkeitstraining entwickelt. Dieses wird im Rahmen der vorliegenden Arbeit weiter überarbeitet und gezielt auf eine zukünftige Nutzung in der Forschung ausgerichtet. Dazu wird in diesem Abschnitt der Stand der zuvor entwickelten Anwendung näher betrachtet, um die Ausgangsbasis für die weiteren Entwicklungs- und Optimierungsschritte transparent darzustellen.

Als Grundlage für die weiteren technischen Verbesserungen, sowie die Ausrichtung auf eine Nutzung in der Forschung, werden zudem vergleichbare VR-basierte Aufmerksamkeitstrainings analysiert und mögliche Erfolgskriterien für derartige Anwendungen herausgearbeitet. Dabei werden speziell für die Forschung entwickelte Anwendungen betrachtet, um dabei wichtige Anforderungen und Gestaltungsprinzipien zu identifizieren. Diese dienen als Orientierung für die Entwicklung von UPAT und werden im weiteren Verlauf entsprechend berücksichtigt.

3.1 Erstellung der zugrunde liegenden Anwendung

Im Rahmen eines zuvor durchgeführten Praxisprojekts wurde der Prototyp und damit auch die Grundlage für die in dieser Arbeit entwickelte Anwendung erstellt. Ziel dieses Projekts war die Konzeption eines VR-basierten Aufmerksamkeitstrainings für Ärztinnen und Ärzte, in dem diese ihre Aufmerksamkeit im Umgang mit einem virtuellen Patienten gezielt trainieren können. Das Training zielt darauf ab, die vom Patienten wahrgenommene Aufmerksamkeit zu verbessern und dadurch potenziell einen positiven Einfluss auf den Behandlungsverlauf auszuüben.

Im Folgenden werden die zentralen Schritte des Entwicklungsprozesses dieses Trainings dargestellt und erläutert. Dabei kommt der eingesetzten Hardware, der Pico Neo 3 Pro Eye als VR-Brille mit integriertem Eye-Tracking, sowie der Einarbeitung in das zugehörige Software Development Kit (SDK) eine besondere Bedeutung zu. Ein weiterer Fokus liegt auf der Erfassung und Verarbeitung der Blickbewegungen des Nutzers, die die Grundlage für die Bewertung des Aufmerksamkeitsverhaltens bilden.

Darüber hinaus wird die Gestaltung des Feedbacks in Form verschiedener Visual Delegates beschrieben, die die Auswirkungen des Aufmerksamkeitsverhaltens des Arztes auf den virtuellen Patienten visuell widerspiegeln.

Abschließend wird der Entwicklungsstand der Anwendung zum Abschluss des Praxisprojekts dargestellt. Darauf aufbauend wird ein Ausblick darauf gegeben, welche Optimierungen und Erweiterungen im Rahmen dieser Bachelorarbeit vorgenommen werden sollen und inwiefern die bestehende Anwendung für den Einsatz im Forschungskontext weiterentwickelt wird.

3.1.1 Technische Grundlagen der Entwicklungsumgebung

Bei der Pico Neo 3 Pro Eye handelt es sich um eine auf den Unternehmenseinsatz ausgelegte VR-Brille mit integriertem Eye-Tracking. Da die entwickelte Anwendung maßgeblich auf der präzisen Erfassung von Blickbewegungen basiert, ist diese Funktionalität eine wichtige Voraussetzung. Die Wahl fiel daher auf dieses Gerät, da es durch die verbauten Eye-Tracking-Kameras, die zugrunde liegende Hardware sowie die Verfügbarkeit eine geeignete und leistungsfähige Lösung für den vorgesehenen Anwendungsfall darstellt.

Die Integration der Pico Neo 3 Pro Eye in die Unity-Entwicklungsumgebung erfolgt über den OpenXR-Standard und das zugehörige Pico OpenXR SDK. OpenXR ist ein plattformübergreifender API-Standard, der darauf abzielt, Virtual- und Augmented-Reality-Geräte über eine gemeinsame Schnittstelle zu bündeln. Dadurch können Anwendungen geräteunabhängig entwickelt und mit minimalem Anpassungsaufwand auf unterschiedlichen XR-Systemen ausgeführt werden. Die vorliegende Anwendung wurde gezielt auf Basis dieses Standards umgesetzt, um eine zukünftige Übertragbarkeit auf weitere XR-Geräte mit integriertem Eye-Tracking zu gewährleisten und damit eine möglichst breite Nutzbarkeit sicherzustellen.

Zur Umsetzung von Eye-Tracking als Eingabemethode wurde zusätzlich das XR Interaction Toolkit verwendet. Dieses stellt eine Schnittstelle zur Integration verschiedener Eingabegeräte und Interaktionsformen bereit und ermöglicht eine geräteübergreifende Implementierung verschiedener Interaktionsmöglichkeiten innerhalb der Anwendung.

Insgesamt basiert die im Praxisprojekt entwickelte Anwendung auf folgendem Tech-Stack:

Software	Beschreibung	Version
Unity	Game-Engine	6000.0.62
OpenXR	Offener API-Standard	1.16.0
XR Interaction Toolkit	Interaction System for XR	3.0.10
PICO OpenXR Plugin	OpenXR Support for PICO Devices	1.4.1
PICO Neo Pro Eye	VR-Brille mit Eye-Tracking	5.9.9.0

Tabelle 5: Genutzter Techstack innerhalb des Praxisprojekts mit kompatiblen Softwareversionen

Während der Entwicklung der Anwendung zeigte sich, dass bestimmte Kombinationen von Softwareversionen zu Instabilitäten wie Anwendungsabstürzen und weiteren

Problemen führen können. Die in der Tabelle aufgeführten Versionen bilden eine geprüfte und untereinander kompatible Konfiguration, die einen stabilen Betrieb der Anwendung gewährleistet.

Darüber hinaus wurde festgestellt, dass die Verwendung des PICO Unity Integration SDK in der vorliegenden Konstellation zu Abstürzen der APK führt und daher für den Anwendungsfall nicht geeignet ist. Stattdessen wurde auf den OpenXR-Standard in Kombination mit dem XR Interaction Toolkit zurückgegriffen, um eine stabile und zugleich geräteübergreifende Implementierung zu gewährleisten.

Vor diesem Hintergrund wurde im Rahmen des Praxisprojekts sowie der weiteren Entwicklung dieser Arbeit bewusst auf die Aktualisierung der verwendeten Softwareversionen verzichtet, um die Systemstabilität nicht zu gefährden.

Zum Debugging der Anwendung wurde zudem das Android Logcat Package eingesetzt, das früher unter dem Namen Android Debug Bridge (ADB) bekannt war. Die Nutzung dieses Tools ist nicht zwingend erforderlich und wird daher nicht im oben aufgeführten Tech-Stack berücksichtigt, spielte jedoch eine wichtige Rolle im Entwicklungsprozess, da es eine detaillierte Analyse von Laufzeitfehlern, Abstürzen sowie systemseitigen Log-Ausgaben der Anwendung ermöglichte.

3.1.2 Trainingsumgebung und der virtuelle Patient

Bei der Gestaltung der Trainingsumgebung wurde versucht, eine klassische Gesprächssituation zwischen einem Arzt und einem virtuellen Patienten möglichst realitätsnah abzubilden. Hierfür wurde ein virtuelles Arztzimmer erstellt, in dem der Nutzer als behandelnder Arzt dem virtuellen Patienten gegenüber sitzt. Zwischen beiden befindet sich ein Schreibtisch, wodurch eine typische Gesprächssituation aus dem medizinischen Alltag simuliert wird.

Ziel der Gestaltung war es, eine möglichst glaubwürdige und zugleich ruhige Trainingsumgebung zu schaffen, in der sich die Nutzenden sicher fühlen und sich auf das Aufmerksamkeitstraining konzentrieren können. Der virtuelle Patient steht dabei klar im Mittelpunkt der Anwendung. Gleichzeitig enthält die Umgebung jedoch verschiedene weitere Objekte, die perspektivisch für zukünftige Forschungszwecke genutzt werden können, um gezielt Distraktoren in das Training zu integrieren.

Ein mögliches Beispiel hierfür ist der Computerbildschirm auf dem Schreibtisch des virtuellen Arztes. Auf diesem könnten künftig zusätzliche Aufgaben oder Informationen eingeblendet werden, die die Aufmerksamkeit des Nutzers bewusst vom Patienten ablenken. Dadurch ließen sich komplexere Aufmerksamkeitssituationen simulieren, in denen der Nutzer mehrere Reize gleichzeitig verarbeiten muss, ohne dabei die wahrgenommene Aufmerksamkeit des Patienten zu vernachlässigen.

Der virtuelle Patient sitzt dem Nutzer während des Trainings dauerhaft in einer einfachen Sitzanimation gegenüber. Darüber hinaus verfügt der Charakter im aktuellen Stand des Praxisprojekts noch über keine erweiterten Verhaltensweisen oder Interaktionen. In

zukünftigen Erweiterungen der Anwendung könnten zusätzlich animationsbasierte Visual Delegates eingesetzt werden, um dem Nutzer Rückmeldung zu seinem Aufmerksamkeitsverhalten zu geben. Denkbar wären beispielsweise Veränderungen in der Körpersprache, der Blickrichtung oder der Mimik des virtuellen Patienten, die dessen emotionale Reaktion auf fehlende Aufmerksamkeit widerspiegeln. Dadurch könnte das Feedback noch natürlicher und stärker in die Interaktion mit dem virtuellen Patienten integriert werden.

Bei den verwendeten 3D-Modellen handelt es sich ausschließlich um öffentlich verfügbare und frei nutzbare Assets aus dem Unity Asset Store.

3.1.3 Erfassung und Verarbeitung von Blickbewegungen mittels Eye-Tracking

Beim Eye-Tracking können verschiedene Parameter wie die Blickrichtung, der Pupillendurchmesser oder die Blinzelrate eines Nutzers erfasst und anschließend softwareseitig verarbeitet werden. Die Pico Neo 3 Pro Eye verfügt dabei über ein vergleichsweise einfaches Eye-Tracking-System, das ausschließlich die Blickrichtung des Nutzers erfasst. Diese Information wird in Unity genutzt, um zu ermitteln, auf welche Objekte oder Bereiche der Nutzer zu einem bestimmten Zeitpunkt fokussiert ist.

Mithilfe des OpenXR-Standards lassen sich die Eye-Tracking-Daten der Pico Neo 3 Pro Eye direkt in Unity als Eingabemethode integrieren. In der vorliegenden Anwendung wird insbesondere die sogenannte Eye-Gaze-Position verwendet, die als Richtungsvektor im dreidimensionalen Raum vorliegt.

Der virtuelle Patient fungiert zugleich als Interaktionsobjekt, auf das der Nutzer seine Aufmerksamkeit richten soll. Die Erfassung des Blickkontakts im XR Simple Interactable-Skript des XR Interaction Toolkits. Die zugrunde liegende Aufmerksamkeitserfassungslogik sowie die Verarbeitung dieser Interaktionen werden im folgenden Abschnitt detailliert erläutert.

Dieser Vektor dient als Grundlage für die weitere Verarbeitung. Aus der Perspektive des Nutzers in Unity, also des Player-Rigs, wird entlang der Blickrichtung ein Raycast in Kegelform projiziert. Auf diese Weise lässt sich das Sichtfeld des Nutzers darstellen und feststellen, welche Objekte sich darin befinden (vgl. Abbildung 8). Dies ermöglicht eine vergleichsweise realitätsnahe Abbildung des Sichtfelds des Nutzers innerhalb der virtuellen Umgebung.

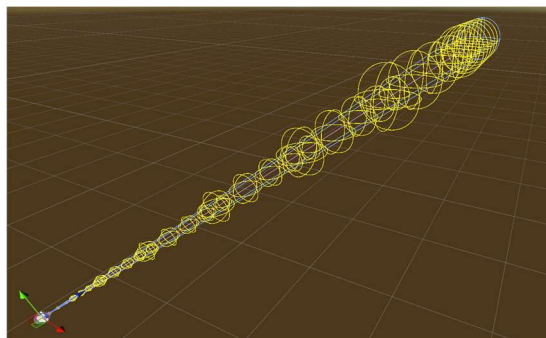


Abbildung 8: Visualisierung des Raycastkegels, der das Sichtfeld des Nutzers darstellt

Dieses Sichtfeld kann im weiteren Verlauf der Anwendung genutzt werden, um zu prüfen, ob der Blickkegel mit bestimmten Objekten in der Szene kollidiert. Hierfür wird das aus dem XR Interaction Toolkit stammende *XR Gaze Interactor*-Skript verwendet, das auf dem Player Rig platziert wird. Auf den Objekten, die durch den Blick des Nutzers erfasst werden sollen, kommt ergänzend das *XR Simple Interactable*-Skript zum Einsatz.

In Kombination ermöglichen diese beiden Komponenten eine gaze-basierte Interaktion: Der XR Gaze Interactor fungiert als eine Art Sender, der kontinuierlich entlang der Blickrichtung Raycasts überprüft, ob ein gültiges Interactable-Objekt getroffen wird. Trifft der Sichtkegel des Nutzers auf ein Objekt mit dem *XR Simple Interactable*-Skript, wird dieses als potenzielles Interaktionsziel erkannt.

Das XR Gaze Interactor stellt verschiedene Events bereit, wie beispielsweise *Hover Enter*, *Hover Exit* oder *Select*, die ausgelöst werden, sobald der Blick des Nutzers auf das Objekt fällt, dieses verlässt oder für eine bestimmte Dauer darauf verweilt. Insbesondere die Hover-Events sind im Kontext dieser Anwendung relevant, da sie es ermöglichen, kontinuierlich zu erfassen, ob ein Objekt aktuell im Fokus des Nutzers liegt.

Durch die Kombination beider Skripte entsteht somit ein System, mit dem sich das Blickverhalten des Nutzers in Echtzeit auswerten lässt. Dies bildet die Grundlage dafür, Aufmerksamkeitszustände zu erkennen, beispielsweise ob der Nutzer den virtuellen Patienten ansieht oder den Blick abwendet.

Im Falle der vorliegenden Anwendung soll dem Nutzer signalisiert werden, dass er den virtuellen Patienten bereits über einen längeren Zeitraum unbeachtet gelassen hat. Hierfür wird ein Skript verwendet, das mithilfe eines Timers prüft, wie lange sich der virtuelle Patient bereits außerhalb des Sichtfelds des Nutzers befindet. Solange Blickkontakt besteht, wird der Timer kontinuierlich zurückgesetzt. Verlässt der Nutzer jedoch den Fokusbereich des Patienten, läuft der Timer ab.

Wird ein zuvor im Unity-Editor definierter Geduldszeitraum überschritten, werden gezielt verschiedene Visual Delegates aktiviert, um dem Nutzer Rückmeldung zu seinem Aufmerksamkeitsverhalten zu geben. Die Intensität der ausgelösten Effekte nimmt dabei über Zeit zu, bis ein definierter Maximalwert erreicht ist. Sowohl die Dauer des Geduldszeitraums als auch die Geschwindigkeit, mit der sich die Intensität des Visual Delegates erhöht, können über das zuvor beschriebene Skript direkt im Unity-Editor konfiguriert werden.

Welche Visual Delegates in der Anwendung eingesetzt werden, wie diese innerhalb von Unity umgesetzt wurden und in welcher Form sich ihre Intensität im Zeitverlauf verändert, wird im folgenden Abschnitt näher erläutert.

3.1.4 Einsatz und Funktionsweise der verwendeten Visual Delegates

Zunächst musste jedoch festgelegt werden, welche Visual Delegates in der Anwendung eingesetzt werden sollten. Ziel war es, dem Nutzer indirekt zu vermitteln, wie sich der

virtuelle Patient während des Gesprächs fühlt, insbesondere in Situationen, in denen der Patient durch mangelnde Aufmerksamkeit ignoriert oder vernachlässigt wird.

Bereits im theoretischen Hintergrund wurde das sogenannte Vignette-Visual-Delegate beschrieben und eingeordnet, das insbesondere Gefühle von Anspannung, Unsicherheit oder Isolation vermitteln soll. Aus diesem Grund wurde dieses Visual Delegate auch im Rahmen des Aufmerksamkeitstrainings eingesetzt. Dabei zeigte sich jedoch, dass der Effekt in einer VR-Brille aufgrund der runden Displays und des größeren Sichtfelds weniger intensiv wahrgenommen wird als auf einem klassischen Monitor.

Um dennoch eine deutlich wahrnehmbare emotionale Wirkung zu erzielen, wurde das Vignette Visual Delegate mit weiteren Visual Delegates kombiniert. Ergänzend kamen ein Film-Grain-Visual Delegate sowie ein Grau-Filter-Visual Delegate zum Einsatz, um die dargestellte emotionale Belastung des virtuellen Patienten stärker auf den Nutzer zu übertragen und so eine intensivere Wirkung auf ihn zu erzielen.

Die Umsetzung dieser Visual Delegates erfolgte mithilfe des Post-Processing-Systems der Unity-Engine. Für die Vignette existiert in diesem System bereits ein vorgefertigter Effekt, der entsprechend angepasst wurde. Dabei wurde insbesondere der *Smoothness-Parameter* aktiviert, um weichere Übergänge am Rand der Vignette zu erzeugen und dadurch einen natürlicheren visuellen Eindruck zu schaffen (vgl. Abbildung 9). Das Zentrum des Effekts wurde auf die Mitte des Sichtfelds des Nutzers ausgerichtet.

Dabei lässt sich das Vignette-Visual-Delegate wie folgt im VD-Frame einordnen:

Depth	Abstraction	Control
FIL	NAT	ACT

Tabelle 6: Einordnung des Vignette Visual Delegates in das VD-Frame

Zusätzlich wurde ein eigenes Skript entwickelt, mit dem sich die Intensität der Vignette über Zeit verändern lässt. Über dieses Skript kann der *Intensity*-Parameter innerhalb eines definierten Zeitraums kontinuierlich von null auf eins erhöht werden. Dadurch verändert sich die Darstellung schrittweise von einem vollständig offenen Sichtfeld hin zu einer stark geschlossenen Vignette.

Wie bereits zuvor beschrieben, wurden ergänzend zum Vignette-Visual-Delegate zusätzlich ein Film-Grain- sowie ein Grau-Filter-Visual-Delegate eingesetzt, um die emotionale Wirkung auf den Nutzer weiter zu verstärken. Auch für diese beiden Effekte wurde das Unity Post-Processing-System verwendet.

Für das Film-Grain-Visual-Delegate steht innerhalb des Systems bereits eine entsprechende Komponente zur Verfügung. Hier wurde zunächst ein geeigneter Körnungstyp ausgewählt, in diesem Fall *Thick 01*, während der *Response*-Parameter auf dem Standardwert von 0,2 belassen wurde. Anschließend wurde auch für diesen Effekt ein eigenes Skript implementiert, das den *Intensity*-Parameter über einen definierten Zeitraum schrittweise erhöht und dadurch die visuelle Körnung zunehmend

verstärkt. Das Film-Grain Visual Delegate wird in der folgenden Tabelle in das VD-Frame eingeordnet:

Depth	Abstraction	Control
FIL	SYM	ACT

Tabelle 7: Einordnung des Film-Grain Visual Delegates in das VD-Frame

Zur Umsetzung des Grau-Filter-Visual-Delegates wurde die ebenfalls im Unity-Post-Processing-System enthaltene *Color Adjustments*-Komponente verwendet. Von den dort verfügbaren Parametern wurde ausschließlich der Sättigungswert genutzt, während alle weiteren Einstellungen deaktiviert blieben. Über ein weiteres Skript wird dieser Sättigungswert im Laufe der Zeit kontinuierlich reduziert. Dadurch verliert das Bild schrittweise an Farbinformation, bis es schließlich vollständig entsättigt und somit in Graustufen dargestellt wird. Auch dieses Visual Delegate kann wie folgt im VD-Frame eingeordnet werden:

Depth	Abstraction	Control
FIL	SYM	ACT

Tabelle 8: Einordnung des Grau-Filter Visual Delegates in das VD-Frame

Für jedes Visual Delegate wurde dabei ein eigenes Skript entwickelt, obwohl sich die grundlegende Funktionsweise der Effekte weitgehend ähnelt. Dies liegt vor allem daran, dass für die einzelnen Visual Delegates unterschiedliche Parameter zu steuern waren und dass es sich bei der Anwendung zunächst um einen funktionalen Prototyp handelt. Im Vordergrund stand daher primär das Testen und Validieren der einzelnen Funktionen, weniger die langfristige Skalierbarkeit oder Modularität der Softwarearchitektur.

Im Rahmen der technischen Weiterentwicklung der Anwendung soll dieser Ansatz jedoch überarbeitet werden. Ziel ist es, die Implementierung modularer und leichter erweiterbarer zu gestalten, sodass zukünftige Visual Delegates effizienter integriert und verwaltet werden können. Die geplanten Anpassungen werden daher in Kapitel 4 detailliert erläutert.

Außerdem fällt bei der Einordnung der verwendeten Visual Delegates in das VD-Frame, dass diese sich in ihrer grundlegenden Gestaltung stark ähneln. So sind alle aktuell implementierten Visual Delegates auf der Filter-Ebene einzuordnen und nutzen visuelle Veränderungen des Sichtfelds, um dem Nutzer Rückmeldung über sein Aufmerksamkeitsverhalten zu geben. Für die Entwicklung der vorliegenden Anwendung sowie die technische Erweiterung von UPAT wurde dies zunächst als ausreichend betrachtet, da der Fokus dieser Arbeit primär auf der Schaffung einer technischen Grundlage für zukünftige Forschung lag. Durch die entwickelte Architektur können jedoch künftig problemlos weitere Arten von Visual Delegates ergänzt werden. Für zukünftige Forschungsarbeiten wäre es daher sinnvoll, zusätzliche Visual Delegates zu entwickeln, die auch andere Ebenen und Ausprägungen des VD-Frames abdecken.

3.1.5 Abschließender Entwicklungsstand und Ausblick

Mit Abschluss des Praxisprojekts wurde ein erster funktionsfähiger Prototyp eines VR-basierten Aufmerksamkeitstrainings für Ärztinnen und Ärzte entwickelt. Die Anwendung nutzt die Pico Neo 3 Pro Eye, eine VR-Brille mit integriertem Eye-Tracking, und wurde auf Basis des OpenXR-Standards umgesetzt, um eine möglichst hohe Gerätekompatibilität sowie die zukünftige Übertragbarkeit auf weitere XR-Systeme zu gewährleisten.

Innerhalb der Anwendung werden die Blickdaten des Nutzers als Eingabe verwendet. Der virtuelle Patient dient dabei als primäres Aufmerksamkeitsziel, das vom Nutzer betrachtet werden soll. Wird der Patient über einen zuvor definierten Zeitraum hinweg ignoriert, werden verschiedene Visual Delegates ausgelöst, die dem Nutzer unmittelbares Feedback zu seinem Aufmerksamkeitsverhalten geben. Die grundlegende Eye-Tracking-basierte Interaktion funktionierte im Praxisprojekt bereits zuverlässig und bildet daher auch weiterhin die technische Grundlage der Anwendung. Im Rahmen der technischen Erweiterung erfolgen daher lediglich kleinere Anpassungen zur besseren Integration weiterer Systeme und Funktionen.

Zur Rückmeldung des Aufmerksamkeitsverhaltens werden mehrere Visual Delegates gleichzeitig eingesetzt. Bei der Umsetzung lag der Fokus zunächst auf einer möglichst schnellen und funktionalen Entwicklung des Prototyps und weniger auf einer langfristig skalierbaren Softwarearchitektur. Aus diesem Grund wurde für jedes Visual Delegate ein separates Skript implementiert, das die jeweiligen Effektparameter über einen bestimmten Zeitraum hinweg verändert. Dieser Ansatz würde bei der Erweiterung um zusätzliche Visual Delegates zu mehrfacher Code-Duplikation führen, da für jeden neuen Effekt erneut eigene Steuerungslogik entwickelt werden müsste.

Im Rahmen der technischen Erweiterung der Anwendung soll daher eine modulare Softwarearchitektur geschaffen werden, die eine einfache Integration neuer Visual Delegates ermöglicht, ohne bestehende Systeme anpassen oder redundanten Code erneut implementieren zu müssen.

Darüber hinaus soll die Anwendung gezielt auf den künftigen Einsatz im wissenschaftlichen Kontext vorbereitet werden. Hierzu gehört insbesondere die Speicherung relevanter Trainingsdaten, beispielsweise der verwendeten Visual-Delegate-Kombinationen oder der Anzahl der Aufmerksamkeitsverluste während einer Sitzung. Diese Daten sollen strukturiert in einer CSV-Datei gespeichert werden, um sie anschließend wissenschaftlich auswerten zu können. Die hierfür notwendige Logging-Architektur war nicht Bestandteil des Praxisprojekts und wird daher vollständig im Rahmen der technischen Erweiterung entwickelt.

Ein weiterer Bestandteil der Erweiterung ist die Entwicklung eines Startmenüs. Über dieses Menü sollen verschiedene Trainingsparameter flexibel angepasst werden können. Dazu zählen unter anderem die Auswahl unterschiedlicher Visual-Delegate-Kombinationen, die Trainingsdauer, die Gedulddauer des virtuellen Patienten sowie die Fade-In- und Fade-Out-Zeiten der eingesetzten Visual Delegates. Zusätzlich soll eine

Einordnung der ausgewählten Visual Delegates im VD-Frame dargestellt werden, um eine nachträgliche Analyse und Vergleichbarkeit der eingesetzten Effekte zu ermöglichen.

Diese geplanten Erweiterungen wurden in wiederholten Abstimmungsgesprächen mit einem der späteren Forschenden erarbeitet und im Verlauf der Arbeit iterativ weiter verfeinert. Ziel war es, die Anwendung gezielt auf den zukünftigen wissenschaftlichen Einsatz auszurichten und dabei die Anforderungen der Forschung möglichst vollständig zu berücksichtigen. Durch diesen iterativen Entwicklungsprozess entstand schrittweise ein Konzept für eine flexible und erweiterbare VR-Anwendung, die sowohl technisch als auch funktional auf die Durchführung wissenschaftlicher Studien ausgerichtet ist.

Des Weiteren wurden vergleichbare Anwendungen analysiert, um bestehende Lösungsansätze in ähnlichen VR-Trainingssystemen zu untersuchen. Die daraus gewonnenen Erkenntnisse flossen ebenfalls in die Konzeption und Weiterentwicklung der eigenen Anwendung ein.

3.2 Anwendungsanalysen ähnlicher Anwendungen

Im Folgenden werden verschiedene vergleichbare VR-Aufmerksamkeitstrainings analysiert. Ziel dieser Analyse ist es, anhand bereits vorhandener und etablierter Anwendungen mögliche Probleme, geeignete Lösungsansätze sowie bewährte Gestaltungskonzepte zu identifizieren. Ein besonderer Fokus liegt dabei auf der Eignung der Anwendungen für den wissenschaftlichen Einsatz sowie auf den Anforderungen, die für eine Nutzung im Forschungskontext erfüllt werden müssen.

Aus den Erkenntnissen dieser Analyse wird gemeinsam mit den zuvor dargestellten wissenschaftlichen Studien sowie den daraus abgeleiteten Kriterien für immersive VR-Anwendungen ein Konzept für VR-basierte Aufmerksamkeitstrainings entwickelt, an dem sich die technische Erweiterung von UPAT orientiert.

Aufgrund der unterschiedlichen Hardwareanforderungen der analysierten Anwendungen, insbesondere hinsichtlich der jeweils verwendeten VR-Brillen, konnten nicht alle Anwendungen auf der dafür vorgesehenen Hardware ausgeführt werden. Die Analyse beschränkt sich daher auf die Ausführung der Projekte im Unity-Editor sowie auf die Untersuchung der Funktionsweise der enthaltenen Skripte und Systemarchitekturen.

Ergänzend umfasst die Analyse die in den jeweiligen GitHub-Repositories bereitgestellten README-Dateien sowie die zugehörigen wissenschaftlichen Veröffentlichungen und Studien. Dadurch sollen sowohl technische Implementierungsdetails als auch konzeptionelle und wissenschaftliche Aspekte der Anwendungen berücksichtigt werden, selbst wenn eine vollständige Ausführung der Anwendungen auf der jeweiligen Zielhardware nicht möglich war.

3.2.1 Wildcard

Bei Wildcard (Buttiglione, 2022) handelt es sich um ein VR-basiertes Aufmerksamkeitstraining, das speziell auf Personen mit neurologischen Entwicklungsstörungen wie ADHS oder Autismus zugeschnitten ist. Wie bereits in Kapitel 2.1.3 beschrieben, nutzt die Anwendung Eye-Tracking als wichtiges Interaktionselement, um Aufmerksamkeit präzise zu erfassen und Nutzenden eine Interaktion über ihre Blickrichtung zu ermöglichen.

Innerhalb der Wildcard-Anwendung gibt es verschiedene Trainingsaufgaben, die von den Nutzenden zu bearbeiten sind. Die Gemeinsamkeit dieser Aufgaben ist, dass der Aufmerksamkeitsfokus kontinuierlich auf bestimmten Charakteren oder Objekten liegen muss, die in der erzählten Handlung hervorgehoben werden, oder dass der Blick dauerhaft auf einem Charakter gehalten werden muss, um im Training fortschreiten zu können.

Die Überprüfung dieses Aufmerksamkeitsfokus erfolgt mithilfe der Blickdaten des Nutzers sowie anhand der Ausrichtung des gehaltenen Controllers. Technisch setzt die Anwendung, ebenso wie UPAT, auf die Pico Neo 3 Pro Eye in Kombination mit dem OpenXR-Standard.

Im Vergleich zu anderen analysierten Anwendungen zeichnet sich Wildcard insbesondere durch eine klar definierte, gut in die Narrative eingebundene und leicht verständliche Primäraufgabe aus, bei der die Nutzenden ihren Blick gezielt auf einen Protagonisten fokussieren müssen. Da sich Wildcard insbesondere in diesem Aspekt positiv von den übrigen analysierten Anwendungen abhebt und die Wirksamkeit dieser Trainingsmechanik im Rahmen der zugehörigen Studie wissenschaftlich gestützt wird, dient dieses Konzept als wichtige Orientierung für die eigene Anwendung.

Während in der Wildcard-Anwendung bewusst tierische Protagonisten eingesetzt werden, um eine erzählerische Umgebung zu schaffen und zugleich möglichst wenig belastend oder bedrohlich auf die Teilnehmenden zu wirken, verfolgt UPAT einen stärker realitätsorientierten Ansatz. Ziel ist es, eine typische Gesprächssituation aus dem Arbeitsalltag behandelnder Ärzte möglichst authentisch abzubilden. Aus diesem Grund wird zwar das grundlegende Konzept der Primäraufgabe aus Wildcard übernommen, die konkrete Umsetzung erfolgt jedoch in angepasster Form.

Anstelle stilisierter Tierfiguren wird in der eigenen Anwendung ein virtueller menschlicher Patient dargestellt, der während des Trainings kontinuierlich fokussiert werden soll. Auch die virtuelle Umgebung orientiert sich stärker an realen Gegebenheiten und stellt eine klassische Arztpraxis dar, anstatt eine abstrahierte oder stilisierte Trainingswelt zu verwenden.

Auf diese Weise sollen die positiven Aspekte der in Wildcard eingesetzten Primäraufgabe mit den Erkenntnissen aus Studien zu medizinischen VR-Trainings kombiniert werden, die darauf hinweisen, dass insbesondere realitätsnahe Umgebungen und glaubwürdig dargestellte Patienten zu einer höheren wahrgenommenen Immersion

beitragen (Zielke et al., 2017). Ziel ist es daher, eine Trainingsaufgabe zu konzipieren, die sowohl leicht verständlich als auch möglichst nah an realen medizinischen Gesprächssituationen orientiert ist und damit optimal zum vorgesehenen Anwendungskontext passt.

3.2.2 VERA

Die Anwendung VERA (Delvigne, 2020) wurde bereits in Kapitel 2.1.3 kurz vorgestellt. Dabei handelt es sich um ein VR-basiertes Aufmerksamkeitstraining für Kinder mit ADHS, bei dem insbesondere die Erfassung und Analyse von Aufmerksamkeitsprozessen im Fokus steht. Hierzu werden unter anderem Eye-Tracking-, EEG- und Bewegungsdaten genutzt, um visuelle Aufmerksamkeit in verschiedenen Trainingsszenarien präzise zu analysieren.

In genau dieser detaillierten Analyse von Aufmerksamkeitsprozessen liegt die besondere Stärke von VERA. Im Vergleich zu den anderen betrachteten Anwendungen fokussiert sich VERA insbesondere darauf, Aufmerksamkeitsdaten so zu erfassen und aufzubereiten, dass diese auch für wissenschaftliche Untersuchungen nutzbar sind. Dazu zählt unter anderem die Erfassung sogenannter *Raw Signals*, beispielsweise der Blinzelrate oder des Pupillendurchmessers der Nutzenden. Darüber hinaus wird eine sogenannte Baseline-Aufgabe eingesetzt, um eine Vergleichsgrundlage für das eigentliche Training zu schaffen.

Eine weitergehende Erfassung zusätzlicher Eye-Tracking-Daten wäre grundsätzlich sinnvoll, ist mit der Pico Neo 3 Pro Eye jedoch nur eingeschränkt möglich. Das eingesetzte Eye-Tracking-System ist vergleichsweise einfach aufgebaut und erlaubt keine Erfassung komplexerer Parameter wie der Blinzelrate oder des Pupillendurchmessers.

Werden die in VERA verwendeten Konzepte zur Datenerfassung und Vergleichbarkeit als Grundlage herangezogen, sollten auch innerhalb der im Rahmen dieser Arbeit entwickelten Anwendung verschiedene Aufmerksamkeitsdaten aufgezeichnet werden. Dazu zählen insbesondere die Blickrichtung des Nutzers, Informationen darüber, wann der virtuelle Patient fokussiert wird, wie lange dieser Fokus besteht, sowie darüber, wie häufig der Blickkontakt im Verlauf des Trainings verloren geht.

Bei der Analyse der in VERA aufgezeichneten Aufmerksamkeitsdaten wird zudem deutlich, dass sämtliche Daten mit Zeitstempeln versehen werden. Dadurch können die erfassten Informationen später konkreten Situationen innerhalb des Trainings zugeordnet und zeitlich analysiert werden. Dieses Vorgehen stellt auch für die im Rahmen der vorliegenden Arbeit erstellten Anwendung einen wichtigen Orientierungspunkt dar.

Darüber hinaus bietet VERA die Möglichkeit, vor dem eigentlichen Training eine sogenannte Baseline-Aufgabe durchzuführen. Diese dient dazu, die Nutzenden zunächst mit der Anwendung vertraut zu machen und gleichzeitig eine Vergleichsgrundlage für spätere Trainingsdaten zu schaffen. Ein solches Vorgehen ist

insbesondere im wissenschaftlichen Kontext weit verbreitet und erscheint auch für UPAT sinnvoll.

Dementsprechend sollte auch innerhalb der eigenen Anwendung eine Baseline-Phase integriert werden, in der bereits Aufmerksamkeitsdaten erfasst werden, ohne das Verhalten der Nutzenden durch Visual Delegates zu beeinflussen. Dadurch könnten vergleichbare Ausgangsdaten erzeugt werden, die anschließend den während des eigentlichen Aufmerksamkeitstrainings erhobenen Daten gegenübergestellt werden können.

3.2.3 Dimensional Search

Dimensional Search (Savickaite et al., 2021) ist das einzige analysierte Aufmerksamkeitstraining, das in dieser Arbeit bislang noch nicht näher betrachtet wurde. Dies liegt insbesondere daran, dass das zugehörige Paper lediglich eine kurze Beschreibung der Anwendung enthält und keine wissenschaftlichen Erkenntnisse zur Wirksamkeit des Trainings liefert.

Dennoch liefert Dimensional Search relevante Ansätze für die Konzeption der eigenen Anwendung. Bei der Anwendung handelt es sich um ein VR-basiertes Aufmerksamkeitstraining, mit dessen Hilfe die visuelle Suchleistung von Nutzenden in 2D-, 2D-VR- und 3D-VR-Umgebungen verglichen werden soll. Hierfür werden innerhalb der unterschiedlichen Darstellungsformen verschiedene farbige Formen eingeblendet und nach kurzer Zeit wieder ausgeblendet. Im Anschluss müssen die zuvor angezeigten farbigen Formen in einer größeren Auswahl an Formen wiedergefunden werden. Ziel der Anwendung ist es, Forschenden die Untersuchung der Zusammenhänge zwischen Tiefenwahrnehmung, räumlicher Aufmerksamkeit und Gedächtnisleistung im Kontext visueller Suchprozesse zu ermöglichen.

Dimensional Search ähnelt UPAT dabei in einem besonders wichtigen Aspekt, den keine der anderen analysierten Anwendungen in vergleichbarer Form erfüllt. Die Anwendung dient nicht ausschließlich der Durchführung einer einzelnen Studie, sondern stellt vielmehr eine flexible Grundlage für unterschiedliche zukünftige Forschungsvorhaben bereit. Entsprechend muss die Anwendung so gestaltet sein, dass Forschende sie an die Anforderungen der jeweiligen Studie anpassen können.

Hierfür wird zu Beginn der Anwendung ein Konfigurationsmenü bereitgestellt, in dem verschiedene Trainingsparameter angepasst werden können. Dimensional Search nutzt dafür sogenannte *Blocks* (vgl. Abbildung 8), innerhalb derer Parameter wie die Anzahl der zu merkenden Formen, die Einblendungsdauer der Zielobjekte oder die Anzahl der durchzuführenden sogenannten *Trials* festgelegt werden können. Unter einem Trial versteht man dabei einen vollständigen Durchlauf einer einzelnen Trainingsaufgabe,

bestehend aus dem Einblenden der zu merkenden Formen sowie dem anschließenden Wiederfinden dieser Formen innerhalb einer größeren Auswahl verschiedener Formen.

Mehrere dieser *Blocks* können anschließend direkt aufeinanderfolgend ausgeführt werden, wodurch unterschiedliche Trainingskonfigurationen innerhalb einer Sitzung getestet werden können.

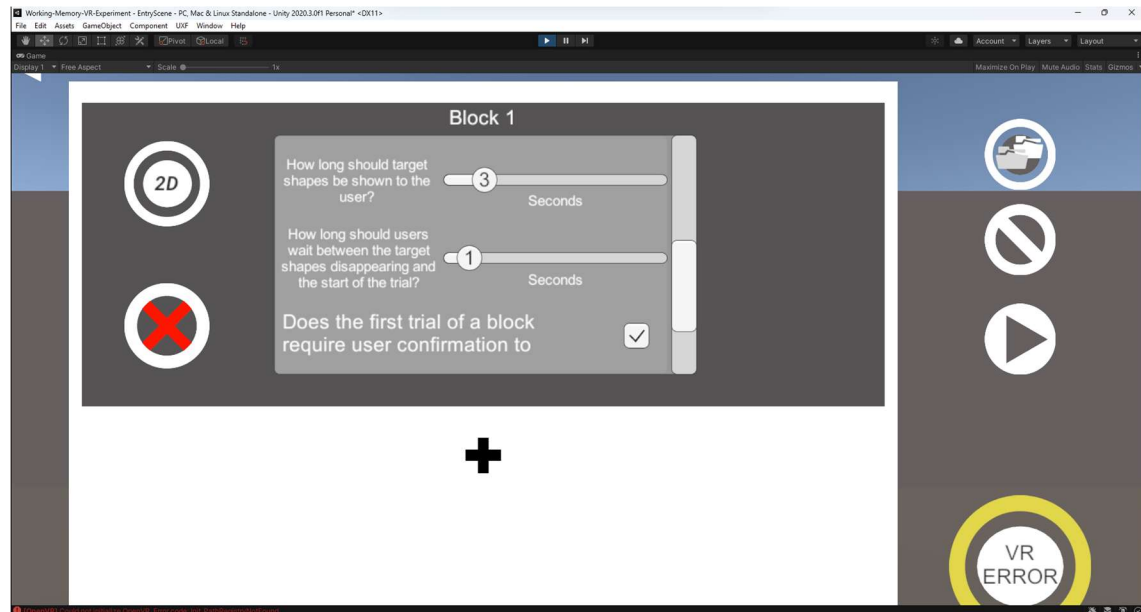


Abbildung 9: Start-Menü in Dimensional Search. Durch das Plussymbol können neue *Blocks* dem Training hinzugefügt werden. Innerhalb der einzelnen *Blocks* können die Trainingsparameter der Anwendung angepasst werden. Innerhalb des Trainings werden alle hier angelegten *Blocks* hintereinander durchlaufen.

Da auch UPAT für den Einsatz in unterschiedlichen wissenschaftlichen Studien durch verschiedene Forschende optimiert werden soll, erscheint eine Orientierung an diesem Ansatz sinnvoll. Eine direkte Übernahme des Block-Systems bietet sich aufgrund der anderen Primäraufgabe, des kontinuierlichen Haltens der Aufmerksamkeit auf einem virtuellen Patienten, jedoch nur eingeschränkt an.

Der grundlegende Gedanke, Forschenden vor Beginn des Trainings die Kontrolle über relevante Trainingsparameter zu ermöglichen, stellt dennoch einen wichtigen Mehrwert für die zukünftige Nutzung von UPAT im wissenschaftlichen Kontext dar und wird daher in abgeänderter Version innerhalb der technischen Erweiterung in UPAT integriert.

3.2.4 Anpassungen für UPAT

Nach Abschluss des Praxisprojekts liegt mit UPAT ein erster funktionsfähiger Prototyp eines VR-basierten Aufmerksamkeitstrainings vor, bei dem die Nutzenden in Echtzeit Feedback zu ihrem Aufmerksamkeitsverhalten erhalten. Im Rahmen dieser Arbeit soll dieses Training nun um weitere zentrale Funktionen erweitert und gezielt auf den zukünftigen Einsatz im wissenschaftlichen Kontext vorbereitet werden.

Durch die Analyse vergleichbarer Anwendungen konnten bereits mehrere Komponenten identifiziert werden, die innerhalb der technischen Erweiterung von UPAT umgesetzt

werden sollen, um eine wissenschaftliche Nutzung der Anwendung zu ermöglichen. Dazu zählen insbesondere:

- die automatisierte Erfassung und Speicherung von Aufmerksamkeitsdaten in Form von CSV-Dateien,
- die Integration einer Baseline-Aufgabe, innerhalb derer dieselben Aufmerksamkeitsdaten wie im eigentlichen Training erfasst werden, jedoch ohne Einflussnahme durch Visual Delegates,
- sowie die Möglichkeit zur einfachen Konfiguration der Trainingsparameter direkt innerhalb der Anwendung, damit Forschende das Training flexibel an die Anforderungen ihrer jeweiligen Studie anpassen können.

Eine gut in die narrative Struktur eingebundene Primäraufgabe, wie sie in der Analyse von Wildcard als besonders relevant herausgearbeitet wurde, ist in UPAT bereits vorhanden. Diese besteht darin, die Aufmerksamkeit kontinuierlich auf den virtuellen Patienten gerichtet zu halten. Die grundlegende Trainingsmechanik bleibt daher zunächst bestehen. Allerdings sollen verschiedene Parameter dieser Primäraufgabe künftig konfigurierbar sein, wodurch auch Anpassungen an den zugrunde liegenden Prozessen und Skripten erforderlich werden.

Die Analyse vergleichbarer Anwendungen bildet jedoch nur einen Teil der Grundlage für die technische Erweiterung. Zusätzlich fließen die zuvor erarbeiteten Gestaltungskriterien für immersive VR-Anwendungen in die Konzeption ein.

Darüber hinaus erfolgte die Planung und Weiterentwicklung der Anwendung in kontinuierlichem Austausch mit einem der Forschenden, der die Anwendung später im wissenschaftlichen Kontext nutzen soll. Durch diese iterative Vorgehensweise und das wiederholte Einholen von Feedback soll sichergestellt werden, dass die entwickelten Funktionen den tatsächlichen Anforderungen der Forschung entsprechen und die Anwendung gezielt auf ihren zukünftigen Einsatz vorbereitet wird. Welche konkreten Funktionsanforderungen dabei identifiziert wurden und wie diese im Rahmen der technischen Erweiterung umgesetzt werden sollen, wird im folgenden Kapitel detailliert beschrieben.

4 Konzeptionierung

In den vergangenen Kapiteln wurden bereits verschiedene wissenschaftliche Studien zu VR-Anwendungen sowie vergleichbare Aufmerksamkeitstrainings analysiert. Dabei konnten verschiedene Gestaltungskonzepte identifiziert werden, die einen wesentlichen Einfluss auf die immersive Wirkung von VR-Trainings haben. Da ein hohes Maß an Immersion maßgeblich zum Trainingserfolg beiträgt, sollten diese Gestaltungsprinzipien bei der Konzeption entsprechender Anwendungen berücksichtigt werden.

Darüber hinaus wurden bestehende Anwendungen untersucht, um potenzielle Probleme frühzeitig zu identifizieren und bereits etablierte Lösungsansätze in die eigene

Entwicklung einfließen zu lassen. Ein besonderer Fokus lag dabei auf der Erfassung wissenschaftlich relevanter Daten sowie auf den Anforderungen, die eine Anwendung für den Einsatz im Forschungskontext erfüllen muss.

Zudem wurde mit einem der Forschenden gesprochen, der die entwickelte Anwendung nach Abschluss der technischen Erweiterung im Rahmen eigener wissenschaftlicher Untersuchungen einsetzen wird. In diesem der technischen Erweiterung vorausgehenden Gespräch wurden verschiedene Anforderungen und gewünschte Funktionen für die Anwendung herausgearbeitet. Dabei wurden folgende gewünschte Funktionen für UPAT genannt:

- Innerhalb des Unity-Editors sollen neue Visual Delegates mit möglichst geringem zusätzlichem Programmieraufwand erstellt werden können, ohne dass bestehende Prozesse oder Skripte angepasst werden müssen.
- Bereits vorhandene Visual Delegates als auch von Forschenden neu angelegte Visual Delegates sollen im Unity-Editor dem VD-Frame zugeordnet werden können.
- Die Parameter einzelner Visual Delegates sollen direkt im Unity-Editor konfigurierbar sein. Die vorgenommenen Einstellungen sollen automatisch in das Training übernommen werden.
- Vor Beginn des Trainings soll innerhalb der Anwendung ein Menü angezeigt werden, in dem einzelne Visual Delegates für den jeweiligen Trainingsdurchlauf aktiviert oder deaktiviert werden können. Dabei soll es auch möglich sein, mehrere Visual Delegates gleichzeitig zu verwenden.
- Zusätzlich soll ein Konfigurationsmenü bereitgestellt werden, in dem verschiedene Trainingsparameter sowie spezifische Parameter der eingesetzten Visual Delegates angepasst werden können.
- Vor dem Start des Trainings soll außerdem zwischen einem Endlosmodus und einem zeitlich begrenzten Trainingsmodus gewählt werden können. Im Zeitmodus soll die Trainingsdauer individuell konfigurierbar sein.
- Während des Trainings sollen verschiedene Aufmerksamkeitsparameter automatisch in einer eigens für den jeweiligen Trainingsdurchlauf erstellten CSV-Datei gespeichert werden. Die erzeugten Dateien sollen dabei eindeutig einem bestimmten Trainingsdurchlauf zugeordnet werden können und zusätzlich sämtliche relevanten Trainingsparameter enthalten.

Um sicherzustellen, dass keine relevanten Anforderungen übersehen werden, wurden im Anschluss User Stories aus Sicht der jeweiligen Nutzergruppen erstellt (vgl. Anhang A1). Innerhalb dieser User Stories wurden zudem konkrete Zielvoraussetzungen und erwartete Funktionalitäten für die jeweiligen Features definiert. Anschließend wurden die erarbeiteten User Stories gemeinsam mit dem Forschenden überarbeitet, abgestimmt und verifiziert. Aus diesen finalisierten User Stories wurden anschließend die konkreten Entwicklungsaufgaben für die technische Umsetzung der einzelnen Funktionen abgeleitet.

Auf Grundlage dieser Erkenntnisse sowie der Erfahrungen aus dem im Praxisprojekt entwickelten Prototypen wird im Folgenden ein Anwendungskonzept als auch ein Softwarearchitekturkonzept für die Entwicklung VR-basierter Aufmerksamkeitstrainings im wissenschaftlichen Kontext erarbeitet. Diese sollen sowohl als Orientierung für zukünftige vergleichbare Anwendungen dienen als auch als Grundlage für die technische Erweiterung von UPAT verwendet werden.

4.1 Anwendungskonzept

Das vorliegende Anwendungskonzept in Form eines Flow-Charts (vgl. Abbildung 10) soll den typischen Ablauf der Anwendung beschreiben und die wichtigsten Prozesse sowie deren Zusammenhänge visualisieren.

Zu Beginn der Anwendung erfolgt zunächst die Konfiguration des Trainings innerhalb des Startmenüs. Hier können die Forschenden auswählen, welche Visual Delegates während des Trainings verwendet werden sollen. Zusätzlich lassen sich verschiedene Trainingsparameter anpassen, beispielsweise die Trainingsdauer, die Geduldszeit des virtuellen Patienten oder spezifische Parameter der eingesetzten Visual Delegates. Die gewählten Einstellungen werden anschließend gemeinsam mit den Namen der aktivierten Visual Delegates in einer CSV-Datei gespeichert, um eine spätere Zuordnung und wissenschaftliche Auswertung des jeweiligen Trainingsdurchlaufs zu ermöglichen.

Nach dem Start des Trainings beginnt die Primäraufgabe der Anwendung, bei der die Aufmerksamkeit möglichst konstant auf den virtuellen Patienten gerichtet bleiben soll. Währenddessen wird kontinuierlich überprüft, ob der Blickkontakt mit dem Patienten besteht oder verloren geht. Solange der Fokus aufrechterhalten wird, läuft das Training ohne Eingriffe weiter.

Wird hingegen ein Fokusverlust erkannt, wird zunächst ein entsprechendes Event in der CSV-Datei gespeichert. Anschließend beginnt die Geduld des virtuellen Patienten abzulaufen. Überschreitet der Fokusverlust diesen zuvor definierten Zeitraum, werden die aktivierten Visual Delegates eingeblendet und verstärken sich mit zunehmender Dauer des Aufmerksamkeitsverlusts.

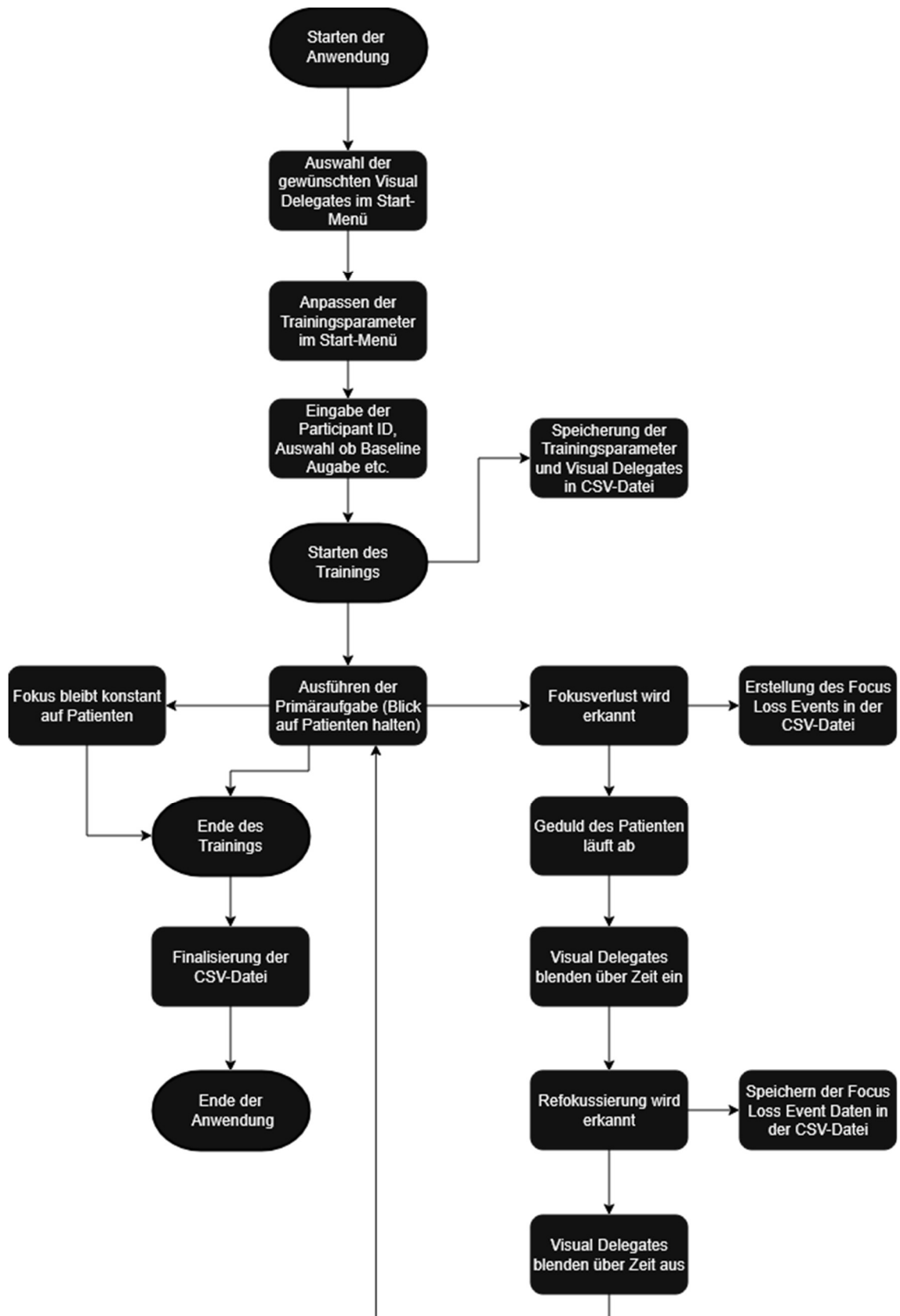


Abbildung 10: Flowchart des Anwendungsablaufs

Sobald eine Refokussierung erkannt wird und der Nutzer den virtuellen Patienten erneut ansieht, werden die Daten des Fokusverlusts in dem zuvor erstellten Event innerhalb der

CSV-Datei dokumentiert. Anschließend werden die aktiven Visual Delegates über Zeit ausgeblendet, bis die normale Trainingssituation wiederhergestellt ist.

Dieser Ablauf wiederholt sich fortlaufend, bis das Training beendet wird oder die zuvor definierte Trainingsdauer erreicht ist. Nach Abschluss des Trainings erfolgt die Finalisierung der CSV-Datei, sodass sämtliche aufgezeichneten Trainings- und Aufmerksamkeitsdaten für eine spätere wissenschaftliche Analyse bereitstehen.

Dieses Anwendungskonzept beschreibt den grundlegenden Ablauf der Anwendung auf einer eher oberflächlichen Ebene. Um darüber hinaus nachvollziehbar darzustellen, wie die einzelnen Prozesse, Systeme und Skripte im Hintergrund miteinander interagieren, wird im folgenden Unterkapitel ein Softwarearchitekturkonzept vorgestellt.

4.2 Softwarearchitekturkonzept

Innerhalb des Softwarearchitekturdiagramms, dargestellt in Abbildung 11, sind alle wichtigen Skripte und Systeme von UPAT sowie deren Zusammenhänge dargestellt. Ziel dieser Architektur ist es insbesondere, eine möglichst modulare und erweiterbare Struktur bereitzustellen, damit zukünftige Forschende neue Visual Delegates oder weitere Funktionen integrieren können, ohne die bestehenden Kernsysteme anpassen zu müssen.

Aus diesem Grund muss die Architektur der Anwendung so gestaltet sein, dass die einzelnen Komponenten möglichst unabhängig voneinander funktionieren und klar voneinander getrennte Aufgabenbereiche haben. Insbesondere die Verwaltung, Aktivierung und Steuerung der Visual Delegates sollen dabei über ein modular aufgebautes System erfolgen, in das neue Effekte integriert werden können, ohne die bestehende Logik zu verändern.

Im Mittelpunkt der Architektur steht dabei die *Visual Delegate Base Class*. Diese fungiert als gemeinsame Basisklasse aller Visual Delegates und definiert die grundlegende Funktionalität, die alle Delegates benötigen. Dazu zählt insbesondere die Möglichkeit, bestimmte Parameter eines Effekts über Zeit zu verändern. Neue Visual Delegates basieren auf dieser Basisklasse und müssen dadurch lediglich ihre spezifische Effektlogik implementieren.

Für Visual Delegates, die auf Unitys Post-Processing-System basieren, gibt es bereits die Post-Processing-Visual-Delegate-Klasse, die wiederum auf der allgemeinen Basisklasse basiert. Die bereits implementierten Effekte wie das *Vignette Visual Delegate*, das *Film Grain Visual Delegate* sowie das *Grey Filter Visual Delegate* basieren alle auf dieser Klasse. Dadurch lassen sich gemeinsame Funktionen, beispielsweise das Hinzufügen eines Effekts zum Post-Processing Volume oder das Ein- und Ausblenden der Effekte, an einer Stelle verwalten.

Die gestrichelt dargestellten *Extension Points* visualisieren die vorgesehenen Erweiterungspunkte der Architektur. Forschende können an diesen Stellen neue Post-Processing-Effekte oder eigene Subklassen für vollständig neue Arten von Visual

Delegates, etwa UI- oder animationsbasierte Effekte, ergänzen, ohne bestehende Systeme verändern zu müssen.

Die Steuerung der im Training aktiven Visual Delegates erfolgt über den *Visual Delegate Manager*. Dieser erhält vom OpenXR-Interaktionssystem Informationen darüber, ob ein *Gaze Select* beziehungsweise *Gaze Unselect* stattgefunden hat. Grundlage hierfür bilden die Eye-Tracking-Daten des Nutzers. Wird ein Fokusverlust erkannt, aktiviert der Manager die aktuell ausgewählten Visual Delegates und erhöht deren ausgewählte Parameter über Zeit. Erfolgt eine Refokussierung des virtuellen Patienten, werden die Effekte zunächst reduziert und schließlich vollständig deaktiviert.

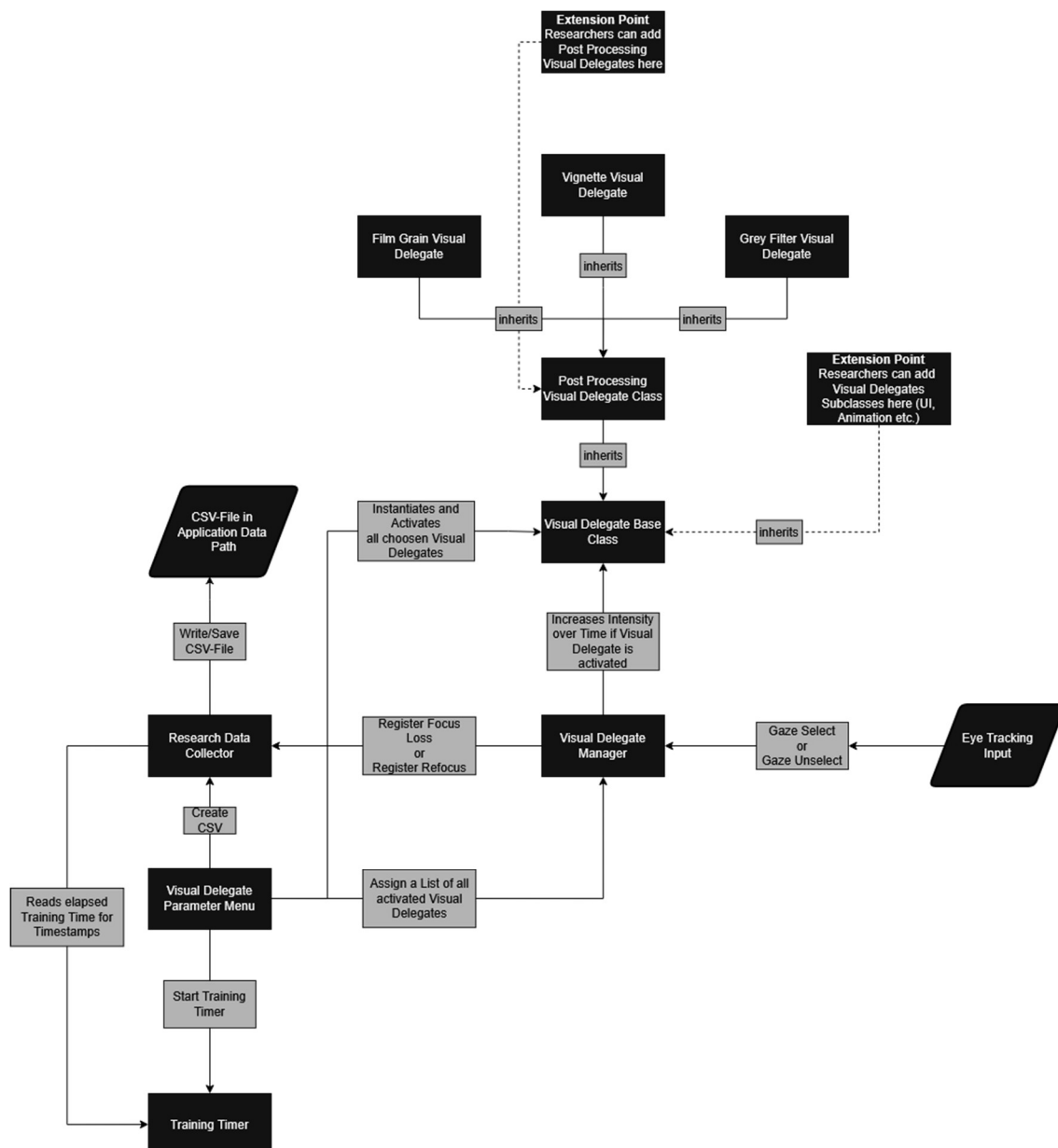


Abbildung 11: Flowchart der Softwarearchitektur

Welche Visual Delegates während eines Trainingsdurchlaufs aktiv sind, wird zuvor innerhalb des *Visual Delegate Parameter Menus* festgelegt. Dieses Menü dient sowohl der Auswahl der gewünschten Visual Delegates als auch der Konfiguration

verschiedener Trainingsparameter. Die dort ausgewählten Visual Delegates werden anschließend in einer Liste an den *Visual Delegate Manager* übergeben und zusätzlich instanziiert oder aktiviert.

Ein weiterer wichtiger Bestandteil der Architektur ist der *Research Data Collector*. Dieses Skript ist für die wissenschaftliche Datenerfassung zuständig. Zu Beginn eines Trainingsdurchlaufs erstellt es eine neue CSV-Datei und speichert während des Trainings relevante Ereignisse wie Fokusverluste, Refokussierungen sowie Trainingsparameter und die dazugehörigen Zeitstempel. Die erzeugten CSV-Dateien werden anschließend auf der genutzten Hardware gespeichert, sodass diese später wissenschaftlich ausgewertet werden können.

Ergänzend dazu existiert ein separater *Training Timer*, der die Dauer des Trainings verwaltet. Dieser wird über das Konfigurationsmenü gestartet und stellt Informationen über die bereits vergangene Trainingszeit bereit. Dadurch lassen sich Zeitstempel und Trainingsereignisse korrekt miteinander verknüpfen.

Insgesamt verfolgt die dargestellte Architektur das Ziel, die verschiedenen Verantwortungsbereiche der Anwendung klar voneinander zu trennen. Dadurch entsteht eine flexible und erweiterbare Struktur, die sowohl die Integration neuer Visual Delegates als auch die spätere Nutzung der Anwendung im wissenschaftlichen Kontext erleichtert.

4.3 Beispielhafter Studienablauf

Wenn Forschende eine Studie mit UPAT durchführen möchten, lässt sich der Ablauf grundsätzlich in zwei Phasen unterteilen: die Vorbereitungs- und die eigentliche Durchführungsphase. Das in Abbildung 12 dargestellte Konzept visualisiert die Vorbereitungsphase, insbesondere die vorgesehenen Interaktionsmöglichkeiten der Forschenden mit der Anwendung.

Zu Beginn der Vorbereitungsphase wird das Git-Repository der Anwendung geklont, um es anschließend im Unity-Editor zu öffnen. Innerhalb des Editors können Forschende die Anwendung anschließend an die Anforderungen ihrer jeweiligen Studie anpassen.

Dabei bestehen verschiedene Anpassungsmöglichkeiten. Zum einen können neue Visual Delegates implementiert werden. Hierfür ist innerhalb der Softwarearchitektur ein entsprechender Extension Point vorgesehen, der es ermöglicht, neue Visual Delegates modular in die bestehende Anwendung zu integrieren. Zum anderen können bestehende Parameter der Visual Delegates angepasst werden, beispielsweise die Intensität, die Einblenddauer oder die Geschwindigkeit der Effekte.

Zusätzlich lassen sich verschiedene Trainingsparameter anpassen. Dazu zählen unter anderem die Geduld des virtuellen Patienten, die Trainingsdauer sowie die Kombination der verwendeten Visual Delegates. Durch diese Konfigurationsmöglichkeiten soll gewährleistet werden, dass Forschende die Anwendung flexibel an unterschiedliche Studiendesigns und Forschungsfragen anpassen können.

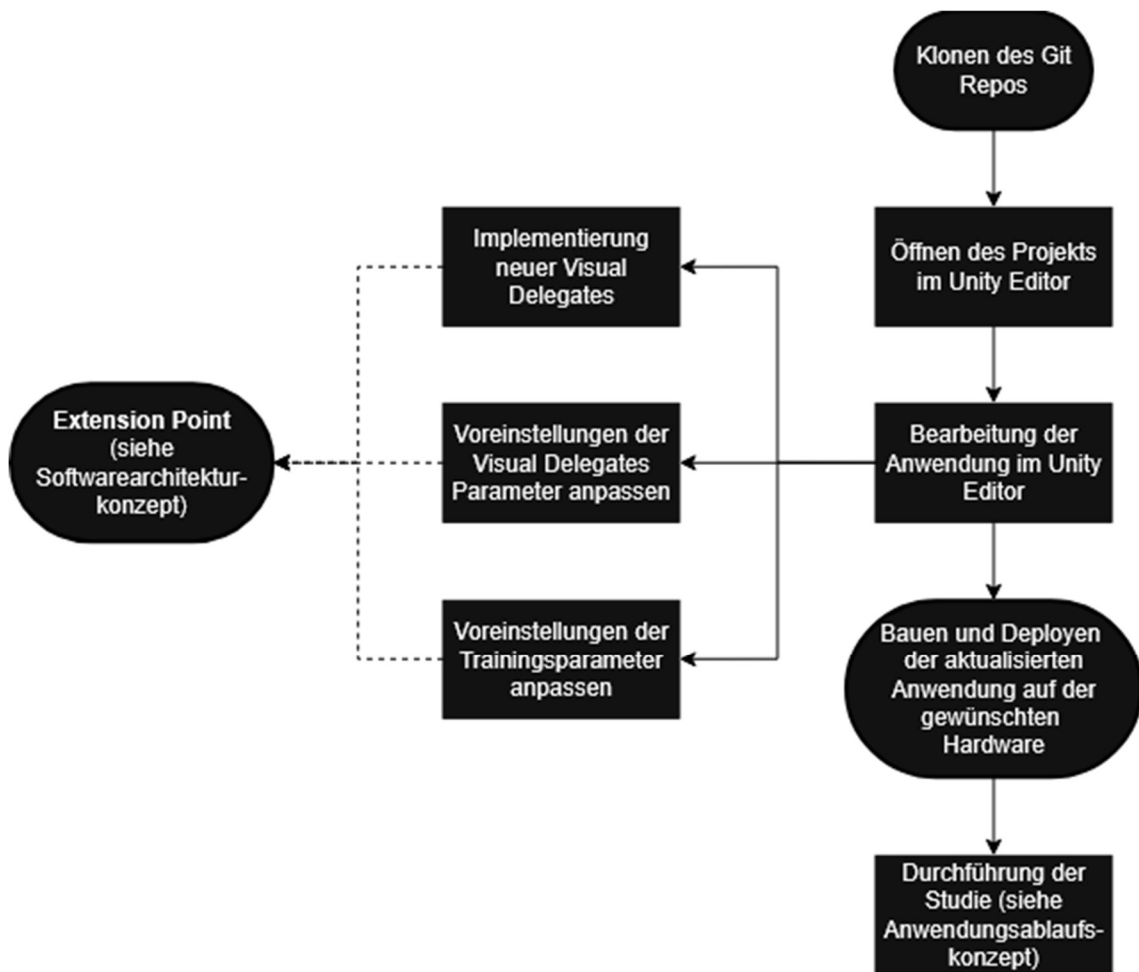


Abbildung 12: Flowchart der Studienvorbereitung

Nach Abschluss der Anpassungen wird eine APK (Android Package Kit) für die aktualisierte Anwendung gebaut und auf der gewünschten Zielhardware deployt. Anschließend kann die eigentliche Durchführung der Studie erfolgen (vgl. Abbildung 10), in deren Rahmen die Probanden das Aufmerksamkeitstraining absolvieren und die definierten Trainings- und Aufmerksamkeitsdaten automatisch aufgezeichnet werden.

5 Technische Erweiterung von UPAT

Mit Abschluss des Praxisprojekts lag bereits ein funktionsfähiges VR-basiertes Aufmerksamkeitstraining vor, in dem den Nutzenden direktes Feedback zu ihrem Aufmerksamkeitsverhalten gegeben wird. Auf Grundlage dieses Prototyps wurde anschließend eine technische Erweiterung konzipiert, um die Anwendung gezielt auf den zukünftigen Einsatz im wissenschaftlichen Kontext vorzubereiten.

Hierfür wurden in den vergangenen Kapiteln zunächst verschiedene vergleichbare Anwendungen aus dem Bereich VR-basierter Aufmerksamkeitstrainings analysiert und relevante Funktionen sowie Gestaltungskonzepte herausgearbeitet. Ergänzend dazu erfolgte ein Austausch mit einem Forschenden, der die Anwendung nach Abschluss der

Erweiterung im Rahmen eigener wissenschaftlicher Untersuchungen einsetzen soll. Gemeinsam wurde dabei eine Liste gewünschter Funktionen und Anforderungen erarbeitet, die im Rahmen der technischen Erweiterung der Anwendung umgesetzt werden sollen.

Darüber hinaus wurden sowohl ein Softwarearchitektur- als auch ein Anwendungskonzept entwickelt, die während der technischen Erweiterung als Orientierung dienten. Diese Konzepte definieren die grundlegende Struktur der Anwendung, die Kommunikation zwischen den einzelnen Systemen sowie die geplanten Erweiterungsmöglichkeiten.

Im Folgenden wird die technische Erweiterung von UPAT detailliert beschrieben. Dabei wird insbesondere auf die konkrete Umsetzung der zuvor konzipierten Systeme, Prozesse und Funktionen eingegangen und erläutert, wie die im Konzept beschriebenen Anforderungen technisch umgesetzt wurden. Der gesamte Quellcode lässt sich dabei unter Anhang A2 finden.

5.1 Visual Delegates

Ein besonders wichtiger Bestandteil des Trainings sind die Visual Delegates, die dem Nutzer direktes Feedback zu seinem Aufmerksamkeitsverhalten geben. Während des Praxisprojekts wurden bereits drei verschiedene Visual Delegates implementiert, allerdings noch in stark prototypischer Form. Für jedes einzelne Visual Delegate war dabei ein separates Skript erforderlich, obwohl sich die zugrunde liegende Logik der Effekte größtenteils ähnelte. Dies führte zu mehrfachen Code-Duplikationen und erschwerte die Erweiterbarkeit der Anwendung.

Im Rahmen der technischen Erweiterung sollen die zugrunde liegenden Skripte und Systeme daher so angepasst werden, dass neue Visual Delegates deutlich einfacher integriert werden können. Ziel ist es, eine modulare Architektur zu schaffen, in der gemeinsame Funktionalitäten an einer Schnittstelle verwaltet werden und neue Effekte lediglich ihre spezifischen Eigenschaften implementieren müssen.

Dadurch soll erreicht werden, dass Forschende zukünftige Visual Delegates mit möglichst geringem zusätzlichem Programmieraufwand im Unity-Editor ergänzen können, ohne bestehende Systeme oder Skripte verändern zu müssen. So soll die Grundlage für eine langfristig erweiterbare Forschungsplattform geschaffen werden.

5.1.1 Visual Delegate Basis

Um diese Erweiterbarkeit der Anwendung zu gewährleisten und Codeduplizierungen möglichst vermeiden zu können, wurde eine Visual-Delegate-Basisklasse eingeführt. Diese basiert auf den in Unity vorhandenen *Scriptable Objects*, die eine spezielle Datenstruktur zur Speicherung und Verwaltung von Daten darstellen.

Scriptable Objects ermöglichen es dabei, Daten unabhängig von Szenenobjekten zu speichern. Zudem lassen sich auf diese Weise generelle, blaupausenartige Datentypen

erstellen, bei denen einzelne Werte je nach Anwendungsfall individuell angepasst werden können. Diese Logik wird in herkömmlichen Videospielen beispielsweise häufig zur Erstellung verschiedener Waffentypen verwendet. Dabei kann dieselbe grundlegende Waffenlogik wiederverwendet werden, während einzelne Eigenschaften wie Schaden, Reichweite oder Feuerrate pro Waffentyp individuell konfiguriert werden können.

In der vorliegenden Anwendung wird dieses Konzept auf Visual Delegates übertragen. Dadurch lassen sich auf Basis derselben Grundstruktur verschiedene Visual Delegates erstellen, während einzelne Eigenschaften individuell angepasst werden können. Dazu zählen beispielsweise der Name des Visual Delegates, die Einordnung innerhalb des VD-Frames sowie der Start- und Zielwert des Parameters, der sich während eines Fokusverlusts verändert.

Die entsprechenden Parameter wurden so angelegt, dass sie direkt im Unity-Editor sichtbar und editierbar sind. Dadurch wird eine intuitive Konfiguration der einzelnen Visual Delegates ermöglicht, ohne zusätzlichen Code schreiben oder den bestehenden Code anpassen zu müssen.

Abbildung 13 zeigt die grundlegende Ansicht eines solchen Visual Delegates im Unity-Editor. Innerhalb dieser Ansicht lassen sich die dargestellten Eigenschaften des Visual Delegates beliebig anpassen.

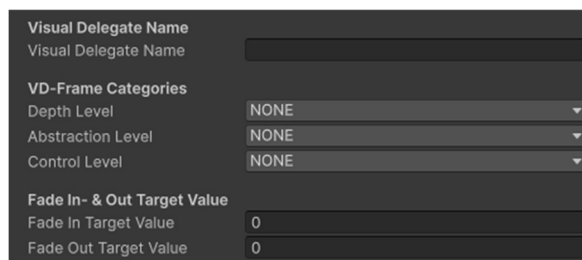


Abbildung 13: Abbildung eines Visual Delegate Scriptable-Objekts im Unity Editor.

Zudem werden in der Visual-Delegate-Basisklasse bereits alle grundlegenden Funktionen deklariert, die für die Implementierung neuer Visual Delegates erforderlich sind. Dadurch stehen diese Funktionen bei der Erstellung neuer Effekte direkt zur Verfügung und müssen nicht erneut implementiert werden.

Eine Ausnahme bildet die Methode *ChangeParameterValueOverTime()*. Diese wird nicht nur deklariert, sondern bereits vollständig in der Basisklasse implementiert, da ihre grundlegende Funktionsweise für alle Visual Delegates identisch ist. Die Methode wird vom Visual Delegate Manager aufgerufen, der im weiteren Verlauf noch detaillierter beschrieben wird.

Innerhalb der Methode wird geprüft, ob ein Visual Delegate eingeblendet oder ausgeblendet werden soll. Abhängig davon wird entweder der definierte Fade-In-Zielwert oder der Fade-Out-Zielwert an den Visual Delegate Manager übergeben. Dieser übernimmt anschließend die eigentliche zeitliche Änderung des jeweiligen Parameters. Auf diese Weise bleibt die Logik zur Auswahl des Zielwerts innerhalb des Visual

Delegates, während die konkrete Fade-In- und Fade-Out-Steuerung über den Visual Delegate Manager erfolgt.

Durch diesen Ansatz wird die Logik der Visual Delegates deutlich modularer aufgebaut. Gleichzeitig können neue Visual Delegates direkt im Unity-Editor angelegt und konfiguriert werden, ohne dass bestehende Systeme angepasst oder redundante Skripte erstellt werden müssen.

Die Visual-Delegate-Basisklasse bildet die unterste Ebene der gesamten Visual-Delegate-Architektur und definiert alle Eigenschaften und Funktionen, die jedes Visual Delegate grundsätzlich besitzen muss. Dazu zählen beispielsweise allgemeine Informationen wie der Name des Visual Delegates, die Einordnung innerhalb des VD-Frames sowie grundlegende Parameter zur Steuerung der Effekttintensität.

Da Visual Delegates jedoch auf unterschiedliche Weise innerhalb von Unity umgesetzt werden können, beispielsweise als Post-Processing-Effekte, UI-Elemente oder animationsbasierte Systeme, ist es sinnvoll, zusätzliche spezialisierte Subklassen einzuführen. Diese sollen die Integration bestimmter Arten von Visual Delegates vereinfachen und gemeinsame Funktionen für die jeweiligen Effektkategorien bereitstellen.

Die entsprechenden Subklassen sowie deren Aufgabenbereiche werden im folgenden Unterkapitel näher erläutert.

5.1.2 Visual Delegate Subklassen

Aufbauend auf der Visual-Delegate-Basisklasse lassen sich Visual Delegates in verschiedene Unterkategorien einteilen. So basieren beispielsweise alle innerhalb des Aufmerksamkeitstrainings verwendeten Visual Delegates auf dem Post-Processing-System von Unity. In Zukunft sollen jedoch auch andere Arten von Visual Delegates, beispielsweise UI- oder animationsbasierte Effekte, integriert werden können. Aus diesem Grund wurde eine zusätzliche Abstraktionsebene zwischen der allgemeinen Visual Delegate Basisklasse und dem eigentlichen konkreten Visual Delegate eingeführt.

Hierzu wurde eine eigene Post-Processing-Visual-Delegate-Klasse für die bereits vorhandenen Post-Processing-Effekte entwickelt. Diese basiert auf der zuvor beschriebenen Visual Delegate Basisklasse und ermöglicht es, die dort definierten Methoden zu überschreiben und um spezifische Funktionalitäten für Post-Processing-Effekte zu erweitern. Dadurch entsteht eine Art Blaupause für alle Visual Delegates, die auf dem Unity Post-Processing-System basieren.

In dieser Klasse befinden sich insbesondere die Eigenschaften und Funktionen, die alle Post-Processing-Visual-Delegates gemeinsam besitzen. Dazu gehört vor allem das Auffinden des globalen Post-Processing-Volumes innerhalb der Unity-Szene sowie das Hinzufügen und Entfernen der jeweiligen Post-Processing-Effekte innerhalb dieses Volumes.

Diese Funktionalität wird insbesondere über die Methoden *GetVolume()* und *EnsureEffect()* umgesetzt. Innerhalb der *GetVolume()*-Methode sucht das Skript zunächst nach dem globalen Volume in der Szene. Dies erfolgt mithilfe des zuvor definierten *PostProcessing*-Tags. Sobald das entsprechende Volume gefunden wurde, kann innerhalb der *EnsureEffect()*-Methode überprüft werden, ob der gewünschte Post-Processing-Effekt bereits auf diesem Volume vorhanden ist oder deaktiviert ist. Falls dies nicht der Fall ist, wird der Effekt automatisch erstellt oder aktiviert.

Darüber hinaus wird innerhalb dieser Klasse die Methode *RemoveVisualDelegate()* vollständig implementiert, da deren grundlegende Funktionsweise bei allen Post-Processing-Visual-Delegates identisch ist. Diese Methode entfernt oder deaktiviert den entsprechenden Post-Processing-Effekt wieder aus dem globalen Volume.

Die beschriebenen Methoden werden dabei direkt über die Interaktion im Startmenü genutzt. Fügt der Nutzer dort ein Visual Delegate dem Training hinzu, wird die *AddVisualDelegateToTraining()*-Methode aufgerufen, die wiederum sowohl die *EnsureEffect()*- als auch die *ApplySettings()*-Methode aufruft und so den entsprechenden Effekt innerhalb des Post-Processing-Volumes aktiviert und die spezifischen Einstellungen des Visual Delegates auf dieses anwendet. Entfernt der Nutzer das Visual Delegate wieder aus der Auswahl, wird entsprechend die *RemoveVisualDelegate()*-Methode ausgelöst.

Die Post-Processing-Visual-Delegate-Klasse wurde als generische Klasse implementiert. Dadurch kann die allgemeine Logik zum Hinzufügen, Entfernen und Verwalten von Post-Processing-Effekten in dieser Klasse definiert werden. Bei der Implementierung eines konkreten Visual Delegates muss anschließend lediglich der generische *T*-Operator durch den entsprechenden Post-Processing-Effekt ersetzt werden. Dadurch reduziert sich der notwendige Code in den eigentlichen Visual-Delegate-Klassen erheblich.

Die eingeführte Architektur ermöglicht darüber hinaus, künftig weitere spezialisierte Unterkategorien hinzuzufügen. Forschende könnten auf diese Weise beispielsweise eigene Klassen für UI- oder animationsbasierte Visual Delegates implementieren. Diese würden sich ähnlich wie die Post-Processing-Klasse verhalten und Aufgaben der jeweiligen Effektart übernehmen. Eine UI-basierte Subklasse könnte beispielsweise automatisch den relevanten UI-Canvas verwalten, während eine animationsbasierte Subklasse die Steuerung entsprechender Animator-Komponenten übernehmen würde.

Aufbauend auf diesen Subklassen wird anschließend das eigentliche konkrete Visual Delegate implementiert, das nur noch die spezifische Logik des jeweiligen Effekts enthalten muss.

5.1.3 Specific Visual Delegate

Innerhalb des eigentlichen spezifischen Visual-Delegate-Skripts werden anschließend die effektspezifische Logik sowie die Definition der gewünschten Parameter umgesetzt.

Anhand der bereits im Training vorhandenen Vignette-Visual-Delegates lässt sich dieser Aufbau beispielhaft erläutern.

Da es sich beim Vignette-Visual-Delegate um ein Post-Processing-basiertes Visual Delegate handelt, basiert dieses auf der zuvor beschriebenen *Post-Processing-Visual-Delegate*-Klasse und ersetzt dabei den generischen *T*-Operator durch den konkreten Unity-Post-Processing-Vignette-Effekt. Bei den beiden weiteren Visual Delegates, die bereits im Training verwendet werden, dem *Film Grain Visual Delegate* sowie dem *Grey Filter Visual Delegate*, wird der generische Typ entsprechend durch *FilmGrain* beziehungsweise *ColorAdjustments* ersetzt.

Zusätzlich muss jedem konkreten Visual Delegate das *CreateAssetMenu*-Attribut hinzugefügt werden. Dadurch kann das jeweilige Visual Delegate direkt im Unity-Editor als Scriptable Object erstellt werden, wie in Abbildung 14 dargestellt.

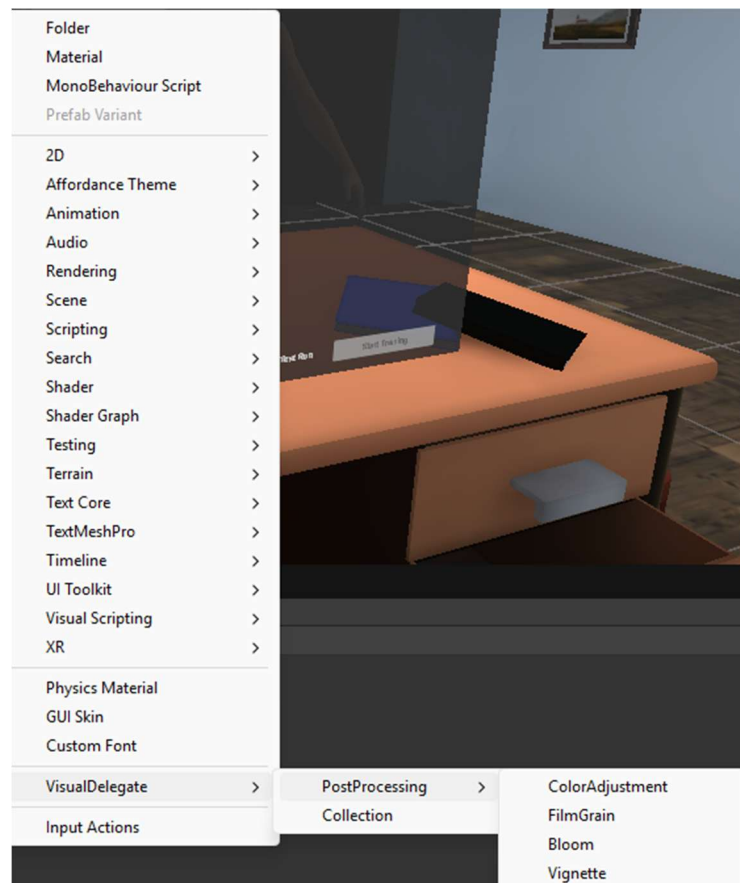


Abbildung 14: Das eingeblendete Kontextmenü in Unity über das sowohl einzelne Visual Delegates, als auch Visual Delegate Collections erstellt werden können.

Im Anschluss werden die gewünschten Parameter des jeweiligen Visual Delegates definiert und, sofern gewünscht, im Unity-Editor sichtbar gemacht. Beim *Vignette Visual Delegate* zählen hierzu beispielsweise die Farbe der Vignette, die Position des Effektzentrums, die Intensität, die Weichheit des Randes sowie die Form der Vignette. Diese Parameter werden anschließend innerhalb der *ApplySettings()*-Methode den tatsächlichen Parametern des Unity-Post-Processing-Effekts zugewiesen.

Da sich die verfügbaren Parameter zwischen den einzelnen Post-Processing-Effekten unterscheiden, kann diese Zuordnung nicht bereits innerhalb der allgemeinen Post-Processing-Subklasse erfolgen, sondern muss individuell in jedem konkreten Visual Delegate implementiert werden.

Zusätzlich wird innerhalb der Methoden *GetChangingParameter()* und *SetChangingParameter()* definiert, welcher Parameter während eines Fokusverlusts dynamisch geändert werden soll. Beim Vignette-Visual-Delegate betrifft dies beispielsweise den *Intensity*-Parameter, wodurch sich die Vignette während eines Aufmerksamkeitsverlusts verstärken beziehungsweise nach einer Refokussierung wieder reduzieren lässt.

Sobald diese Logik implementiert ist, kann das Visual Delegate über das in Abbildung 14 dargestellte Menü direkt in Unity erstellt werden. Die definierten Parameter lassen sich anschließend komfortabel im Unity-Editor anpassen, was eine schnelle und intuitive Konfiguration ermöglicht.

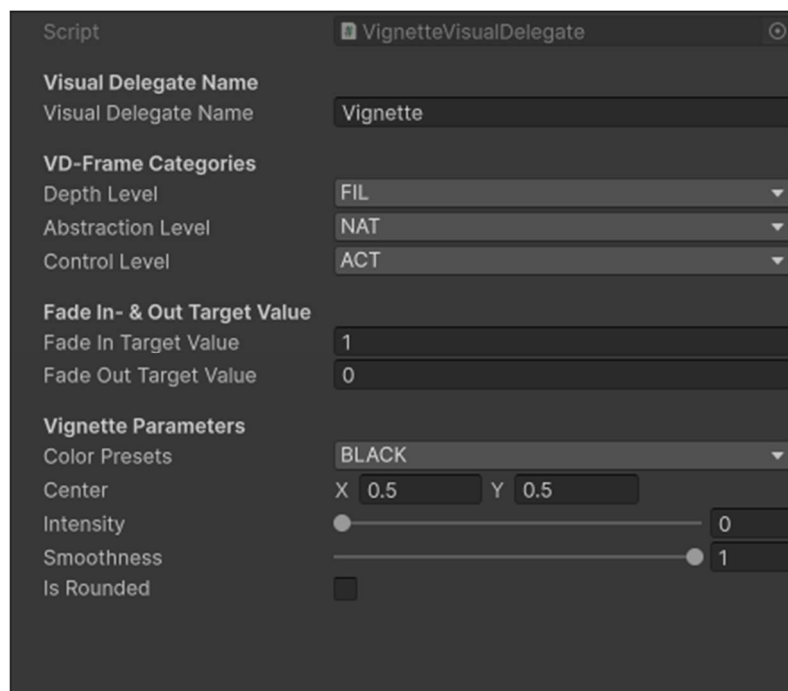


Abbildung 15: Ansicht des Vignette-Visual-Delegates im Unity-Editor.

Abbildung 15 zeigt eben diese Editoransicht des Vignette Visual Delegates. Neben den bereits innerhalb der Visual Delegate Base Class definierten Eigenschaften, wie dem Namen des Visual Delegates oder der Einordnung innerhalb des VD-Frames, lassen sich dort auch sämtliche effektspezifischen Parameter konfigurieren.

Durch diesen mehrstufigen Aufbau sowie das Zusammenspiel der verschiedenen Abstraktionsebenen wird sichergestellt, dass neue Visual Delegates mit möglichst geringem zusätzlichem Programmieraufwand erstellt werden können. Gleichzeitig ermöglicht die Architektur es Forschenden, zukünftig auch vollständig neue Arten von Visual Delegates in das Aufmerksamkeitstraining zu integrieren.

Die Kontrolle der erstellten Visual Delegates erfolgt über einen *Visual Delegate Manager*. Dieser übernimmt die Verwaltung, Aktivierung und Deaktivierung der aktuell im Training verwendeten Visual Delegates und fungiert somit als Schnittstelle zwischen der Aufmerksamkeitserfassung und den eigentlichen visuellen Effekten. Die genaue Funktionsweise sowie die technische Umsetzung des Visual Delegate Managers werden daher im folgenden Abschnitt detailliert erläutert.

5.1.4 Visual Delegate Manager

Der *Visual Delegate Manager* übernimmt die Steuerung der im Training eingesetzten Visual Delegates. Innerhalb des Skripts werden zunächst die relevanten Trainingsparameter definiert. Dazu zählen der Zeitraum, der vergehen muss, bevor Visual Delegates nach einem Fokusverlust aktiviert werden, sowie die Dauer des Fade-Ins und des Fade-Outs. Diese sind auch im Unity-Editor sichtbar und anpassbar, wie in Abbildung 16 dargestellt. Zusätzlich enthält der Manager eine Liste aller Visual Delegates, die im aktuellen Trainingsdurchlauf verwendet werden.

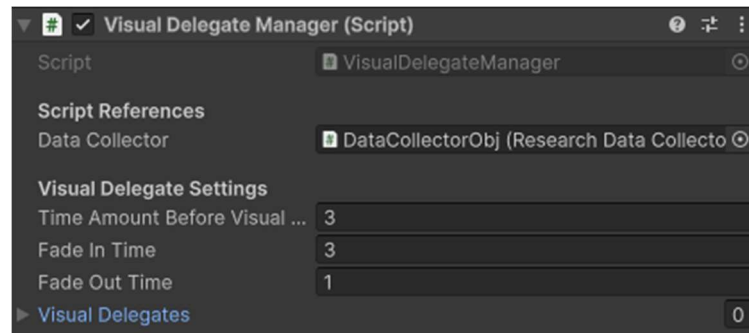


Abbildung 16: Ansicht des Visual Delegate Manager Skripts im Unity Editor.

Die Steuerung der Visual Delegates basiert dabei auf der *GazeSelect()* und der *GazeUnselect()* Methode. Wird der virtuelle Patient nicht mehr fokussiert, wird über *GazeUnselect()* ein Timer gestartet. Gleichzeitig werden die Fokusverlustdaten, wie die Fokusverlustnummer und der dazugehörige Zeitstempel, an den *ResearchDataCollector* übermittelt. Läuft der Timer vollständig ab, wird auf allen aktiven Visual Delegates die *ChangeParameterValueOverTime()* Methode ausgeführt.

Sobald der Nutzer den virtuellen Patienten erneut fokussiert, wird die *GazeSelect()*-Methode aufgerufen. Dabei werden laufende Routinen gestoppt, der Timer zurückgesetzt und die Refokussierungszeit berechnet. Die Refokussierungszeit beschreibt dabei, wie lange der Nutzer benötigt hat, um den Patienten nach einem Fokusverlust wieder anzuschauen. Auch dieser Wert wird an den *ResearchDataCollector* übergeben. Anschließend werden die Visual Delegates wieder ausgeblendet.

Die eigentliche Veränderung der Effektintensität erfolgt über *StartVisualDelegateFade()* und die zugehörige *FadeRoutine()*. Dabei wird der aktuelle Parameterwert eines Visual Delegates ausgelesen und über die definierte Dauer hinweg schrittweise an den Zielwert angenähert. Auf diese Weise lassen sich verschiedene Visual Delegates über ein Skript

steuern und unabhängig von ihrer konkreten technischen Umsetzung ein- oder ausblenden.

5.1.5 Visual Delegate Collections

Bereits während des Praxisprojekts wurden verschiedene Visual Delegates miteinander kombiniert, um den gewünschten Einfluss auf das Nutzerverhalten zu erzielen. Mit den bisher in diesem Kapitel beschriebenen Skripten lassen sich bereits einzelne Visual Delegates erstellen und anschließend im Start-Menü zum Training hinzufügen. Diese Lösung wurde zunächst auch in dieser Form umgesetzt.

Im Rahmen einer späteren Feedbackrunde mit dem Forschenden, der die Anwendung zukünftig nutzen soll, stellte sich jedoch heraus, dass es praktischer wäre, häufig verwendete Kombinationen von Visual Delegates bereits direkt im Unity-Editor zusammenzustellen und anschließend im Start-Menü lediglich aus diesen vorbereiteten Kombinationen auswählen zu können.

Aus diesem Grund wurde eine zusätzliche Art von Scriptable Object entwickelt, die sogenannte *Visual Delegate Collection*. Hierbei handelt es sich um ein einfaches Scriptable Object, das als Sammlung mehrerer Visual Delegates dient. Forschende können diese Sammlungen, ähnlich wie einzelne Visual Delegates, direkt über das in Abbildung 14 dargestellte Kontextmenü in Unity erstellen.

Eine Visual Delegate Collection besitzt dabei grundsätzlich einen Namen, eine kurze Beschreibung sowie eine Liste aller zugehörigen Visual Delegates. Sämtliche dieser Eigenschaften lassen sich direkt im Unity-Editor konfigurieren, wie in Abbildung 17 dargestellt.

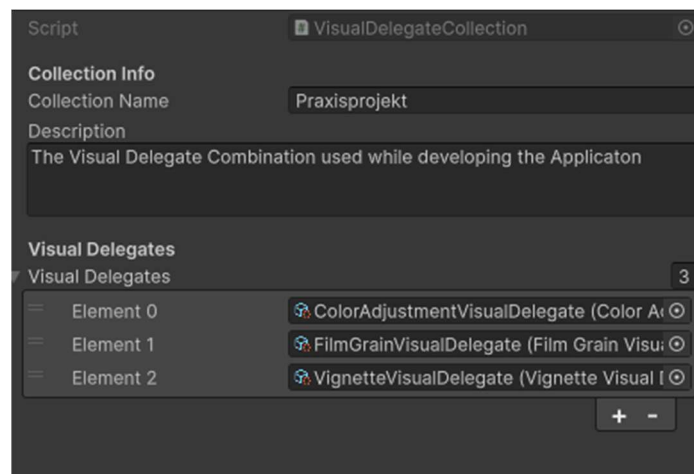


Abbildung 17: Darstellung der im Praxisprojekt genutzten Visual Delegate Collection, bestehend aus Grau-Filter-, Film-Grain- und Vignette-Visual Delegate, im Unity Editor

Durch diesen Ansatz lassen sich häufig genutzte Kombinationen von Visual Delegates organisieren und einfach wiederverwenden. Gleichzeitig wird die Bedienung des Start-Menüs vereinfacht, da Forschende nicht mehr jedes einzelne Visual Delegate separat auswählen müssen, sondern sich direkt auf vorbereitete Effektkombinationen stützen können.

Eine wichtige Voraussetzung für dieses System besteht allerdings darin, dass die Visual Delegate Collection Scriptable Objects im Resources-Ordner von Unity abgelegt werden müssen. Dies ist notwendig, da das Start-Menü die vorhandenen Collections automatisch lädt und hierfür auf das Unity-*Resources*-System zurückgreift.

Um die Struktur im Projekt übersichtlicher zu gestalten, wurde im Resources-Ordner ein zusätzlicher Unterordner namens *Visual Delegate Collections* angelegt. In diesem sollten sämtliche erstellten Visual Delegate Collections gespeichert werden.

Das Start-Menü greift anschließend gezielt auf diesen Ordner zu, lädt alle dort vorhandenen Collections automatisch und stellt diese innerhalb der Benutzeroberfläche zur Auswahl bereit.

5.2 Startmenü

Das Startmenü wird direkt nach dem Start der Anwendung eingeblendet. Hierfür wurde innerhalb der Trainingsszene in Unity ein eigener UI-Canvas erstellt, der sämtliche für die Trainingskonfiguration benötigten UI-Elemente enthält. Abbildung 18 zeigt den vollständigen Aufbau dieses Startmenüs.

Im Verlauf dieses Kapitels werden die einzelnen UI-Elemente sowie die jeweils dahinterliegende programmierte Logik schrittweise erläutert.

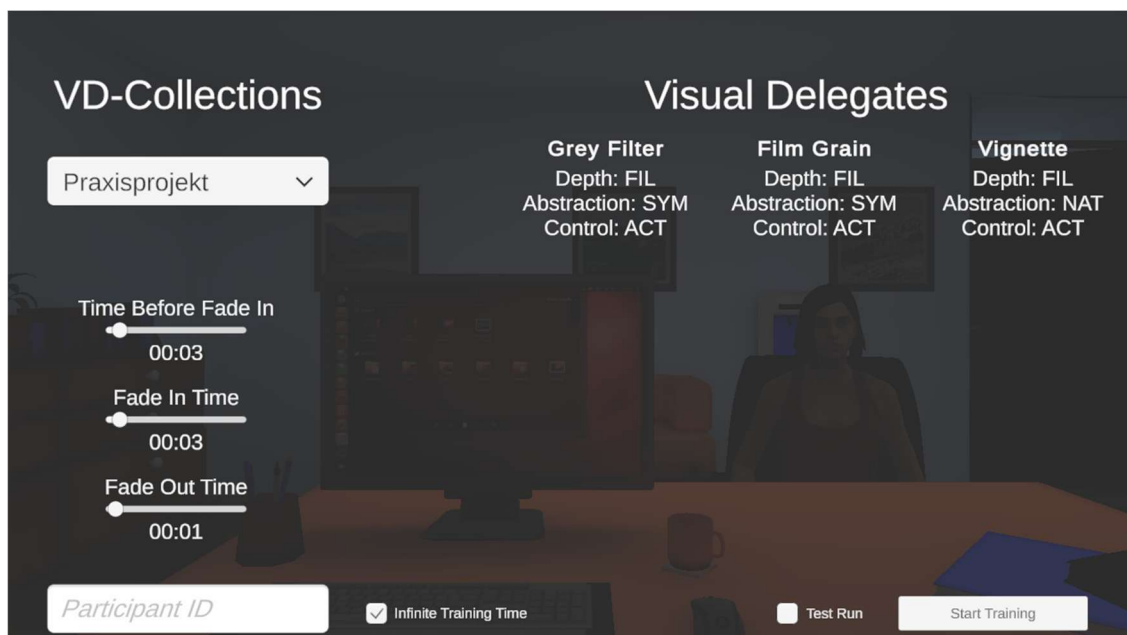


Abbildung 18: Darstellung des Startmenüs. In diesem können Visual Delegate Collections ausgewählt werden, sowie verschiedene Trainingsparameter eingestellt werden.

5.2.1 Visual Delegate Collection Dropdown Menü

In der oberen linken Ecke des Canvas befindet sich das *Visual Delegate Collection*-Dropdown-Menü. Über dieses kann ausgewählt werden, welche Visual-Delegate-Collection im Training verwendet werden soll. Zur Umsetzung wurde die in Unity vorhandene UI-*Dropdown*-Komponente verwendet, die anschließend automatisch über

das *VisualDelegateParameterMenu*-Skript, genauer gesagt über die Methode *PopulateDropdown()*, mit allen verfügbaren Visual-Delegate-Collections befüllt wird.

Hierfür werden zunächst alle Objekte des Typs *VisualDelegateCollection* aus dem entsprechenden *Visual Delegate Collections*-Ordner geladen. Dazu wird die in Unity integrierte Methode *Resources.LoadAll()* verwendet. Diese ermöglicht es, Assets unterschiedlichster Art direkt aus dem *Resources*-Ordner zu laden, auch nachdem die Anwendung bereits gestartet wurde.

Aus diesem Grund müssen die zuvor erstellten Visual-Delegate-Collections innerhalb dieses *Resources*-Ordners gespeichert werden, damit diese zur Laufzeit von der Anwendung gefunden und automatisch in das Dropdown-Menü eingetragen werden können.

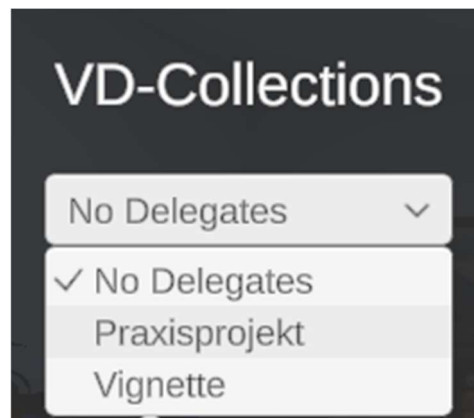


Abbildung 19: Darstellung des Visual Delegate Collections Dropdown Menü's.

Nachdem alle verfügbaren Collections geladen wurden, werden deren Namen automatisch als auswählbare Optionen im Dropdown-Menü angezeigt.

5.2.2 Darstellung der Visual Delegates in der ausgewählten Collection

Wenn der Nutzer eine Visual-Delegate-Collection auswählt, werden alle darin enthaltenen Visual Delegates inklusive ihres Namens sowie ihrer Einordnung innerhalb des VD-Frames auf der rechten Seite des Canvas dargestellt. Die Aktualisierung dieser Ansicht erfolgt über das *OnDropdownChanged()*-Unity-Event, das automatisch ausgelöst wird, sobald innerhalb des Dropdown-Menüs eine neue Auswahl getroffen wird.

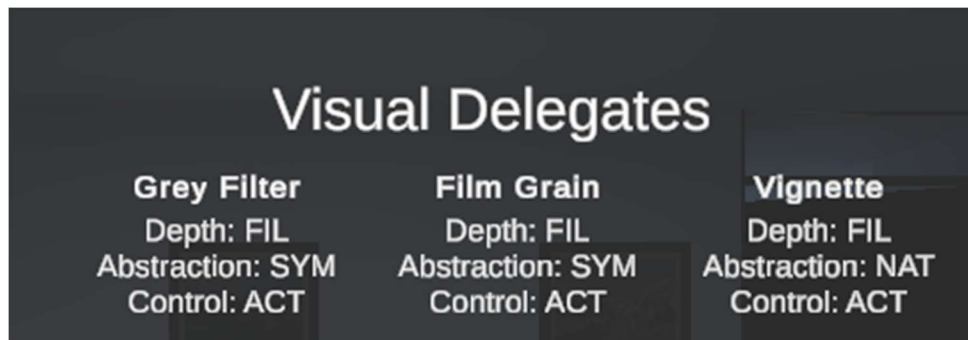


Abbildung 20: Darstellung der verschiedenen Visual Delegates innerhalb der ausgewählten Collection.

Innerhalb dieses Events werden zunächst alle aktuell in der Benutzeroberfläche dargestellten Visual Delegates entfernt, um zu verhindern, dass Visual Delegates verschiedener Collections gleichzeitig angezeigt werden. Anschließend werden die Visual Delegates der neu ausgewählten Visual-Delegate-Collection geladen und im Menü angezeigt.

Hierfür wird über alle Visual Delegates in der Collection iteriert. Für jedes einzelne Visual Delegate wird anschließend ein sogenanntes *Visual Delegate Item* erzeugt. Dabei handelt es sich um ein zuvor erstelltes Prefab, das aus einem UI-Panel mit zwei Text-Elementen besteht (vgl. Abbildung 19).

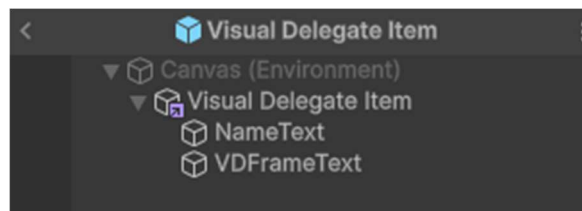


Abbildung 21: Aufbau des Visual Delegate Item Prefabs im Unity-Editor.

Die beiden Textfelder dienen dabei zur Darstellung des Namens des jeweiligen Visual Delegates sowie dessen Einordnung innerhalb des VD-Frames. Dadurch erhalten Forschende bereits vor dem Start des Trainings eine direkte Übersicht darüber, welche Visual Delegates in der ausgewählten Collection enthalten sind und welchen Kategorien sie zugeordnet werden. Diese Text-Objekte werden dabei automatisch durch ein eigenes Skript mit dem Namen des Visual Delegates sowie mit der Einordnung des Visual Delegates im VD-Frame befüllt. Dieses sogenannte *DelegateUIItem*-Skript bekommt dabei das jeweilige Visual Delegate übergeben und füllt die Textfelder mit den im Visual Delegate hinterlegten Informationen.

Durch die Verwendung eines DelegateUIItem-Prefabs kann die Darstellung der Visual Delegates automatisch erzeugt werden, ohne dass die einzelnen UI-Elemente manuell im Canvas angelegt werden müssen. Ergänzend dazu wird eine *Grid Layout Group*-Komponente verwendet, die die Anordnung der dargestellten Visual Delegates automatisch organisiert.

Dadurch können beliebig viele Visual Delegates in der Benutzeroberfläche dargestellt werden, ohne die Positionierung der einzelnen UI-Elemente händisch anpassen zu

müssen. Diese werden dabei automatisch innerhalb des vorgegebenen Rasters angeordnet und gleichmäßig positioniert. Dies erleichtert insbesondere die Erweiterbarkeit der Anwendung, da so auch zukünftig erstellte Visual Delegates automatisch korrekt dargestellt werden, sobald diese einer Visual-Delegate-Collection hinzugefügt werden.

5.2.3 Trainingsparameter-Einstellungen

Unterhalb des *Visual Delegate Collection*-Dropdown-Menüs befinden sich mehrere Schieberegler zur Konfiguration verschiedener Zeitparameter des Trainings. Über diese lassen sich die Geduldszeit des virtuellen Patienten sowie die Fade-In- und Fade-Out-Dauer der innerhalb der ausgewählten Collection enthaltenen Visual Delegates anpassen. Diese werden in Unity über UI-Slider-Elemente umgesetzt.

Auch hier wird das Unity-Event-System verwendet. Sobald ein Schieberegler bewegt wird, wird automatisch ein entsprechendes Event ausgelöst, das den aktualisierten Wert direkt an den *Visual Delegate Manager* weitergibt. Dadurch werden Änderungen unmittelbar innerhalb der Anwendung übernommen, ohne dass zusätzliche Bestätigungsschritte erforderlich sind.



Abbildung 22: Darstellung der Schieberegler, mit denen die einzelnen Zeitparameter des Trainings eingestellt werden können.

Die hier definierten Zeitparameter werden anschließend im Visual Delegate Manager verwendet, um das Verhalten der aktiven Visual Delegates während des Trainings zu steuern. Die Geduldszeit bestimmt dabei, wie lange der virtuelle Patient ignoriert werden darf, bevor Visual Delegates aktiviert werden. Die Fade-In- und Fade-Out-Zeiten legen hingegen fest, wie schnell sich die Effekte ein- beziehungsweise ausblenden.

In der unteren linken Ecke des Canvas befindet sich das Eingabefeld für die *Participant-ID*. Hierfür wird ein Unity-UI-Text-Input-Field verwendet, in das Text eingegeben werden kann. Der eingegebene Wert wird anschließend in einer Variable gespeichert und an den Research Data Collector weitergeleitet, damit die erzeugten CSV-Dateien eindeutig einem bestimmten Studienteilnehmer zugeordnet werden können.

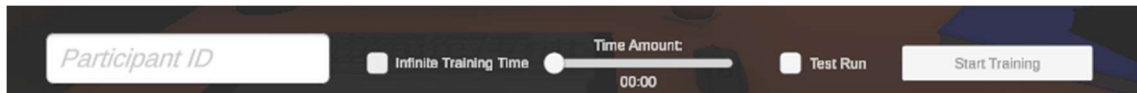


Abbildung 23: Darstellung des unteren Teils des Startmenüs. In diesem lassen sich die Participant-ID, die Trainingszeit, sowie die Art des Durchlaufs einstellen.

Durch die Verwendung eines Text-Input-Feldes wird in der VR-Brille automatisch die virtuelle Tastatur des jeweiligen XR-Systems eingeblendet. Dadurch kann die Participant-ID direkt in der VR-Anwendung eingegeben werden, ohne eine externe physische Tastatur zu benötigen.

Auf diese Weise wird eine möglichst flexible und freie Benennung der Studienteilnehmer ermöglicht. Forschende können dadurch eigene ID-Systeme oder bestehende Studienkennzeichnungen problemlos in der Anwendung verwenden.

Rechts neben dem Eingabefeld für die *Participant-ID* befindet sich ein Schalter, mit dem zwischen einem zeitlich unbegrenzten Modus und einem zeitlich begrenzten Trainingsmodus gewechselt werden kann. Dieser wurde mithilfe eines Unity-UI-*Toggle*-Elements umgesetzt. Wird der Toggle betätigt, wird automatisch eine zuvor definierte Methode ausgeführt.

In der vorliegenden Anwendung handelt es sich um die Methode *InfiniteTimeSwitch()*. Diese überprüft zunächst, welcher Trainingsmodus aktuell aktiv ist. Wird der zeitlich begrenzte Modus ausgewählt, blendet die Methode zusätzlich einen UI-Slider ein, über den die gewünschte Trainingsdauer festgelegt werden kann. Wird hingegen der unbegrenzte Modus aktiviert, wird dieser Slider ausgeblendet, da in diesem Fall keine feste Trainingsdauer erforderlich ist.

Der Slider zur Einstellung der Trainingszeit funktioniert nach demselben Prinzip wie die zuvor beschriebenen Schieberegler. Änderungen am Wert werden über das Unity-Event-System direkt verarbeitet und, in diesem Fall, an den *Trainingstimer* der Anwendung weitergegeben.

Auf der rechten Seite des Menüs befindet sich zusätzlich ein *Test-Run*-Toggle-Switch, mit dem festgelegt werden kann, ob der aktuelle Trainingsdurchlauf ein Testlauf ist. Auch dieser wurde mithilfe eines Unity-UI-*Toggle*-Elements umgesetzt.

Wird der Testmodus aktiviert, wird die zuvor eingegebene *Participant-ID* automatisch durch den Text „TEST“ ersetzt. Dadurch lassen sich Testdurchläufe später in den erzeugten CSV-Dateien sowie bei der wissenschaftlichen Auswertung eindeutig von regulären Studiendurchläufen unterscheiden.

5.2.4 Start-Training Button

Am unteren rechten Rand des Canvas befindet sich der *Start-Training-Button*. Wird dieser vom Nutzer betätigt, wird die Methode *StartUserTesting()* aufgerufen. Diese übernimmt die Initialisierung des eigentlichen Trainingsdurchlaufs.

Zunächst werden alle zum Startmenü gehörenden Canvas-Elemente ausgeblendet, um den Fokus vollständig auf die Trainingsumgebung zu lenken. Anschließend wird die

Methode *ApplyVisualDelegates()* aufgerufen. Diese iteriert über alle Visual Delegates in der ausgewählten Visual-Delegate-Collection und führt auf jedem einzelnen die *AddVisualDelegateToTraining()*-Methode aus. Dadurch werden sämtliche ausgewählten Visual Delegates inklusive ihrer zuvor konfigurierten Parameter dem aktuellen Trainingsdurchlauf hinzugefügt und initialisiert.

Dabei wird auch eine Liste aller aktiven Visual Delegates an den *Visual Delegate Manager* übergeben. Dieser nutzt die Liste während des Trainings, um ausschließlich die aktuell ausgewählten Visual Delegates zu steuern und deren Intensität abhängig vom Aufmerksamkeitsverhalten des Nutzers anzupassen.

Darüber hinaus wird innerhalb der *StartUserTesting()*-Methode die *Time Scale* der Szene von 0 auf 1 gesetzt. Der Wert 1 entspricht dabei dem Unity-Standard und sorgt dafür, dass die Zeit innerhalb der Szene normal weiterläuft. Dadurch werden zeitbasierte Prozesse wie Animationen, Coroutines oder die Timerlogik des Visual Delegate Managers erst ab dem Start des Trainings aktiviert. Auf diese Weise soll verhindert werden, dass Timer bereits während der Konfigurationsphase des Trainings ablaufen.

Parallel dazu erhält der Research Data Collector auch sämtliche innerhalb des Startmenüs definierten Trainingsparameter. Basierend auf diesen Informationen wird automatisch eine neue CSV-Datei erstellt, in der sowohl die gewählten Einstellungen als auch die während des Trainings aufgezeichneten Aufmerksamkeitsdaten gespeichert werden.

5.3 Research Data Collector

Der *Research Data Collector* ist für die automatisierte Erfassung und Speicherung der während eines Trainingsdurchlaufs entstehenden Forschungsdaten verantwortlich. Beim Start der Anwendung wird zunächst im *Application.persistentDataPath* ein eigener Speicherordner angelegt, sofern dieser noch nicht existiert. Dadurch werden die erzeugten CSV-Dateien an einem persistenten Speicherort abgelegt und bleiben so auch nach dem Beenden der Anwendung verfügbar.

Zu Beginn eines Trainingsdurchlaufs wird mithilfe der Methode *CreateCSV()* eine neue CSV-Datei erstellt. Der Dateiname besteht aus der Participant-ID, dem Namen der ausgewählten Visual-Delegate-Collection und einem fortlaufenden Index. Dadurch kann jede Datei eindeutig einem bestimmten Trainingsdurchlauf zugeordnet werden.

In der CSV-Datei werden zunächst die verwendeten Visual Delegates dokumentiert. Dabei werden neben dem Namen auch die jeweilige Einordnung innerhalb des VD-Frames sowie die eingestellten Fade-In- und Fade-Out-Zeiten gespeichert. Anschließend wird ein separater Bereich für Trainingsereignisse angelegt. Dort werden Fokusverluste, Refokussierungszeiten sowie die Zeitabstände zwischen den einzelnen Fokusverlusten protokolliert.

Die Methode *RegisterFocusLoss()* wird vom *VisualDelegateManager* aufgerufen, sobald der Nutzer den virtuellen Patienten aus dem Fokus verliert. Dabei wird die Anzahl der

bisherigen Fokusverluste erhöht und der Zeitpunkt des aktuellen Fokusverlusts anhand des *Training-Timers* gespeichert. Zusätzlich wird berechnet, wie viel Zeit seit dem vorherigen Fokusverlust vergangen ist.

Sobald der Nutzer den virtuellen Patienten erneut fokussiert, wird die *RegisterRefocusTime()*-Methode aufgerufen. Diese speichert die Dauer bis zur erneuten Fokussierung, den Start- und Endzeitpunkt des Fokusverlusts sowie den zeitlichen Abstand zum vorherigen Fokusverlust in der CSV-Datei. Die Werte werden dabei mit Zeitstempeln versehen, sodass sie später bestimmten Trainingssituationen zugeordnet werden können.

Am Ende des Trainings wird mithilfe der *FinishCSV()*-Methode eine Zusammenfassung hinzugefügt. Diese enthält die gesamte Trainingszeit, die Gesamtzahl der Fokusverluste sowie die durchschnittliche Refokussierungszeit. Anschließend wird die Datei geschlossen und dauerhaft gespeichert. Zusätzlich wird *FinishCSV()* beim Beenden der Anwendung aufgerufen, um Datenverluste zu vermeiden, falls die Anwendung unerwartet beendet wird. In Anhang A3 lässt sich die Darstellung einer beispielhaften CSV-Datei finden.

5.4 Trainingstimer

Das *TrainingTimer*-Skript verwaltet die zeitliche Steuerung eines Trainingsdurchlaufs. Über die Methode *StartTimer()* wird der Timer mit der zuvor im Startmenü festgelegten Trainingsdauer initialisiert.

Innerhalb der *Update()*-Methode wird die vergangene Trainingszeit kontinuierlich mithilfe von *Time.deltaTime* erhöht. Parallel dazu wird die angezeigte Trainingszeit über *UpdateDisplayedTime()* aktualisiert und im zugehörigen UI-Textfeld im Format Minuten:Sekunden angezeigt. Dadurch kann die aktuell vergangene Trainingszeit während des Trainings innerhalb der Anwendung angezeigt werden. Diese Anzeige erfolgt über einen kleinen Canvas, der hinter dem Nutzer platziert ist. Über diesen Canvas lässt sich die vergangene Trainingszeit einsehen. Zusätzlich bietet er die Möglichkeit, die Anwendung manuell zu beenden, falls während des Trainings technische Probleme auftreten oder der Durchlauf vorzeitig abgebrochen werden muss.

Solange der Timer aktiv ist, wird zusätzlich überprüft, ob die zuvor definierte maximale Trainingsdauer erreicht wurde. Ist dies der Fall, wird die Zeit auf den Maximalwert gesetzt, der Timer deaktiviert und die Methode *TimerFinished()* aufgerufen. In der aktuellen Umsetzung beendet diese Methode die Anwendung über *Application.Quit()*.

Darüber hinaus stellt das Skript über *GetElapsedTime()* die aktuell vergangene Trainingszeit für andere Systeme bereit. Diese Funktion ist insbesondere für den *Research Data Collector* relevant, da dadurch Fokusverluste, Refokussierungen und weitere Ereignisse innerhalb des Trainings mit einem konkreten Zeitstempel versehen werden können.

5.5 Aufbau innerhalb der Szene

Da Forschende UPAT auf ihre eigenen Bedürfnisse anpassen sollen, müssen sowohl das gesamte Projekt als auch insbesondere die Szene, in der das Training stattfindet, klar strukturiert und übersichtlich aufgebaut sein. Gleichzeitig ist es wichtig, dass die Anwendung möglichst einfach heruntergeladen, geöffnet und ausgeführt werden kann, ohne dass umfangreiche Vorkonfigurationen erforderlich sind.

Aus diesem Grund wurde die Trainingsszene bewusst modular und logisch organisiert. Wie in Abbildung 24 zu sehen ist, sind die verschiedenen Bestandteile der Szene innerhalb der Unity-Hierarchie klar voneinander getrennt. Managerskripte, wie der *XR Interaction Manager*, der *Visual Delegate Manager*, der *Research Data Collector* oder der *TrainingTimer* befinden sich gesammelt innerhalb eigener entsprechend benannter Objekte innerhalb der Szene. Dadurch können Forschende relevante Systeme schnell finden und bearbeiten, ohne die gesamte Szene durchsuchen zu müssen.

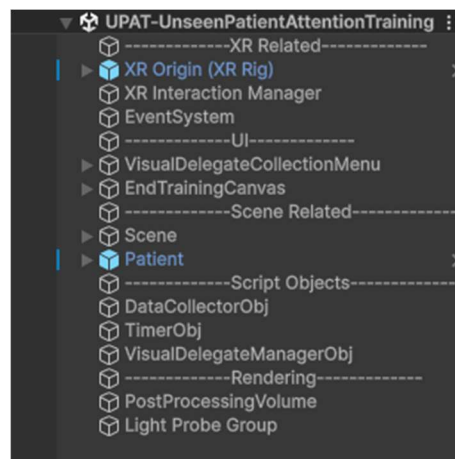


Abbildung 24: Aufbau der Trainingsszene in Unity. Jedes Manager skript hat ein eigenes Objekt in der Szene. Zudem wurde der Szenenaufbau durch Trennobjekte übersichtlich gehalten.

Auch die anderen Objekte in der Trainingsumgebung wurden strukturiert aufgebaut. Der virtuelle Patient, die Interaktionssysteme, die Menüs sowie die Post-Processing-Komponenten sind klar voneinander getrennt organisiert. Gleichzeitig wurde darauf geachtet, dass neue Komponenten oder zusätzliche Forschungsfeatures möglichst einfach ergänzt werden können, ohne bestehende Systeme grundlegend verändern zu müssen.

Auch die Ordnerstruktur im Projekt wurde bewusst übersichtlich und nachvollziehbar gestaltet. Die einzelnen Ordner sind entsprechend ihrem Inhalt klar benannt und voneinander getrennt, wodurch sich beispielsweise Modelle, Animationen, Texturen, Skripte oder Visual Delegates schnell auffinden lassen.

Darüber hinaus wurden die wichtigsten Systeme so vorbereitet, dass nach dem Klonen des Projekts und dem Öffnen in Unity keine umfangreiche Einrichtung erforderlich ist. Dadurch sollen die Einstiegshürde für Forschende möglichst gering gehalten und die Nutzung der Anwendung in zukünftigen Studien vereinfacht werden.

5.6 Berücksichtigung der Gestaltungsprobleme aus der Epicsave-Studie

Im Kapitel 2.2.1 wurden verschiedene Gestaltungsprobleme von Virtual-Reality-Trainings im medizinischen Kontext herausgearbeitet. Diese wurden bei der Entwicklung der vorliegenden Anwendung gezielt berücksichtigt und nach Möglichkeit vermieden.

So wurde beispielsweise vollständig auf künstliche Fortbewegung innerhalb der virtuellen Umgebung verzichtet. Stattdessen sitzt der virtuelle Avatar des Nutzers während des gesamten Trainings auf einem Stuhl. Da der Nutzer in der realen Umgebung während des Trainings in der Regel auch auf einem Stuhl sitzt, sollte dies das Auftreten von Cyber Sickness weitgehend verhindern.

Für die Interaktion in den Menüs werden Ray-Pointer verwendet, wodurch die Bedienung in der VR-Umgebung intuitiv ist. Darüber hinaus wird der Controller ausschließlich in den Menüoberflächen benötigt, da während des eigentlichen Trainings ausschließlich Eye-Tracking als Eingabemethode genutzt wird. Dadurch bleibt die Bedienung insgesamt bewusst einfach und möglichst natürlich gestaltet.

Zusätzlich wurde bei der Gestaltung der virtuellen Umgebung auf eine ausreichend realistische Darstellung des medizinischen Kontexts geachtet, ohne dabei einen unnötig hohen Fokus auf fotorealistische Grafikqualität zu legen. Dadurch soll eine immersive, zugleich technisch stabile und leicht verständliche Trainingsumgebung geschaffen werden.

Sämtliche zuvor herausgearbeiteten Gestaltungsprinzipien wurden in der Anwendung berücksichtigt und konsequent umgesetzt.

6 Diskussion

Im Rahmen dieser Arbeit wurde der bestehende Prototyp von UPAT konzeptionell und technisch erweitert, um ihn stärker auf eine zukünftige Nutzung im Forschungskontext auszurichten. Die Ergebnisse der technischen Erweiterung umfassen die modulare Visual-Delegate-Architektur, die Konfigurierbarkeit wichtiger Trainingsparameter über ein Startmenü sowie die automatisierte Speicherung relevanter Trainings- und Aufmerksamkeitsdaten.

In der folgenden Diskussion werden die im Rahmen dieser Arbeit erzielten Ergebnisse eingeordnet und mit den zuvor vorgestellten wissenschaftlichen Arbeiten in Beziehung gesetzt. Dabei wird untersucht, inwieweit die entwickelten Konzepte und technischen Lösungen mit den Erkenntnissen der bestehenden Forschung übereinstimmen. Darüber hinaus werden die bestehenden Limitationen der entwickelten Anwendung aufgezeigt sowie mögliche Ansatzpunkte für zukünftige Weiterentwicklungen und weiterführende Forschungsarbeiten diskutiert.

Die in der EPICSAVE-Studie beschriebenen Gestaltungsprobleme medizinischer VR-Trainings wurden bei der Entwicklung von UPAT gezielt berücksichtigt. So bleibt der

Nutzer während des gesamten Trainings in sitzender Position und muss sich nicht virtuell durch die Umgebung bewegen. Dadurch werden potenzielle Auslöser von Cyber Sickness reduziert. Auch die Interaktion wurde bewusst einfach gehalten. Während Controller lediglich zur Bedienung der Menüs benötigt werden, basiert das eigentliche Training vollständig auf Eye-Tracking. Diese Designentscheidungen entsprechen den in der EPICSAVE-Studie hervorgehobenen Anforderungen an hohe Präsenz und eine stabile Nutzerführung.

Gleichzeitig führt die Reduktion der Interaktionsmöglichkeiten dazu, dass die Trainingssituation vergleichsweise statisch wirkt. Eine solche Gesprächssituation ist zwar technisch leicht kontrollierbar und bietet gute Voraussetzungen für wissenschaftliche Untersuchungen, könnte jedoch die wahrgenommene Realitätsnähe des virtuellen Patienten verringern. Gerade im medizinischen Kontext ist die emotionale Wirkung des Gegenübers von besonderer Bedeutung, da sie maßgeblich beeinflussen kann, ob Nutzende die Situation als authentische Arzt-Patienten-Interaktion wahrnehmen. Zukünftig könnten zusätzliche Animationen, Gesten oder andere Verhaltensweisen des virtuellen Patienten dazu beitragen, die Immersion weiter zu steigern.

Auch die von Zielke et al. (2017) hervorgehobene Bedeutung virtueller Patienten und nonverbaler Kommunikation findet sich in UPAT wieder. Die Trainingslogik basiert wesentlich auf dem Aufrechterhalten von Blickkontakt und greift damit einen großen Aspekt nonverbaler Kommunikation auf, der in klassischen monitorbasierten Trainings nur eingeschränkt abgebildet werden kann. Gleichzeitig bleibt die nonverbale Kommunikation des virtuellen Patienten im aktuellen Entwicklungsstand begrenzt. Der Patient reagiert weder durch Mimik noch durch Körpersprache noch durch eigenes Blickverhalten auf die Handlungen des Nutzers. Stattdessen erfolgt das Feedback primär über Visual Delegates in Form visueller Effekte. Diese Lösung bietet zwar eine hohe Kontrollierbarkeit und eignet sich daher für wissenschaftliche Untersuchungen, wirkt jedoch weniger natürlich als eine direkte Verhaltensänderung des virtuellen Patienten. Im Rahmen der technischen Erweiterung wurde deshalb eine Schnittstelle geschaffen, über die künftig auch animationsbasierte Visual Delegates integriert werden können. Dadurch wäre es möglich, das Feedback stärker auf das Verhalten des virtuellen Patienten zu verlagern und die Natürlichkeit der Interaktion zu erhöhen.

Die im theoretischen Hintergrund vorgestellten Studien zu VR-basierten Aufmerksamkeitstrainings stützen grundsätzlich die Annahme, dass Virtual Reality für diesen Anwendungsbereich geeignet ist. Tarnig et al. (2022) konnten insbesondere positive Effekte in den Bereichen Focused Attention und Alternating Attention nachweisen. UPAT adressiert derzeit vor allem die Focused Attention, da die Aufmerksamkeit der Nutzenden über einen längeren Zeitraum auf den virtuellen Patienten gerichtet und dort gehalten werden soll. Aspekte der Alternating Attention werden hingegen bislang nur indirekt berücksichtigt. Die virtuelle Umgebung enthält zwar potenzielle Distraktoren, etwa den Computerbildschirm im Behandlungszimmer.

Diese sind jedoch noch nicht in die Trainingsaufgabe eingebunden. Eine zukünftige Erweiterung um gezielte Nebenaufgaben oder kontrollierte Ablenkungen könnte dazu beitragen, auch wechselnde Aufmerksamkeit realitätsnäher zu trainieren.

In diesem Zusammenhang ist auch die Analyse der Wildcard-Anwendung relevant. Die dort eingesetzte Primäraufgabe ist klar definiert, leicht verständlich und wurde im Rahmen der zugehörigen Studie empirisch evaluiert. Im Vergleich dazu weist die Primäraufgabe von UPAT zwar einen hohen Bezug zum medizinischen Anwendungsfall auf, bietet jedoch noch Potenzial für eine stärkere spielerische oder aufgabenorientierte Einbindung. Die Weiterentwicklung der Primäraufgabe stellt daher einen wichtigen Ansatzpunkt für zukünftige Arbeiten dar.

Die Überarbeitung der Visual-Delegate-Architektur ist aus softwaretechnischer Sicht eines der wichtigsten Ergebnisse der Arbeit. Durch die Einführung einer Basisklasse, die Nutzung von Scriptable Objects und die Möglichkeit, Visual Delegates in Collections zusammenzufassen, wurde die Anwendung deutlich erweiterbar. Dies ist im Forschungskontext besonders relevant, da unterschiedliche Visual Delegates gezielt getestet und miteinander verglichen werden sollen. Die Architektur schafft somit die Voraussetzungen für die Verwendung von UPAT in der Forschung.

Inhaltlich lässt sich der Einsatz von Visual Delegates in UPAT gut mit den Arbeiten von Misztal et al. (2020) sowie von Misztal und Schild (2022) begründen. Visual Delegates dienen dazu, nicht direkt wahrnehmbare Zustände visuell erfahrbar zu machen. In UPAT wird dieses Prinzip auf fehlende Aufmerksamkeit übertragen. Ein Aufmerksamkeitsverlust wird nicht nur als Messwert erfasst, sondern auch durch visuelle Effekte für den Nutzer erfahrbar gemacht.

Bei der Verwendung dieser Visual Delegates im Training könnte es allerdings auch zu Problemen kommen. So wäre es denkbar, dass Nutzende nicht langfristig eine aufmerksamere Haltung gegenüber dem virtuellen Patienten entwickeln, sondern primär lernen, auf das Auftreten der visuellen Effekte zu reagieren. In diesem Fall würde UPAT weniger die Aufmerksamkeit selbst trainieren als vielmehr ein Reaktionsverhalten auf bestimmte Visual Delegates. Um diesen Effekt zukünftig untersuchen zu können, wurde im Rahmen der technischen Erweiterung die Möglichkeit einer Baseline-Aufgabe integriert. Wie bereits in der Analyse der Anwendung VERA herausgearbeitet wurde, ermöglicht eine solche Baseline die Erfassung vergleichbarer Aufmerksamkeitsdaten ohne Einflussnahme durch Visual Delegates und schafft damit eine Grundlage für weiterführende wissenschaftliche Untersuchungen.

Die wichtigste Limitation dieser Arbeit besteht darin, dass keine empirische Evaluation der entwickelten Anwendung durchgeführt wurde. Die im theoretischen Hintergrund vorgestellten Studien liefern zwar eine Grundlage für die Konzeption von UPAT, ihre Ergebnisse lassen sich jedoch nicht unmittelbar auf die entwickelte Anwendung übertragen. Ebenso wurde bislang keine Nutzbarkeitsstudie durchgeführt. Aussagen über die tatsächliche Benutzerfreundlichkeit, die Akzeptanz der Anwendung oder die Wirksamkeit des Trainings können daher derzeit nicht getroffen werden.

Darüber hinaus befindet sich die Anwendung trotz der vorgenommenen technischen Erweiterungen weiterhin in einem frühen Entwicklungsstadium. Insbesondere die Primäraufgabe des Trainings sowie die Interaktionsmöglichkeiten des virtuellen Patienten bieten Potenzial für weitere Verbesserungen. Auch die Integration zusätzlicher Visual Delegates, animierter Rückmeldungen oder komplexerer Aufmerksamkeitsszenarien könnte die Realitätsnähe und den Trainingswert der Anwendung künftig erhöhen.

Zusammenfassend lässt sich festhalten, dass die im Rahmen dieser Arbeit entwickelte technische Erweiterung die Grundlage für den zukünftigen wissenschaftlichen Einsatz von UPAT geschaffen hat. Durch die modulare Visual-Delegate-Architektur, die Konfigurierbarkeit wichtiger Trainingsparameter sowie die automatisierte Erfassung relevanter Aufmerksamkeitsdaten wurden wichtige Voraussetzungen für die Durchführung wissenschaftlicher Studien geschaffen. Ein empirischer Nachweis der Wirksamkeit des Trainings steht jedoch noch aus und muss Gegenstand zukünftiger Forschungsarbeiten sein. Die vorliegende Arbeit stellt somit weniger den Abschluss der Entwicklung als vielmehr die Grundlage für die weitere wissenschaftliche Untersuchung und Weiterentwicklung von VR-basierten Aufmerksamkeitstrainings im medizinischen Kontext dar.

7 Fazit

Ziel dieser Bachelorarbeit war die konzeptionelle und technische Weiterentwicklung des im Rahmen eines Praxisprojekts entwickelten VR-basierten Aufmerksamkeitstrainings UPAT, um dieses gezielt auf den Einsatz im Forschungskontext vorzubereiten. Hierfür wurden zunächst die wissenschaftlichen Grundlagen zu VR-Trainingssimulationen, Aufmerksamkeitstrainings, Eye-Tracking sowie Visual Delegates untersucht. Ergänzend erfolgte eine Analyse vergleichbarer Anwendungen, um Anforderungen und Erfolgsfaktoren für wissenschaftlich nutzbare VR-Aufmerksamkeitstrainings zu identifizieren.

Auf Basis dieser Erkenntnisse wurde ein Konzept für die Weiterentwicklung von UPAT entwickelt und anschließend technisch umgesetzt. Zu den wichtigsten Ergebnissen der Arbeit zählen die Einführung einer modularen Visual-Delegate-Architektur, die Forschenden die einfache Erstellung und Konfiguration neuer Visual Delegates ermöglicht, die Entwicklung eines Startmenüs zur Anpassung relevanter Trainingsparameter sowie die Implementierung einer automatisierten Datenerfassung.

Darüber hinaus wurde die Anwendung so erweitert, dass Forschende unterschiedliche Trainingskonfigurationen und Visual-Delegate-Kombinationen ohne Änderungen am Quellcode testen können. Damit schafft die entwickelte Softwarearchitektur die Grundlage für zukünftige Forschungsarbeiten, in denen verschiedene Visual Delegates, Trainingsparameter oder Aufmerksamkeitsstrategien systematisch untersucht werden können.

Gleichzeitig zeigt die Arbeit auch die Grenzen des aktuellen Entwicklungsstandes auf. Obwohl UPAT nun die technischen Voraussetzungen für wissenschaftliche Studien erfüllt, wurde die Wirksamkeit des Trainings bislang nicht empirisch untersucht. Es kann daher derzeit keine Aussage darüber getroffen werden, ob das Training tatsächlich zu einer Verbesserung der Aufmerksamkeit in realen Arzt-Patienten-Interaktionen führt. Ebenso fehlt bislang eine Evaluation der Usability und Nutzerakzeptanz der Anwendung.

Für zukünftige Arbeiten ergeben sich daraus mehrere Ansatzpunkte. Insbesondere sollte die entwickelte Anwendung in kontrollierten Studien untersucht werden, um sowohl die Nutzbarkeit als auch die tatsächliche Wirksamkeit des Aufmerksamkeitstrainings zu evaluieren. Darüber hinaus bieten die entwickelte Visual-Delegate-Architektur und die vorhandenen Erweiterungsschnittstellen die Möglichkeit, neue Formen von Visual Delegates, beispielsweise animations- oder UI-basierte Rückmeldungen des virtuellen Patienten, zu integrieren und deren Einfluss auf das Nutzerverhalten zu untersuchen. Auch die Erweiterung der Primäraufgabe um komplexere Aufmerksamkeitsszenarien stellt einen vielversprechenden Ansatz für zukünftige Forschung dar.

Zusammenfassend wurde mit dieser Arbeit eine technische und konzeptionelle Grundlage geschaffen, auf deren Basis zukünftige wissenschaftliche Untersuchungen zu VR-basierten Aufmerksamkeitstrainings im medizinischen Kontext durchgeführt werden können. Die entwickelte Anwendung stellt damit keinen Abschluss der Forschung dar, sondern bildet vielmehr den Ausgangspunkt für die empirische Untersuchung eines bislang wenig erforschten Anwendungsfeldes.

Literaturverzeichnis

- Acid Nerve/Devolver Digital. (2021). *Death's Door* [PC]. https://store.steampowered.com/app/894020/Deaths_Door/
- Buttiglione, M. D. (2022). *Wildcard* [Pico Neo 3 Pro Eye]. <https://github.com/marco-buttiglione/Wildcard>
- Crytek. (2019). *Hunt: Showdown 1896* [PC]. <https://www.huntshowdown.com/>
- Delvigne, V. (2020). *VERA* [HTC-Vive Pro Eye]. <https://github.com/numediart/VERA>
- Delvigne, V., Ris, L., Dutoit, T., Wannous, H., & Vandeborre, J.-P. (2020). VERA: Virtual Environments Recording Attention. *2020 IEEE 8th International Conference on Serious Games and Applications for Health (SeGAH)*, 1–7. <https://doi.org/10.1109/SeGAH49190.2020.9201699>
- Hildenbrand, G. M., Perrault, E. K., & Rnoh, R. H. (2022). Patients' Perceptions of Health Care Providers' Dismissive Communication. *Health Promotion Practice*, 23(5), 777–784. <https://doi.org/10.1177/15248399211027540>
- Howard, M. C., Gutworth, M. B., & Jacobs, R. R. (2021). A meta-analysis of virtual reality training programs. *Computers in Human Behavior*, 121, 106808. <https://doi.org/10.1016/j.chb.2021.106808>
- Jagosh, J., Donald Boudreau, J., Steinert, Y., MacDonald, M. E., & Ingram, L. (2011). The importance of physician listening from the patients' perspective: Enhancing diagnosis, healing, and the doctor–patient relationship. *Patient Education and Counseling*, 85(3), 369–374. <https://doi.org/10.1016/j.pec.2011.01.028>
- Kyaw, B. M., Saxena, N., Posadzki, P., Vseteckova, J., Nikolaou, C. K., George, P. P., Divakar, U., Masiello, I., Kononowicz, A. A., Zary, N., & Car, L. T. (2019). Virtual Reality for Health Professions Education: Systematic Review and Meta-Analysis by the Digital Health Education Collaboration. *Journal of Medical Internet Research*, 21(1), e12959. <https://doi.org/10.2196/12959>
- Lampropoulos, G., Del Bosque, A., Fernández-Arias, P., & Vergara, D. (2025). Virtual Reality in Medical Education, Healthcare Education, and Nursing Education: An

- Overview. *Multimodal Technologies and Interaction*, 9(7), 75.
<https://doi.org/10.3390/mti9070075>
- Lin, W., Chen, L., Xiong, W., Ran, K., & Fan, A. (2025). Measuring the sense of presence and learning efficacy in immersive virtual assembly training. *International Journal of Mechanical Engineering Education*, 03064190241308659.
<https://doi.org/10.1177/03064190241308659>
- Merchant, Z., Goetz, E. T., Cifuentes, L., Keeney-Kennicutt, W., & Davis, T. J. (2014). Effectiveness of virtual reality-based instruction on students' learning outcomes in K-12 and higher education: A meta-analysis. *Computers & Education*, 70, 29–40. <https://doi.org/10.1016/j.compedu.2013.07.033>
- Misztal, S., Carbonell, G., & Schild, J. (2022). Experiencing Age-Related Movement Impairment Through Visual Delegation in VR Can Substitute Haptic Impairments of an Age Simulation Suit. *2022 IEEE 10th International Conference on Serious Games and Applications for Health(SeGAH)*, 1–8.
<https://doi.org/10.1109/SEGAH54908.2022.9978590>
- Misztal, S., & Schild, J. (2022). Visual Delegate Generalization Frame – Evaluating Impact of Visual Effects and Elements on Player and User Experiences in Video Games and Interactive Virtual Environments. *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, CHI '22*, 1–20.
<https://doi.org/10.1145/3491102.3501885>
- Patti, A., Vona, F., Barberio, A., Buttiglione, M. D., Crusco, I., Mores, M., & Garzotto, F. (2024). *Training Attention Skills in Individuals with Neurodevelopmental Disorders using Virtual Reality and Eye-tracking technology* (arXiv:2404.15960). arXiv.
<https://doi.org/10.48550/arXiv.2404.15960>
- Savickaite, S., Parker, A., & Millington, E. (2021). *Dimensional Search: Visual Search and Working Memory VR Experiment* [PC, HTC-Vive]. <https://github.com/sarune-savickaiteexeter/Working-Memory-VR-Experiment>

- Scorgie, D., Feng, Z., Paes, D., Parisi, F., Yiu, T. W., & Lovreglio, R. (2024). Virtual reality for safety training: A systematic literature review and meta-analysis. *Safety Science*, 171, 106372. <https://doi.org/10.1016/j.ssci.2023.106372>
- Selaskowski, B., Asché, L. M., Wiebe, A., Kannen, K., Aslan, B., Gerding, T. M., Sanchez, D., Ettinger, U., Kölle, M., Lux, S., Philipsen, A., & Braun, N. (2023). Gaze-based attention refocusing training in virtual reality for adult attention-deficit/hyperactivity disorder. *BMC Psychiatry*, 23(1), 74. <https://doi.org/10.1186/s12888-023-04551-z>
- Singh Ospina, N., Phillips, K. A., Rodriguez-Gutierrez, R., Castaneda-Guarderas, A., Gionfriddo, M. R., Branda, M. E., & Montori, V. M. (2019). Eliciting the Patient's Agenda- Secondary Analysis of Recorded Clinical Encounters. *Journal of General Internal Medicine*, 34(1), 36–40. <https://doi.org/10.1007/s11606-018-4540-5>
- Sung, H., Kim, M., Park, J., Shin, N., & Han, Y. (2024). Effectiveness of Virtual Reality in Healthcare Education: Systematic Review and Meta-Analysis. *Sustainability*, 16(19), 8520. <https://doi.org/10.3390/su16198520>
- Tarng, W., Pan, I.-C., & Ou, K.-L. (2022). Effectiveness of Virtual Reality on Attention Training for Elementary School Students. *Systems*, 10(4), 104. <https://doi.org/10.3390/systems10040104>
- Treyarch/Activision. (2012). *Call of Duty: Black Ops 2* [PC]. <https://www.activision.com/games/call-of-duty/call-of-duty-black-ops-2>
- Zielke, M. A., Zakhidov, D., Hardee, G., Evans, L., Lenox, S., Orr, N., Fino, D., & Mathialagan, G. (2017). Developing Virtual Patients with VR/AR for a natural user interface in medical teaching. *2017 IEEE 5th International Conference on Serious Games and Applications for Health (SeGAH)*, 1–8. <https://doi.org/10.1109/SeGAH.2017.7939285>

Anhang

Anhang A.1 – User Stories

Die verwendeten User-Stories, die während der Konzeptionierungsphase des Projekts erstellt wurden.

https://www.notion.so/User-Stories-07f23d7a4f5642ee9a3c0051a4ad1222?source=copy_link#330e77a7c0ab800e9e83c070824e52ed

Anhang A.2 - Quellcode

Der vollständige Quellcode der Anwendung UPAT kann im zugehörigen Git-Repository eingesehen werden.

<https://github.com/floriankrawinkel/UPAT>

Anhang A.3 – Beispielhafte CSV-Datei

Die folgende CSV-Datei zeigt beispielhaft die während eines Trainingsdurchlaufs aufgezeichneten Forschungsdaten.

```
VisualDelegates
DelegateName,Depth,Abstraction,Control,FadeIn,FadeOut
Grey Filter,FIL,SYM,ACT,3,1
Film Grain,FIL,SYM,ACT,3,1
Vignette,FIL,NAT,ACT,3,1

Events
FocusLossCount,FocusLossStartTime,FocusLossEndTime, RefocusTime,TimeBetweenFocusLosses
1,8.793,14.506,5.710,0.000
2,16.153,23.786,7.629,7.359
3,26.673,31.872,5.199,10.521

Summary
TimeInTraining,OverallFocusLossCount,AverageRefocusTime
37.040,3,6.179
```

Erklärung

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer oder der Verfasserin/des Verfassers selbst entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

Köln, 02.06.2026

Ort, Datum

Unterschrift